

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO
ĐỒ ÁN CHUYÊN NGÀNH

Thiết kế và hiện thực một
gateway IoT dạng mô-đun có
khả năng cấu hình

NGÀNH: KỸ THUẬT MÁY TÍNH

HỘI ĐỒNG:
GVHD:

—o0o—

SVTH 1: Hồng Thiện Nhân (2111900)
SVTH 2: Trần Hải Thảo Quảng (2212767)

TP. HỒ CHÍ MINH, THÁNG 12 NĂM 2025

Lời cam đoan

Nhóm chúng em xin cam đoan rằng đồ án với đề tài “Thiết kế và hiện thực một gateway IoT dạng mô-đun có khả năng cấu hình” là công trình nghiên cứu do chính nhóm thực hiện dưới sự hướng dẫn của giảng viên hướng dẫn và người đồng hướng dẫn.

Toàn bộ nội dung, số liệu và kết quả trình bày trong đồ án đều do nhóm tự nghiên cứu, thiết kế và triển khai, không sao chép hay kế thừa từ bất kỳ đồ án hoặc khóa luận nào đã được thực hiện trước đó. Các kiến thức nền tảng và tài liệu tham khảo (nếu có) chỉ được sử dụng nhằm phục vụ cho việc tìm hiểu lý thuyết và đều có nguồn gốc rõ ràng.

Nhóm chúng em xin hoàn toàn chịu trách nhiệm trước Nhà trường và Khoa về tính trung thực của nội dung đồ án này.

Lời cảm ơn

Nhóm chúng em xin bày tỏ lòng biết ơn chân thành đến quý Thầy/Cô giảng viên Khoa Khoa học và Kỹ thuật Máy tính, Trường Đại học Bách Khoa – Đại học Quốc gia Thành phố Hồ Chí Minh, những người đã trang bị cho chúng em nền tảng kiến thức chuyên môn vững chắc và tư duy kỹ thuật cần thiết trong suốt quá trình học tập.

Nhóm chúng em xin gửi lời cảm ơn sâu sắc đến ThS. Huỳnh Phúc Nghi, giảng viên hướng dẫn, người đã trực tiếp định hướng, theo sát và tận tình góp ý cho nhóm trong suốt quá trình phân tích yêu cầu, thiết kế hệ thống và triển khai đồ án. Những hướng dẫn sát thực tế, định hướng kỹ thuật rõ ràng cùng kinh nghiệm triển khai hệ thống của thầy đã giúp nhóm tiếp cận đề tài theo đúng hướng của một đồ án kỹ sư, đảm bảo tính ứng dụng và khả thi.

Bên cạnh đó, nhóm chúng em xin trân trọng cảm ơn ThS. Huỳnh Hoàng Kha, người đồng hướng dẫn, đã hỗ trợ nhóm trong quá trình tìm hiểu công nghệ, lựa chọn giải pháp phần cứng – phần mềm, cũng như đóng góp nhiều ý kiến chuyên môn quý báu trong quá trình hiện thực và hoàn thiện hệ thống.

Nhóm chúng em cũng xin trân trọng cảm ơn Trường Đại học Bách Khoa – Đại học Quốc gia Thành phố Hồ Chí Minh và Khoa Khoa học và Kỹ thuật Máy tính đã tạo điều kiện thuận lợi về cơ sở vật chất, tài liệu và môi trường học tập để nhóm có thể triển khai đồ án một cách hiệu quả.

Cuối cùng, nhóm chúng em xin gửi lời cảm ơn đến gia đình, bạn bè và những người thân đã luôn động viên, hỗ trợ về mặt tinh thần trong suốt quá trình thực hiện đồ án.

Mặc dù nhóm đã nỗ lực hoàn thành đồ án theo đúng mục tiêu đề ra, do giới hạn về thời gian và nguồn lực, đồ án không tránh khỏi những thiếu sót. Nhóm chúng em kính mong nhận được sự góp ý và chỉ dẫn từ quý Thầy/Cô để sản phẩm có thể tiếp tục được cải tiến và hoàn thiện hơn.

Tóm tắt

Nội dung của đề án hướng đến việc nghiên cứu, thiết kế và hiện thực một IoT gateway dạng mô-đun có khả năng khả cấu hình linh hoạt, nhằm giải quyết các hạn chế của các giải pháp gateway truyền thống trong các hệ thống IoT hiện nay. Trong bối cảnh mỗi kịch bản ứng dụng IoT đòi hỏi tập hợp giao thức và chuẩn giao tiếp khác nhau, việc phát triển các gateway tích hợp sẵn nhiều chức năng không cần thiết dẫn đến lãng phí tài nguyên và làm tăng chi phí triển khai.

Trong giai đoạn 1, nhóm tập trung tìm hiểu các khái niệm nền tảng liên quan đến IoT gateway, làm rõ vai trò của gateway trong hạ tầng IoT, đồng thời phân tích các công nghệ, giao thức và chuẩn giao tiếp phổ biến đang được sử dụng. Trên cơ sở đó, một kiến trúc gateway định hướng mô-đun được đề xuất, trong đó phần cứng được thiết kế theo dạng lắp ghép nhằm cho phép lựa chọn các module phù hợp với từng kịch bản ứng dụng, trong khi phần mềm hỗ trợ quy trình cấu hình tối giản và thân thiện với người dùng.

Trong giai đoạn hiện thực, đề tài tiến hành xây dựng nguyên mẫu (prototype) của hệ thống nhằm kiểm chứng tính khả thi của kiến trúc đề xuất. Các thử nghiệm ban đầu cho thấy hệ thống hoạt động ổn định, đáp ứng được các yêu cầu chức năng cơ bản và thể hiện tiềm năng tối ưu hóa chi phí triển khai so với các giải pháp tích hợp truyền thống. Kết quả đạt được tạo nền tảng cho việc tiếp tục hoàn thiện và mở rộng hệ thống trong các giai đoạn phát triển sản phẩm tiếp theo, hướng tới một IoT gateway có khả năng ứng dụng trong thực tế.

Mục lục

1	Phần Mở Đầu	11
1.1	Nội dung Đề tài	11
1.2	Mục tiêu Giai đoạn 1 (Đồ án Chuyên ngành)	11
1.3	Mục tiêu Giai đoạn 2 (Đồ án Tốt nghiệp)	12
2	Tổng quan Đồ án	13
2.1	Lý do chọn đề tài	13
2.2	Mục tiêu Đồ án	13
2.3	Phạm vi Đồ án	14
2.4	Phương pháp thực hiện	14
3	Cơ sở Lý thuyết	16
3.1	Tổng quan về Lĩnh vực	16
3.1.1	Khái niệm Internet vạn vật (IoT)	16
3.1.2	Vị trí và vai trò của IoT gateway	17
3.2	Cơ sở lý thuyết về IoT Gateway	17
3.2.1	Khái niệm IoT gateway	17
3.2.2	Vị trí của IoT gateway trong kiến trúc IoT	18
3.2.3	Các vai trò chính của IoT gateway	18
3.2.4	Cấu trúc của IoT gateway	20
3.2.4.1	Kiến trúc phần cứng	20
3.2.4.2	Kiến trúc phần mềm	21
3.2.5	Nguyên lý hoạt động cơ bản của IoT Gateway	22
3.3	Phân tích các sản phẩm trên thị trường	24
3.3.1	Các công nghệ kết nối phổ biến	24
3.3.2	Các giao thức truyền dữ liệu	24
3.3.3	Các mô hình triển khai gateway	24
3.3.4	Các tính năng quan trọng của gateway trên thị trường	25
3.3.5	Ví dụ về các gateway thực tế trên thị trường	26
3.3.6	Xu hướng phát triển hiện tại	28
3.4	Cơ sở lý thuyết về Thiết kế phần cứng	29
3.4.1	Thiết kế High-Speed PCB	29
3.4.1.1	Khái niệm High-Speed PCB Design	29
3.4.1.2	Tính toán vận tín hiệu (Signal Integrity) và đường hồi dòng (Return Path)	29
3.4.1.3	Kiểm soát trở kháng	31
3.4.1.4	Quy tắc bố trí để giảm nhiễu, crosstalk và cải thiện EMI	33

3.4.2	Power Distribution Network – PDN	35
4	Phân tích và Thiết kế	37
4.1	Phân tích yêu cầu	37
4.1.1	Yêu cầu chức năng	37
4.1.2	Yêu cầu phi chức năng	38
4.1.3	Các ràng buộc thiết kế	39
4.2	Giải pháp đề xuất	39
4.2.1	Giải pháp phần cứng	40
4.2.2	Giải pháp phần mềm	40
4.2.2.1	Kiến trúc phần mềm tổng thể	41
4.2.2.2	Giải pháp phần mềm cho Master WAN-MCU	41
4.2.2.3	Giải pháp phần mềm cho MCU LAN (Slave LAN-MCU)	42
4.2.2.4	Cơ chế cấu hình và cập nhật	43
4.2.2.5	Đánh giá giải pháp phần mềm	43
4.3	Đặc tả sản phẩm	44
4.3.1	Thông số phần cứng và Hiệu năng xử lý	44
4.3.2	Khả năng kết nối và Mở rộng mô-đun	44
4.3.3	Đặc tả Hệ thống nguồn và Bảo vệ	45
4.3.4	Giao diện người dùng và Đặc điểm cơ khí	45
4.3.5	Đặc tả Phần mềm	45
4.4	Khảo sát và Thử nghiệm	45
4.4.1	Khảo sát các mô hình kiến trúc Gateway IoT phổ biến	45
4.4.2	Khảo sát mức độ module hóa phần cứng và khả năng mở rộng giao thức	46
4.4.3	Khảo sát kiến trúc phần mềm gateway: monolithic vs phân lớp/module	47
4.4.4	Khảo sát cơ chế cấu hình và cập nhật firmware trong triển khai thực tế	47
4.5	Thiết kế	48
4.5.1	Thiết kế Kiến trúc Hệ thống (System Architecture Design)	48
4.5.1.1	Sơ đồ khối Hệ thống	48
4.5.1.2	Mô tả thành phần hệ thống	49
4.5.1.3	Thiết kế Module hóa	50
4.5.1.4	Xử lý dữ liệu	52
4.5.2	Thiết kế phần cứng PCB (PCB Hardware Design)	57
4.5.2.1	Thiết kế sơ đồ khối phần cứng	57
4.5.2.2	Thiết kế Mạch nguyên lý (Schematic Design)	59
4.5.2.3	Thiết kế Layout (Layout Design)	72
4.5.2.4	Phân tích và Mô phỏng Tín hiệu (Signal Integrity Simulation)	83
4.5.3	Firmware và Software	91
5	Kịch bản kiểm thử - Quy trình Board Bring-up và Verification	100
5.1	Quy trình và Kết quả Khởi động Bo mạch	100
5.1.1	Kiểm tra ngoại quan (Visual Inspection)	100
5.1.2	Kiểm tra thông số nguội (Cold Check Analysis)	101
5.1.3	Kiểm tra cấp nguồn giới hạn (Smoke Test / Hot Check)	102
5.1.4	Xác thực thông số điện áp	103
5.2	Quy trình Kiểm tra sản phẩm	103
5.2.1	Kiểm tra trạng thái Logic và Điều khiển (Logic State Verification)	104
5.2.2	Kiểm tra Chất lượng Nguồn (Power Integrity Analysis)	104
5.2.3	Kiểm tra Chức năng Phần cứng (Functional Verification)	105

5.2.4	Kiểm tra Firmware và Software	107
5.2.4.1	Mục tiêu	108
5.2.4.2	Quy trình nạp và khởi động firmware	108
5.2.4.3	Nội dung kiểm thử firmware (theo nhóm chức năng)	108
6	Triển khai và Vận hành Hệ thống	113
6.1	Triển khai Lắp ráp Phần cứng	113
6.2	Khởi động và Kích hoạt Phần mềm	115
6.2.1	Khởi động hệ thống (Boot & RTOS Init)	115
6.2.2	Nạp và áp dụng cấu hình vận hành (Load/Apply Configuration)	115
6.2.3	Thiết lập kết nối WAN và đồng bộ thời gian	115
6.2.4	Kết nối Cloud và vòng lặp vận hành dữ liệu	115
6.2.5	Kích hoạt cấu hình tại chỗ (Local Configuration)	116
6.2.6	Cập nhật Firmware (FOTA)	116
6.3	Kịch bản Tương tác Người dùng	116
6.3.1	Cấu hình ban đầu bằng Ứng dụng PC	116
6.3.2	Giám sát vận hành qua Giao diện Web (Dashboard/Status)	117
6.3.3	Bảo trì và Cập nhật từ xa (OTA)	117
6.4	Kết quả Triển khai Sản phẩm	118
7	Kết luận	119
7.1	Tổng kết Giai đoạn 1	119
7.2	Thách thức	119
7.3	Hướng phát triển trong giai đoạn tiếp theo	119
7.3.1	Hoàn thiện và khắc phục các hạn chế thiết kế	120
7.3.2	Nâng cấp và Mở rộng năng lực Phần cứng	120
7.3.3	Phát triển Firmware chuyên sâu và tối ưu hóa hiệu suất	120
7.3.4	Tối ưu hóa chi phí và Hướng tới sản xuất	121
8	Phụ lục	122
8.1	Tài liệu tham khảo	122
8.2	Mã nguồn/Tài liệu thiết kế	124

Danh sách bảng

3.1	Các giá trị trở kháng đặc trưng theo các chuẩn giao thức truyền thông phổ biến.	31
4.1	Pinout Module Adapter	51
5.1	Kết quả kiểm tra ngoại quan.	101
5.2	Kết quả kiểm tra trở kháng nguội các đường nguồn.	102
5.3	Kết quả kiểm tra nóng và dòng tiêu thụ tĩnh.	103
5.4	Kết quả đo kiểm điện áp.	103
5.5	Kết quả kiểm tra các mức Logic điều khiển.	104
5.6	Kết quả kiểm tra chất lượng nguồn điện.	105
5.7	Kết quả kiểm tra chức năng.	107
5.8	Kiểm thử Scan/Read cấu hình	110
5.9	Kiểm thử ghi cấu hình Internet (internet_type) và tham số chung	110
5.10	Kiểm thử Wi-Fi (Personal/Enterprise)	110
5.11	Kiểm thử LTE và PPP	111
5.12	Kiểm thử MQTT (broker/topic/telemetry/downlink)	111
5.13	Kiểm thử cấu hình LoRa TDMA	111
5.14	Kiểm thử cấu hình CAN	111
5.15	Kiểm thử RS 485/Modbus (raw frame)	111
5.16	Kiểm thử firmware (tổng hợp)	112

Danh sách hình vẽ

3.1	Hạ tầng Internet vạn vật (IoT)	16
3.2	Vị trí của IoT gateway trong kiến trúc IoT	18
3.3	Cấu trúc của IoT gateway	20
3.4	Minh hoạ hiện tượng ringing do không khớp trở kháng	29
3.5	Minh hoạ return path liên tục dưới trace trên mặt phẳng tham chiếu	30
3.6	Minh hoạ một số cách định tuyến trace trên PCB ảnh hưởng trở kháng	33
3.7	Minh hoạ hiện tượng nhiễu xuyên kênh giữa các đường truyền đặt gần nhau	34
3.8	Các phần của PDN được phân tách theo dải tần số mà chúng ảnh hưởng	36
4.1	Sơ đồ khối Hệ thống	48
4.2	Sơ đồ chân Module Adapter	52
4.3	Sequence diagram: luồng xử lý dữ liệu end-to-end uplink	53
4.4	Sequence diagram: luồng xử lý dữ liệu end-to-end downlink	54
4.5	Sequence diagram: giao tiếp LAN-MCU ↔ WAN-MCU qua SPI	55
4.6	Sequence diagram: cập nhật cấu hình từ App PC	56
4.7	Sơ đồ khối hệ thống	57
4.8	Sơ đồ hệ thống nguồn	58
4.9	Sơ đồ nguyên lý hệ thống	60
4.10	Sơ đồ nguyên lý ESP32 Slave	60
4.11	Sơ đồ nguyên lý ESP32 Master	62
4.12	Sơ đồ nguyên lý mạch nguồn 1	63
4.13	Sơ đồ nguyên lý mạch nguồn 2	64
4.14	Sơ đồ nguyên lý mạch USB	65
4.15	Sơ đồ nguyên lý mạch USB Switch	65
4.16	Sơ đồ nguyên lý mạch IO Expanders (Master)	66
4.17	Khuyến nghị cấp nguồn cho IC TCA6424ARGJR từ datasheet	67
4.18	Kết quả mô phỏng mạch delay khởi động nguồn VCCI cho IO Expander	68
4.19	Sơ đồ nguyên lý mạch RTC	69
4.20	Sơ đồ nguyên lý mạch Micro SD Card	69
4.21	Sơ đồ nguyên lý mạch WAN	70
4.22	Sơ đồ nguyên lý mạch WAN Adapter	71
4.23	Sơ đồ nguyên lý mạch LAN Adapter	72
4.24	Layer Stack-up Management	73
4.25	Thông số kích thước cho đường tín hiệu Coplanar Single-ended 50Ω (Đơn vị: mm)	74
4.26	Thông số kích thước cho cặp tín hiệu vi sai (Coplanar Differential-pair) 90Ω (Đơn vị: mm)	74
4.27	Cấu trúc Stack-up và thông số vật liệu tiêu chuẩn của JLCPCB	74
4.28	Sơ đồ quy hoạch phân vùng chức năng trên PCB (Floor-planning)	75

4.29	Kỹ thuật tạo vòng bảo vệ (Guard Ring) và via stitching xung quanh khối thạch anh để cô lập nhiễu.	76
4.30	Kỹ thuật bọc mass và stitching via dọc theo đường tín hiệu USB tốc độ cao. . .	77
4.31	Minh họa tín hiệu Slave_I2C0 được định tuyến liên tục trên mặt phẳng GND đồng nhất, tránh các vị trí lớp phủ đã bị cắt.	77
4.32	Vị trí các tụ điện trong nguồn xung +5V0_SYS.	78
4.33	Điểm chuyển mạch trong nguồn xung +5V0_SYS.	79
4.34	Định tuyến hồi tiếp trong nguồn xung +5V0_SYS.	79
4.35	Tản nhiệt trong nguồn xung +5V0_SYS.	80
4.36	Mạng phân phối nguồn +3V3.	81
4.37	Định tuyến USB.	81
4.38	Định tuyến SDIO giữa ESP32-S3 Slave với Micro SD Card.	82
4.39	Biểu đồ phân bố trở kháng (Impedance Heatmap) đường tín hiệu đơn cực. . . .	83
4.40	Biểu đồ phân bố độ trễ lan truyền (Propagation Delay) đường tín hiệu đơn cực. .	83
4.41	Bảng tổng hợp thông số mô phỏng đường tín hiệu đơn cực.	84
4.42	Biểu đồ phân bố trở kháng (Impedance Heatmap) đường vi sai.	84
4.43	Biểu đồ phân bố độ trễ lan truyền (Propagation Delay) đường vi sai.	84
4.44	Bảng tổng hợp thông số mô phỏng đường vi sai.	85
4.45	Cấu trúc mạng phân phối nguồn.	85
4.46	Biểu đồ phân bố điện áp (Voltage Drop Heatmap) trên nguồn +5V0_SYS. . . .	86
4.47	Biểu đồ phân bố điện áp (Voltage Drop Heatmap) trên nguồn +5V0.	87
4.48	Biểu đồ phân bố điện áp (Voltage Drop Heatmap) trên nguồn +3V3.	87
4.49	Biểu đồ phân bố mật độ dòng điện (Current Density Heatmap) trên nguồn +5V0_SYS.	88
4.50	Mật độ dòng điện cao đột biến tại pinout nguồn xung +5V0_SYS.	89
4.51	Biểu đồ phân bố mật độ dòng điện (Current Density Heatmap) trên nguồn +5V0. .	89
4.52	Biểu đồ phân bố mật độ dòng điện (Current Density Heatmap) trên nguồn +3V3. .	90
4.53	Mật độ dòng điện cao đột biến tại pinout nguồn xung +3V3.	90
4.54	MCU LAN Firmware	92
4.55	MCU WAN Firmware	93
4.56	Lưu đồ giải thuật Data Communication Handler – giao tiếp cấu hình qua USB/UART	97
4.57	Lưu đồ giải thuật FOTA cho Dual-MCU Gateway	98
6.1	Assembly Diagram.	113
6.2	Mô hình 3D kết nối thiết bị.	114
6.3	Hình ảnh sản phẩm thực tế.	114
6.4	App config trên PC	117

Chương 1

Phần Mở Đầu

1.1 Nội dung Đề tài

IoT gateway là một trong những thành phần cốt lõi của hầu hết các hệ thống IoT hiện nay, đóng vai trò cầu nối giữa các giao thức và chuẩn giao tiếp khác nhau. Mỗi ứng dụng cụ thể lại đòi hỏi tập hợp giao thức và chuẩn giao tiếp riêng; vì vậy, mỗi kịch bản sử dụng khác nhau yêu cầu IoT gateway hỗ trợ các giao thức và chuẩn giao tiếp tương ứng.

Tuy nhiên, việc thiết kế lại từ đầu IoT gateway cho từng trường hợp riêng lẻ rất tốn kém về thời gian và nhân lực. Để giảm chi phí này, nhiều thiết bị trên thị trường được phát triển theo hướng tích hợp sẵn một dải rộng các giao thức và chuẩn giao tiếp phổ biến. Cách tiếp cận đó giúp đáp ứng nhu cầu đa dạng, nhưng việc duy trì các giao thức và chuẩn giao tiếp không sử dụng lại gây lãng phí và tăng chi phí triển khai.

Nhằm giải quyết hạn chế trên, đề án đặt mục tiêu khảo sát, thiết kế, và hiện thực một IoT gateway có khả năng cấu hình linh hoạt, tập trung vào hai đặc tính:

1. **Modular** – Phần cứng thiết kế dạng lắp ghép, chỉ gắn các module cần thiết, qua đó loại bỏ phần dư thừa và tối ưu chi phí.
2. **Configurability** – Quy trình cấu hình tối giản, thân thiện với người dùng, giúp rút ngắn thời gian triển khai và giảm yêu cầu kỹ năng chuyên sâu.

Nhờ kết hợp hai đặc tính này, mô hình IoT gateway đề xuất không chỉ đáp ứng nhu cầu linh hoạt của các hệ thống IoT mà còn tối ưu hóa tổng chi phí sở hữu và vận hành.

1.2 Mục tiêu Giai đoạn 1 (Đề án Chuyên ngành)

Trong giai đoạn này, nhóm sinh viên chủ yếu tập trung tìm hiểu các định nghĩa, khái niệm liên quan; tìm hiểu các công nghệ, chuẩn giao tiếp, và các giao thức được sử dụng phổ biến trong các hạ tầng IoT hiện nay, từ đó đưa ra đặc tả chi tiết cho sản phẩm rồi thiết kế ở mức độ khái niệm (proof of concept), và hiện thực ở mức độ nguyên mẫu (prototype).

Nhiệm vụ (yêu cầu về nội dung và số liệu ban đầu):

1. Tìm hiểu:
 - (a) Định nghĩa, khái niệm của IoT gateway.

- (b) Vị trí và vai trò của IoT gateway trong hạ tầng IoT.
 - (c) Cấu trúc và nguyên lý hoạt động cơ bản của các IoT gateway.
 - (d) Các công nghệ, giao thức, và chuẩn giao tiếp được sử dụng phổ biến trong hạ tầng IoT.
 - (e) Các thiết bị IoT gateway khả cấu hình đã có trên thị trường, tập trung vào các chức năng, đặc điểm nổi bật, công nghệ, giao thức, và các chuẩn giao tiếp được hỗ trợ.
2. Thiết kế khái niệm và đặc tả chi tiết cho sản phẩm sẽ thực hiện.
 3. Khảo sát và thử nghiệm các kỹ thuật hiện thực sản phẩm, phác thảo các sơ đồ khối và sơ đồ nguyên lý.
 4. Hiện thực nguyên mẫu. Lưu ý, giai đoạn này sinh viên chưa cần làm ra sản phẩm hoàn chỉnh, chỉ cần demo được ý tưởng và chức năng cơ bản của phần cứng và phần mềm.

1.3 Mục tiêu Giai đoạn 2 (Đồ án Tốt nghiệp)

Đây là giai đoạn mà nhóm sinh viên sẽ tiến hành hoàn thiện sản phẩm dựa trên những kết quả ban đầu đã đạt được ở giai đoạn trước, bao gồm việc tích hợp các thành phần lên mạch in và hoàn thiện hệ thống phần mềm (firmware, software).

Nhiệm vụ (yêu cầu về nội dung và số liệu ban đầu):

1. Phân tích ưu nhược điểm của phiên bản đang có, hiệu chỉnh lại đặc tả chi tiết cho sản phẩm, và lên kế hoạch chi tiết cho quá trình hoàn thiện sản phẩm.
2. Thiết kế, hiện thực, và gỡ lỗi mạch in.
3. Lập trình phần mềm (firmware, software).
4. Kiểm thử chức năng và hiệu năng hệ thống.

Chương 2

Tổng quan Đề án

2.1 Lý do chọn đề tài

Trong các hệ thống Internet of Things (IoT) hiện nay, IoT gateway giữ vai trò trung tâm trong việc kết nối các thiết bị đầu cuối với hạ tầng mạng và nền tảng đám mây. Gateway không chỉ thực hiện chức năng chuyển tiếp dữ liệu mà còn đảm nhiệm các tác vụ quan trọng như chuyển đổi giao thức, xử lý dữ liệu tại biên, quản lý thiết bị và thực thi các cơ chế bảo mật.

Tuy nhiên, thực tế triển khai cho thấy mỗi hệ thống IoT lại có yêu cầu khác nhau về loại thiết bị, chuẩn giao tiếp và giao thức truyền thông. Việc thiết kế hoặc lựa chọn một gateway cố định, tích hợp sẵn nhiều giao thức không cần thiết dẫn đến lãng phí tài nguyên, tăng chi phí phần cứng và giảm tính linh hoạt khi mở rộng hoặc thay đổi thành phần của hệ thống.

Mặt khác, các gateway thương mại hiện nay thường đi theo hai xu hướng: hoặc tích hợp quá nhiều chức năng dẫn đến giá thành cao và phức tạp trong vận hành, hoặc quá đơn giản, khó mở rộng và không đáp ứng được các bài toán IoT đa dạng trong thực tế. Ngoài ra, việc cấu hình gateway trong nhiều trường hợp vẫn phụ thuộc nhiều vào việc chỉnh sửa firmware, gây khó khăn cho quá trình triển khai và bảo trì tại hiện trường.

Xuất phát từ những vấn đề trên, nhóm chúng em lựa chọn đề tài “Thiết kế và hiện thực một gateway IoT dạng mô-đun có khả năng cấu hình” với định hướng xây dựng một giải pháp kỹ thuật linh hoạt, có thể tùy biến theo nhu cầu sử dụng, tối ưu chi phí và phù hợp với các kịch bản triển khai IoT thực tế.

2.2 Mục tiêu Đề án

Mục tiêu tổng quát của đề án là thiết kế và hiện thực một mô hình IoT gateway, đáp ứng yêu cầu linh hoạt, khả cấu hình và dễ triển khai trong thực tế.

Cụ thể, đề án hướng tới các mục tiêu sau:

- Thiết kế một kiến trúc IoT gateway dạng mô-đun, cho phép lắp ghép và thay thế các module phần cứng theo từng kịch bản sử dụng cụ thể.
- Xây dựng giải pháp phần cứng và phần mềm phù hợp để hỗ trợ nhiều chuẩn giao tiếp và giao thức phổ biến trong hệ thống IoT.

- Hiện thực nguyên mẫu (prototype) gateway nhằm kiểm chứng tính khả thi của kiến trúc đề xuất.
- Xây dựng cơ chế cấu hình linh hoạt, cho phép thay đổi tham số hệ thống và lựa chọn module mà không cần chỉnh sửa hoặc biên dịch lại firmware.
- Đảm bảo hệ thống hoạt động ổn định, dễ mở rộng và phù hợp cho các bài toán triển khai IoT ở quy mô nhỏ đến trung bình.

Thông qua đề án, nhóm chúng em hướng tới việc áp dụng kiến thức đã học vào bài toán thiết kế – hiện thực một hệ thống kỹ thuật hoàn chỉnh, có tính ứng dụng cao.

2.3 Phạm vi Đề án

Trong khuôn khổ đề án, nhóm chúng em tập trung vào các nội dung sau:

- Phân tích vai trò và yêu cầu kỹ thuật của IoT gateway trong hạ tầng IoT.
- Thiết kế kiến trúc tổng thể của gateway theo hướng mô-đun, bao gồm cả phần cứng và phần mềm.
- Lựa chọn và tích hợp các chuẩn giao tiếp, giao thức truyền thông tiêu biểu phục vụ cho mục tiêu kiểm chứng kiến trúc.
- Hiện thực nguyên mẫu gateway với các chức năng cơ bản: thu thập dữ liệu từ thiết bị đầu cuối, xử lý cục bộ, truyền dữ liệu lên nền tảng phía trên và nhận lệnh điều khiển ngược lại.
- Xây dựng công cụ và quy trình cấu hình hệ thống ở mức cơ bản.

Đề án không tập trung vào việc phát triển một sản phẩm thương mại hoàn chỉnh, cũng không đi sâu vào các thuật toán xử lý dữ liệu nâng cao hay tối ưu hóa hiệu năng ở quy mô lớn. Các nội dung nâng cao như triển khai trên diện rộng hoặc tích hợp sâu với nền tảng cloud thương mại sẽ được xem là hướng phát triển trong các giai đoạn tiếp theo.

2.4 Phương pháp thực hiện

Đề án được triển khai theo hướng tiếp cận của một dự án kỹ thuật, kết hợp giữa phân tích yêu cầu, thiết kế hệ thống và hiện thực nguyên mẫu. Các phương pháp chính được sử dụng bao gồm:

- Khảo sát và phân tích: tìm hiểu các mô hình IoT gateway phổ biến, các giải pháp thương mại hiện có, từ đó xác định ưu – nhược điểm và rút ra yêu cầu kỹ thuật phù hợp cho hệ thống đề xuất.
- Thiết kế hệ thống: xây dựng kiến trúc tổng thể của gateway, bao gồm phân tách chức năng phần cứng và phần mềm, xác định vai trò của từng khối chức năng và luồng dữ liệu trong hệ thống.
- Lựa chọn giải pháp kỹ thuật: chọn nền tảng phần cứng, giao thức và công nghệ phù hợp với mục tiêu tối ưu chi phí, tính linh hoạt và khả năng mở rộng.
- Hiện thực và tích hợp: triển khai nguyên mẫu gateway, tích hợp các module phần cứng và phần mềm theo thiết kế đã đề ra.

- Kiểm thử và đánh giá: kiểm tra hoạt động của hệ thống trong các kịch bản cơ bản, đánh giá mức độ đáp ứng mục tiêu đề ra và rút ra nhận xét cho hướng hoàn thiện tiếp theo.

Hướng triển khai này giúp đồ án tập trung vào bài toán kỹ thuật, đảm bảo các giải pháp được thiết kế và hiện thực có tính khả thi cao trong thực tế.

Chương 3

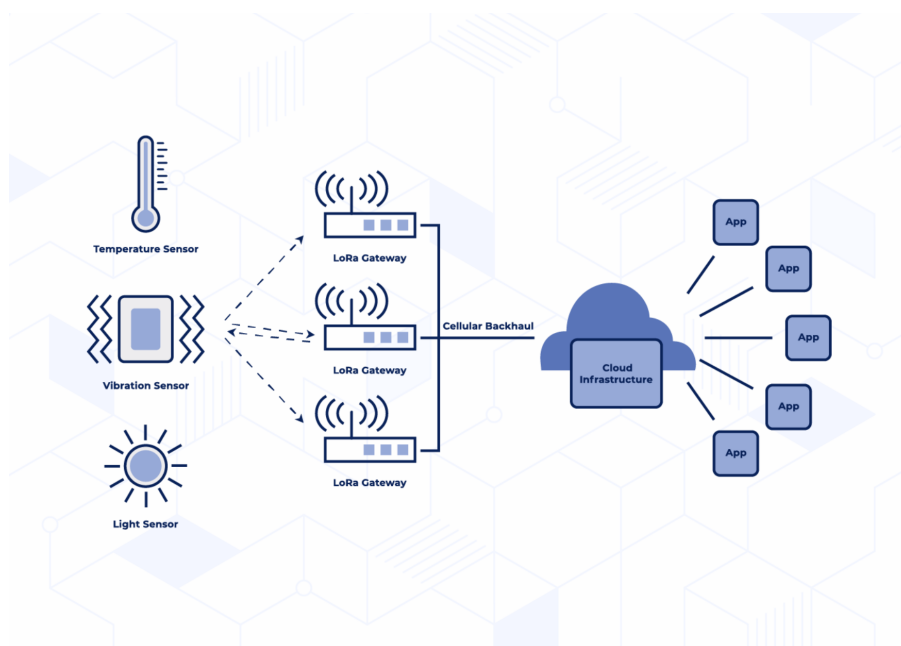
Cơ sở Lý thuyết

3.1 Tổng quan về Lĩnh vực

3.1.1 Khái niệm Internet vạn vật (IoT)

Internet vạn vật (Internet of Things – IoT) là một mô hình kết nối các đối tượng vật lý, thiết bị và cảm biến với Internet, cho phép chúng thu thập, trao đổi và phân tích dữ liệu mà không cần sự can thiệp trực tiếp từ con người. IoT tạo ra một mạng lưới các thiết bị không đồng nhất như thiết bị gia dụng, máy móc công nghiệp và cảm biến, có khả năng giao tiếp với nhau thông qua nhiều giao thức và tiêu chuẩn khác nhau.

Trong bối cảnh phát triển công nghệ hiện nay, hệ thống IoT đóng vai trò ngày càng quan trọng trong các lĩnh vực khác nhau, từ nhà thông minh, thành phố thông minh, công nghiệp 4.0, cho đến nông nghiệp dữ liệu. Sự đa dạng của các thiết bị và giao thức truyền thông tạo ra nhu cầu cấp thiết cho các giải pháp trung gian có khả năng tích hợp và quản lý hiệu quả.



Hình 3.1: Hạ tầng Internet vạn vật (IoT)

3.1.2 Vị trí và vai trò của IoT gateway

IoT gateway đóng vai trò như một cầu nối quan trọng giữa các thiết bị IoT và các nền tảng đám mây, hoạt động như một trung tâm điều phối chính, cho phép truyền thông và luồng dữ liệu trong hạ tầng Internet vạn vật. Gateway là thành phần then chốt giữa lớp cảm nhận (perception layer), gồm các cảm biến và thiết bị và lớp mạng (network layer) kết nối đến các dịch vụ đám mây.

Theo tiêu chuẩn công nghiệp, một gateway IoT được định nghĩa là một thiết bị phần cứng hoặc ứng dụng phần mềm đóng vai trò là điểm kết nối giữa máy chủ ứng dụng IoT và các thiết bị đầu cuối. Tiêu chuẩn ITU-T Y.4418 định nghĩa gateway là "một đơn vị trong Internet vạn vật có chức năng kết nối các thiết bị với mạng truyền thông, thực hiện việc chuyển đổi cần thiết giữa các giao thức được sử dụng trong mạng truyền thông và các giao thức mà thiết bị sử dụng."

IoT gateway hoạt động như những trung tâm thông minh, liên kết các thiết bị IoT và đám mây với nhau, đảm nhiệm việc chuyển đổi giao tiếp giữa các thiết bị IoT cũng như lọc và xử lý dữ liệu thành thông tin hữu ích. Chúng tạo ra cầu nối giữa các cảm biến và cơ cấu chấp hành (actuators) ở một phía, và các nền tảng hậu kỳ (quản lý dữ liệu, thiết bị và người dùng) ở phía còn lại.

3.2 Cơ sở lý thuyết về IoT Gateway

3.2.1 Khái niệm IoT gateway

IoT gateway trong lĩnh vực Internet of Things đóng vai trò là nút trung gian giữa các mạng cảm biến và nền tảng đám mây hoặc trung tâm dữ liệu được kết nối thông qua Internet. Mục tiêu chính của gateway là xử lý tính không đồng nhất do các thiết bị và giao thức khác nhau tạo ra, thu thập lượng lớn dữ liệu ở nhiều định dạng khác nhau và chuyển tiếp dữ liệu đã xử lý lên các hệ thống mức cao hơn.

Để hệ thống IoT vận hành và được quản lý đúng cách, dữ liệu thu thập từ thiết bị biên cần được làm sạch, tiền xử lý và lọc trước khi gửi đến trung tâm dữ liệu, tùy theo yêu cầu của từng ứng dụng. IoT gateway được thiết kế để đảm nhiệm những chức năng này một cách hiệu quả tại biên mạng.

Về mặt chức năng, IoT gateway có thể được hiểu thông qua ba năng lực cốt lõi:

1. **Chuyển tiếp thông điệp (Message Forwarding):** khả năng truyền tải dữ liệu giữa các thiết bị và hệ thống xử lý tập trung một cách đáng tin cậy, bảo toàn ngữ nghĩa và chất lượng dịch vụ (QoS) mong muốn.
2. **Xử lý cục bộ (Local Processing):** thực hiện các tác vụ điện toán biên (edge computing) để xử lý, tổng hợp hoặc ra quyết định tại chỗ, giúp giảm độ trễ, tiết kiệm băng thông và giảm phụ thuộc vào kết nối liên tục với cloud.
3. **Khả năng mở rộng tài nguyên (Resource Openness):** cung cấp các giao diện và dịch vụ chung cho nhiều ứng dụng IoT, cho phép quản lý thiết bị, phát hiện và truy cập các tài nguyên (sensors, actuators, dữ liệu, ...) một cách thống nhất.

Nhờ ba năng lực trên, IoT gateway không chỉ đóng vai trò là điểm trung chuyển dữ liệu mà còn là một nút điện toán biên thông minh, giúp giảm độ trễ, tối ưu băng thông và hỗ trợ ra

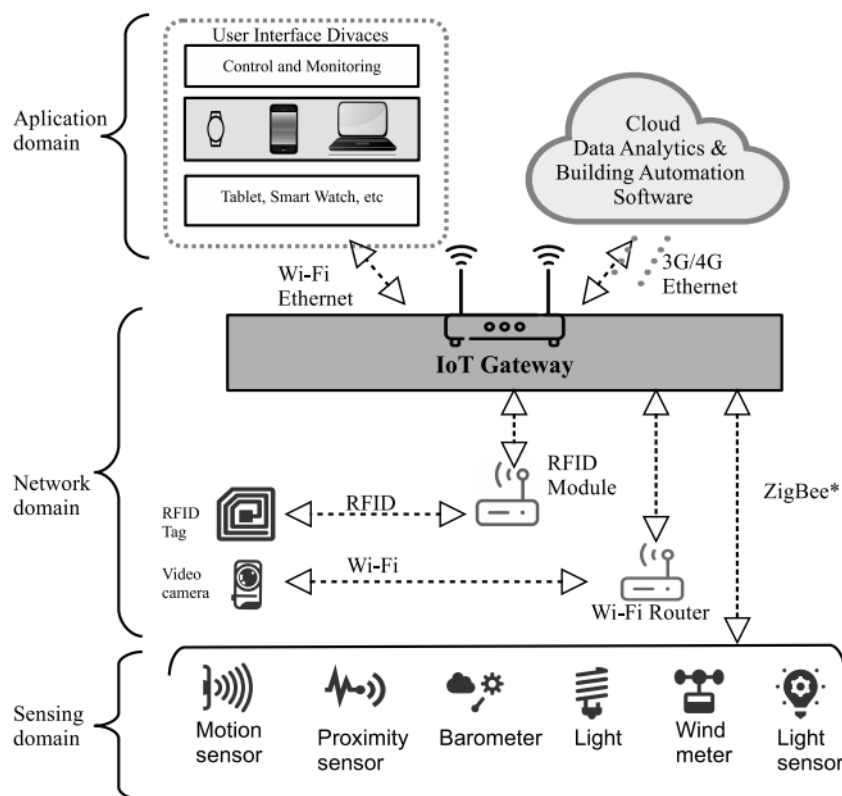
quyết định gần thời gian thực ngay tại biên mạng.

3.2.2 Vị trí của IoT gateway trong kiến trúc IoT

IoT gateway chiếm vị trí chiến lược trong kiến trúc nhiều lớp của hệ thống IoT. Theo tiêu chuẩn của IEEE, trong ba lớp của Internet vạn vật (gồm lớp cảm nhận, lớp mạng và lớp ứng dụng), gateway là thành phần thiết yếu vì lớp cảm nhận và lớp mạng thuộc về các mạng dị thể (heterogeneous networks) vốn không thể giao tiếp trực tiếp với nhau.

Các IoT gateway đóng vai trò là liên kết trung gian, cho phép thông tin thu thập từ các cảm biến IoT được chuyển đổi và đóng gói thành các gói dữ liệu mạng để truyền đến các máy chủ backend hoặc nền tảng đám mây. Nhờ đó, IoT gateway trở thành điểm then chốt trong việc đảm bảo luồng dữ liệu ổn định và tin cậy trong toàn bộ hạ tầng IoT.

Trong bối cảnh phân cấp mạng IoT, các IoT gateway hoạt động như các điểm hội tụ dữ liệu (aggregation points), nơi nhiều thiết bị IoT hợp nhất luồng dữ liệu của chúng trước khi truyền lên các hệ thống cấp cao hơn. Vị trí chiến lược này cho phép IoT gateway quản lý và tối ưu hóa luồng dữ liệu từ nhiều thiết bị đầu cuối, đảm bảo sử dụng băng thông hiệu quả và giảm tải xử lý cho hạ tầng đám mây.



Hình 3.2: Vị trí của IoT gateway trong kiến trúc IoT

3.2.3 Các vai trò chính của IoT gateway

IoT Gateway không chỉ là một điểm kết nối đơn thuần mà là một hệ thống đa chức năng: tổng hợp dữ liệu từ nhiều thiết bị đầu cuối, chuyển đổi giao thức không đồng nhất, thực thi bảo

mật, và xử lý dữ liệu tại biên. Các vai trò này phối hợp chặt chẽ để giúp IoT gateway trở thành thành phần trọng tâm của bất kỳ kiến trúc IoT hiệu quả.

Tổng hợp và quản lý dữ liệu (data aggregation and management)

IoT gateway thu thập dữ liệu từ nhiều thiết bị và cảm biến IoT, hợp nhất các luồng thông tin trước khi chuyển tiếp đến các nền tảng đám mây hoặc trung tâm dữ liệu. Chức năng tổng hợp này giúp giảm lưu lượng mạng và cho phép xử lý dữ liệu hiệu quả hơn. Gateway có thể thực hiện các hoạt động như lọc dữ liệu trùng lặp, phân loại theo loại hoặc nguồn, và tính toán các chỉ số tổng hợp.

Chuyển đổi giao thức (protocol translation)

Một trong những chức năng quan trọng nhất của IoT gateway là khả năng chuyển đổi hai chiều giữa các giao thức truyền thông khác nhau. Thiết bị có thể chuyển dữ liệu từ các giao thức công nghiệp cũ như Modbus sang các giao thức IoT hiện đại như MQTT hoặc HTTP, giúp tích hợp liền mạch giữa hạ tầng hiện có và hệ thống IoT mới. Khả năng này rất cần thiết để xây dựng một kiến trúc truyền thông thống nhất trong môi trường thiết bị không đồng nhất.

Khả năng điện toán biên (edge computing capabilities)

Các IoT gateway hiện đại ngày càng tích hợp chức năng điện toán biên, cho phép xử lý và phân tích dữ liệu tại chỗ mà không cần phụ thuộc hoàn toàn vào kết nối đám mây. Cách tiếp cận xử lý phân tán này giúp phản hồi theo thời gian thực đối với các sự kiện quan trọng, giảm tải, giảm độ trễ cho các hệ thống xử lý trung tâm và tiết kiệm băng thông. Điểm toán biên cho phép thực hiện các tác vụ như phát hiện bất thường, cảnh báo ngưỡng vượt, hoặc ra quyết định tác động nhanh chóng.

Thực thi bảo mật (security enforcement)

IoT gateway thường được xem như lớp phòng tuyến bảo mật đầu tiên của hạ tầng IoT. Thông qua gateway, các cơ chế như kiểm soát truy cập, xác thực và phân quyền thiết bị/người dùng, mã hóa dữ liệu (ví dụ: TLS/DTLS, VPN) và lọc gói (firewall, ACL) được triển khai tập trung ở một điểm. Gateway đóng vai trò điểm thực thi chính sách bảo mật (policy enforcement point), kiểm tra và áp dụng các quy tắc đối với cả lưu lượng từ thiết bị lên cloud (uplink) lẫn từ cloud về thiết bị (downlink). Bên cạnh đó, gateway có thể tích hợp các chức năng phát hiện bất thường, ghi nhật ký truy vết (audit log) và quản lý chứng chỉ số, giúp phục vụ công tác giám sát, phân tích sự cố và đảm bảo tuân thủ các yêu cầu an toàn thông tin của hệ thống.

Quản lý kết nối và thiết bị (connectivity and device management)

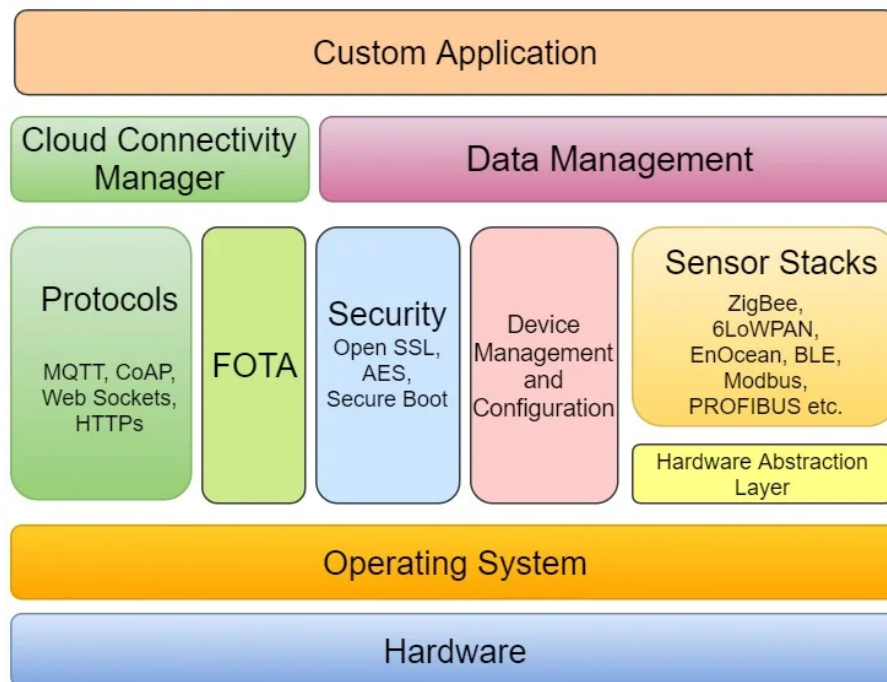
IoT gateway không chỉ duy trì kết nối mạng cho các thiết bị đầu cuối mà còn đóng vai trò nền tảng quản lý thiết bị tập trung. Thông qua gateway, hệ thống có thể giám sát trạng thái kết nối, chất lượng liên kết, cấu hình tham số hoạt động và điều khiển thiết bị từ xa. Gateway thường hỗ trợ các chức năng như tự động phát hiện (device discovery), đăng ký/hủy đăng ký thiết bị, cập nhật firmware qua mạng (FOTA) và chẩn đoán lỗi từ xa. Việc tập trung các chức năng này tại gateway giúp đơn giản hóa công tác vận hành, giảm tải cho nền tảng cloud và cho phép quản trị viên quản lý toàn bộ hệ sinh thái IoT một cách thống nhất, hiệu quả.

Lưu trữ cục bộ và bộ nhớ đệm (local storage and caching)

Nhiều IoT gateway cung cấp khả năng lưu trữ tạm thời, cho phép lưu đệm dữ liệu trong trường hợp mất kết nối và đồng bộ lại khi kết nối được khôi phục. Tính năng này giúp đảm bảo tính liên tục và toàn vẹn của dữ liệu trong các môi trường có kết nối mạng không ổn định. Bộ nhớ đệm cho phép gateway tiếp tục hoạt động độc lập trong một khoảng thời gian nhất định ngay cả khi kết nối với cloud bị ngắt.

3.2.4 Cấu trúc của IoT gateway

Một IoT Gateway hiện đại được thiết kế theo kiến trúc nhiều lớp (layered architecture), trong đó mỗi lớp đảm nhận một nhóm chức năng cụ thể và phối hợp chặt chẽ với các lớp khác. Cấu trúc này không chỉ tạo ra sự tách biệt rõ ràng giữa phần cứng và phần mềm, mà còn cho phép gateway hoạt động như một nền tảng tính toán linh hoạt, có thể mở rộng và dễ bảo trì. Về tổng thể, kiến trúc của gateway có thể chia thành hai khối chính: khối phần cứng (hardware layer) đảm nhiệm việc thu nhận và truyền dẫn tín hiệu vật lý, và khối phần mềm nhiều lớp (multi-layer software stack) thực hiện các chức năng xử lý, quản lý và giao tiếp cấp cao.



Hình 3.3: Cấu trúc của IoT gateway

3.2.4.1 Kiến trúc phần cứng

Lớp phần cứng là nền tảng vật lý của gateway, cung cấp khả năng kết nối, tính toán và lưu trữ cần thiết cho toàn bộ hệ thống. Các thành phần chính của kiến trúc phần cứng bao gồm:

Bộ xử lý trung tâm và bộ nhớ

Gateway sử dụng vi xử lý hoặc vi điều khiển có năng lực xử lý đủ mạnh kết hợp với bộ nhớ RAM và bộ nhớ Flash/eMMC để lưu trữ hệ điều hành, firmware và dữ liệu tạm thời. Tùy vào yêu cầu ứng dụng, gateway có thể sử dụng Single Board Computer (SBC) như Raspberry Pi,

hoặc các nền tảng công nghiệp chuyên dụng có khả năng hoạt động trong môi trường khắc nghiệt.

Các module giao tiếp và kết nối

Đây là thành phần cốt lõi giúp gateway kết nối với thiết bị biên và mạng Internet:

- Kết nối có dây: Ethernet (Fast Ethernet/Gigabit), RS-232, RS-485/422, Modbus RTU, CAN bus phục vụ giao tiếp với các thiết bị công nghiệp, cảm biến và PLC.
- Kết nối không dây tầm ngắn: Bluetooth Low Energy (BLE), Zigbee, Z-Wave, Wi-Fi (2.4/5 GHz) để kết nối với các thiết bị IoT trong phạm vi cục bộ như cảm biến, công tắc, thiết bị gia dụng thông minh.
- Kết nối không dây tầm xa: LoRa/LoRaWAN, NB-IoT, LTE-M, 4G/5G để truyền dữ liệu lên cloud hoặc server từ xa.

Nguồn điện và quản lý năng lượng

Khối nguồn DC/DC hoặc AC/DC converter cung cấp nguồn ổn định cho toàn bộ hệ thống, thường hỗ trợ nguồn đầu vào rộng và tích hợp các mạch bảo vệ chống quá áp, quá dòng, chống đảo cực để đảm bảo gateway hoạt động tin cậy trong môi trường công nghiệp. Một số gateway còn hỗ trợ Power over Ethernet (PoE) để giảm thiểu cáp nguồn riêng biệt.

Khối bảo mật phần cứng

Nhiều gateway hiện đại tích hợp thêm các chip bảo mật chuyên dụng như Trusted Platform Module (TPM) hoặc Secure Element. Các phần tử này được dùng để lưu trữ khóa mã hóa, chứng chỉ số và hỗ trợ các cơ chế như Secure Boot, mã hóa bộ nhớ, xác thực thiết bị bằng chứng chỉ X.509, qua đó nâng cao mức độ an toàn bảo mật ngay từ tầng phần cứng.

Các cổng mở rộng và I/O

Gateway thường cung cấp các chân GPIO, ADC/DAC (chuyển đổi tương tự/số), cổng USB, khe cắm thẻ nhớ SD/microSD và khe SIM để mở rộng khả năng kết nối và lưu trữ.

3.2.4.2 Kiến trúc phần mềm

Trên nền tảng phần cứng, kiến trúc phần mềm của gateway được tổ chức thành các lớp chức năng độc lập, tạo thành một hệ thống phân tầng rõ ràng từ lớp phần cứng đến lớp ứng dụng. Các lớp chính bao gồm:

Lớp hệ điều hành (Operating System)

Hệ điều hành cung cấp nền tảng phần mềm cơ bản để quản lý tài nguyên phần cứng, lập lịch tiến trình, quản lý bộ nhớ và cung cấp môi trường thực thi cho các dịch vụ phía trên. Tùy yêu cầu, gateway có thể sử dụng RTOS cho các tác vụ thời gian thực hoặc Linux nhúng (OpenWrt, Debian, Ubuntu Core, ...) cho các ứng dụng phức tạp hỗ trợ container và tích hợp cloud.

Lớp trừu tượng phần cứng (Hardware Abstraction Layer – HAL)

HAL cung cấp tập API thống nhất để các mô-đun phần mềm tương tác với ngoại vi phần cứng (UART, SPI, I²C, radio, ...) mà không phụ thuộc vào chi tiết triển khai của từng nền tảng gateway. Điều này giúp tăng tính khả chuyển, cho phép cùng một mã nguồn ứng dụng có thể chạy trên các bo mạch phần cứng khác nhau chỉ bằng cách thay đổi HAL.

Lớp ngăn xếp cảm biến (Sensor Stacks)

Lớp này hiện thực các driver và ngăn xếp giao thức cho mạng cảm biến và thiết bị như ZigBee, 6LoWPAN, EnOcean, BLE, Modbus, PROFIBUS, ... Nó chịu trách nhiệm thu thập dữ liệu từ

các nút cảm biến/chấp hành, chuyển về định dạng nội bộ thống nhất để các mô-đun quản lý dữ liệu và ứng dụng có thể sử dụng.

Lớp giao thức truyền thông (Protocols)

Lớp giao thức cung cấp các protocol ở tầng ứng dụng phục vụ trao đổi dữ liệu với cloud hoặc hệ thống bên ngoài, điển hình như MQTT, CoAP, WebSockets, HTTP/HTTPS,... Các mô-đun này được xây dựng trên nền TCP/UDP của hệ điều hành và được các khối quản lý dữ liệu, quản lý cloud sử dụng để gửi/nhận các gói tin.

Lớp cập nhật firmware từ xa (FOTA/OTA)

Mô-đun FOTA chịu trách nhiệm kiểm tra, tải về, xác thực và cài đặt các bản cập nhật phần mềm/firmware từ cloud. Ngoài việc đảm bảo gateway luôn được vá lỗi và cập nhật tính năng mới, lớp này còn hỗ trợ cơ chế rollback trong trường hợp cập nhật thất bại, giúp hệ thống duy trì trạng thái ổn định.

Lớp bảo mật (Security)

Lớp bảo mật cung cấp các dịch vụ như mã hoá, quản lý khoá và chứng chỉ, thư viện bảo mật (ví dụ TLS/OpenSSL, AES), cũng như cơ chế secure boot và xác thực thiết bị. Các dịch vụ này được các lớp giao thức, FOTA, quản lý thiết bị và ứng dụng sử dụng xuyên suốt để đảm bảo tính bảo mật, toàn vẹn và xác thực của dữ liệu.

Lớp quản lý và cấu hình thiết bị (Device Management and Configuration)

Lớp này quản lý vòng đời của thiết bị IoT: đăng ký/hủy đăng ký, lưu trữ thuộc tính (kiểu thiết bị, vị trí, tham số hoạt động), giám sát trạng thái, cấu hình tham số, cũng như hỗ trợ cập nhật firmware và chẩn đoán từ xa. Đây là điểm tập trung cho việc quản lý tập trung số lượng lớn node cảm biến/actuator.

Lớp quản lý dữ liệu (Data Management)

Lớp quản lý dữ liệu chịu trách nhiệm tiếp nhận luồng dữ liệu từ sensor stacks, thực hiện lọc, tiền xử lý, tổng hợp và lưu trữ đệm. Mô-đun này giúp giảm băng thông bằng việc chỉ gửi dữ liệu cần thiết, bảo vệ dữ liệu khi kết nối cloud không ổn định (buffering và đồng bộ lại khi có mạng), đồng thời hỗ trợ các chức năng xử lý biên như phát hiện bất thường hoặc áp dụng rule engine.

Lớp quản lý kết nối cloud (Cloud Connectivity Manager)

Cloud Connectivity Manager đảm nhiệm việc thiết lập và duy trì kết nối hai chiều giữa gateway và nền tảng cloud: quản lý phiên làm việc, gửi heartbeat, xử lý cơ chế reconnect, xác thực gateway với cloud và ánh xạ trạng thái thiết bị trên cloud. Lớp này sử dụng các giao thức ở mô-đun *Protocols* và cung cấp API thống nhất cho lớp ứng dụng.

Lớp ứng dụng tùy biến (Custom Application)

Trên cùng là lớp ứng dụng tùy biến, nơi hiện thực logic nghiệp vụ cụ thể của hệ thống: các kịch bản tự động hoá, xử lý cảnh báo và sự kiện, dashboard cục bộ cho cấu hình/giám sát, cũng như các dịch vụ tích hợp với hệ SCADA/MES/ERP hoặc nền tảng quản lý vận hành khác. Các ứng dụng này tận dụng toàn bộ dịch vụ do các lớp phía dưới cung cấp để xây dựng giải pháp IoT hoàn chỉnh.

3.2.5 Nguyên lý hoạt động cơ bản của IoT Gateway

Quá trình hoạt động của gateway là một vòng lặp liên tục: thu thập dữ liệu từ thiết bị đầu cuối, xử lý và lọc tại chỗ, truyền gói tin lên cloud, nhận lệnh điều khiển từ cloud, và gửi lệnh xuống thiết bị. Trong suốt quá trình này, gateway phải đảm bảo tính nhất quán dữ liệu, bảo

mật thông tin, phát hiện và xử lý lỗi, cũng như duy trì kết nối liên tục ngay cả trong điều kiện mạng không ổn định. Những yêu cầu này làm cho độ tin cậy, hiệu suất, và bảo mật trở thành những tiêu chí bắt buộc chứ không phải tùy chọn trong bất kỳ triển khai thực tế nào.

Thu thập dữ liệu (Data Ingestion)

Gateway tiếp nhận dữ liệu thô từ các cảm biến và thiết bị có cơ cấu chấp hành thông qua các giao thức kết nối được hỗ trợ (như ZigBee, LoRa, BLE, Modbus...). Đây là bước đầu tiên, giúp gateway đóng vai trò là điểm thu gom dữ liệu từ lớp thiết bị (perception layer). Dữ liệu thô này có thể ở các định dạng khác nhau tùy thuộc vào loại cảm biến và giao thức sử dụng.

Xử lý cục bộ (Local Processing)

Trước khi truyền dữ liệu đi, gateway có thể thực hiện các tác vụ xử lý tại chỗ như lọc nhiễu, loại bỏ dữ liệu trùng lặp, tổng hợp số đo, hoặc phát hiện các bất thường thông qua rule engine hoặc thuật toán phân tích biên (edge analytics). Nhờ đó, gateway giúp giảm lưu lượng truyền và tiết kiệm chi phí lưu trữ trên nền tảng đám mây. Các tác vụ xử lý này có thể được cấu hình linh hoạt theo nhu cầu của ứng dụng cụ thể.

Định tuyến giao thức (Protocol Routing)

Gateway mã hóa dữ liệu bằng các giao thức bảo mật trước khi truyền lên nền tảng đám mây, giúp bảo vệ thông tin nhạy cảm trong quá trình di chuyển qua mạng. Trên nền tảng đám mây, dữ liệu tiếp tục được phân tích, trực quan hóa và lưu trữ để tạo ra thông tin chi tiết phục vụ cho việc hình thành hành động hoặc kích hoạt các quy trình tự động. Sau khi xử lý, dữ liệu được định tuyến đến các hệ thống đích phù hợp như dịch vụ đám mây, MQTT broker hoặc cơ sở dữ liệu cục bộ.

Điều khiển và quản lý từ xa (Remote Control and Management)

Ngoài các nhiệm vụ truyền dữ liệu, gateway cũng nhận các thông tin từ nền tảng đám mây hoặc hệ thống quản lý để gửi lệnh điều khiển xuống các thiết bị biên, cập nhật phần mềm (OTA update), hoặc thay đổi tham số vận hành. Cơ chế này giúp duy trì khả năng quản lý tập trung và đảm bảo thiết bị luôn hoạt động theo đúng cấu hình mong muốn.

Dự phòng và lưu đệm (Failover and Caching)

Trong trường hợp mất kết nối mạng hoặc đám mây, gateway có khả năng lưu trữ tạm thời dữ liệu cục bộ (caching) và đồng bộ lại khi kết nối được phục hồi. Một số gateway còn hỗ trợ tính năng clustering (cụm gateway) để tăng tính sẵn sàng và đảm bảo hoạt động liên tục. Cơ chế này là rất quan trọng đối với các ứng dụng yêu cầu tính tin cậy cao.

Thực thi bảo mật (Security Enforcement)

Gateway là tuyến phòng thủ đầu tiên trong hệ thống IoT, thực hiện các biện pháp bảo mật như mã hóa dữ liệu (encryption), xác thực thiết bị (authentication), và lọc gói tin (packet filtering) để đảm bảo an toàn cho dữ liệu cả trong quá trình truyền và khi lưu trữ. Gateway thực hiện kiểm soát truy cập, ngăn chặn các kết nối trái phép, và giám sát hoạt động để phát hiện các bất thường bảo mật.

3.3 Phân tích các sản phẩm trên thị trường

3.3.1 Các công nghệ kết nối phổ biến

Trong hạ tầng IoT hiện đại, các gateway cần hỗ trợ đa dạng công nghệ kết nối để tích hợp với nhiều loại thiết bị khác nhau. Các công nghệ kết nối tầm gần như **WiFi**, **Bluetooth Low Energy (BLE)**, **Zigbee**, **Z-Wave** được sử dụng rộng rãi trong các ứng dụng gia dụng và văn phòng. Trong khi đó, các công nghệ **LPWAN** như **LoRaWAN**, **NB-IoT**, **LTE-M** phù hợp với những kịch bản yêu cầu phạm vi phủ sóng rộng và tiêu thụ năng lượng thấp.

Bên cạnh đó, các kết nối có dây như **Ethernet**, **RS-485**, **RS-232**, **CAN** vẫn giữ vai trò quan trọng trong môi trường công nghiệp nhờ độ ổn định cao và khả năng chống nhiễu tốt. Việc hỗ trợ đa giao thức kết nối giúp gateway tương thích với hạ tầng thiết bị hiện hữu, đồng thời tạo điều kiện thuận lợi cho việc mở rộng và nâng cấp hệ thống trong tương lai.

3.3.2 Các giao thức truyền dữ liệu

Tại tầng ứng dụng, các giao thức truyền dữ liệu phổ biến gồm:

- **MQTT**: phù hợp nhất cho môi trường hạn chế tài nguyên nhờ mô hình publish-subscribe nhẹ, hỗ trợ cơ chế QoS và hoạt động tốt trên đường truyền không ổn định.
- **CoAP**: được thiết kế cho thiết bị nhẹ, hoạt động trên nền UDP giúp giảm overhead truyền thông, thích hợp cho mạng cảm biến và ứng dụng cần tối giản băng thông.
- **HTTP/HTTPS**: được ưu tiên trong các ứng dụng cần bảo mật cao, dễ tích hợp trực tiếp với hệ thống web, REST API và các dịch vụ cloud truyền thống.
- **AMQP**: hướng tới các hệ thống doanh nghiệp cần hàng đợi thông điệp tin cậy, đảm bảo giao nhận và tích hợp với middleware.
- **XMPP**: phù hợp cho các ứng dụng cần giao tiếp hai chiều theo thời gian gần thực và mô hình nhắn tin mở rộng giữa các nút IoT.

3.3.3 Các mô hình triển khai gateway

Trong thực tế, kiến trúc IoT thường áp dụng hai mô hình triển khai gateway chính: mô hình phân tán, trong đó thiết bị đầu cuối đồng thời đảm nhiệm vai trò gateway, và mô hình tập trung với một gateway trung tâm phục vụ nhiều thiết bị đầu cuối.

Thiết bị đầu cuối kiêm chức năng gateway (mô hình phân tán)

Ở mô hình này, mỗi thiết bị đầu cuối (end device) vừa thực hiện chức năng cảm biến/điều khiển, vừa tích hợp đầy đủ khả năng kết nối mạng và giao tiếp trực tiếp với cloud hoặc server trung tâm (ví dụ: camera IP thông minh, đồng hồ điện thông minh, ...).

Điều này đòi hỏi bản thân thiết bị:

- có tài nguyên phần cứng đủ mạnh (CPU, RAM, bộ nhớ lưu trữ) để vận hành hệ điều hành, ngăn xếp giao thức mạng, các cơ chế bảo mật và logic ứng dụng;
- có nguồn năng lượng ổn định (thường là nguồn AC hoặc pin dung lượng lớn) do phải duy trì kết nối mạng và xử lý liên tục.

Ưu điểm của mô hình phân tán là mỗi nút có thể hoạt động độc lập, giảm phụ thuộc vào một gateway trung tâm và nâng cao tính sẵn sàng của hệ thống. Tuy nhiên, khi số lượng thiết bị tăng cao, việc để tất cả nút tự kết nối lên cloud có thể:

- làm tăng số lượng kết nối và lưu lượng trên mạng backbone, tiềm ẩn nguy cơ nghẽn mạng;
- làm tăng chi phí phần cứng trên từng thiết bị do cần tích hợp thêm tài nguyên xử lý và kết nối;
- khiến công tác quản lý, cập nhật và bảo trì phần mềm trên diện rộng trở nên phức tạp.

Nhiều thiết bị đầu cuối kết nối đến một gateway trung tâm (mô hình tập trung)

Trong mô hình tập trung, các cảm biến và thiết bị trường chủ yếu thực hiện chức năng đo lường/điều khiển cơ bản, giao tiếp qua các kết nối tầm ngắn hoặc fieldbus (RS-485/Modbus, CAN, Zigbee,...) đến một IoT gateway trung tâm. Gateway chịu trách nhiệm:

- tập trung thu thập, tiền xử lý và tổng hợp dữ liệu;
- chuyển đổi giao thức và quản lý kết nối lên cloud;
- thực thi bảo mật, quản lý thiết bị và hỗ trợ cập nhật phần mềm.

Mô hình này cho phép:

- đơn giản hoá và giảm chi phí phần cứng của thiết bị đầu cuối, đồng thời giảm yêu cầu về năng lượng tại từng nút;
- hạn chế số lượng kết nối trực tiếp tới cloud, giúp dễ dàng quản lý và giảm nguy cơ nghẽn ở lớp mạng;
- thuận lợi trong việc bảo trì, nâng cấp gateway mà ít ảnh hưởng đến toàn bộ hệ thống cảm biến.

Nhược điểm là gateway trở thành thành phần trọng yếu; nếu không được thiết kế cơ chế dự phòng (redundancy), nó có thể trở thành *điểm lỗi đơn* (single point of failure) của toàn hệ thống.

3.3.4 Các tính năng quan trọng của gateway trên thị trường

Các gateway phổ biến hiện nay thường tích hợp những tính năng sau:

- Hỗ trợ đa giao thức kết nối (WiFi, Ethernet, Zigbee, LoRa, BLE, LTE, ...)
- Chuyển đổi giao thức hai chiều (protocol translation)
- Xử lý dữ liệu tại biên (edge computing)
- Lưu trữ dữ liệu cục bộ (local storage)
- Quản lý và điều khiển thiết bị từ xa
- Hỗ trợ cập nhật firmware OTA
- Hệ thống bảo mật mạnh: mã hóa, xác thực, kiểm soát truy cập
- Hỗ trợ giao diện cấu hình thân thiện
- Tích hợp dễ dàng với nền tảng cloud (AWS IoT, Azure IoT, Adafruit IO)

3.3.5 Ví dụ về các gateway thực tế trên thị trường

M5Stack IoT gateway series

M5Stack cung cấp các giải pháp gateway IoT dựa trên nền tảng ESP32 với khả năng kết nối đa dạng (WiFi, BLE, LoRa). Các thiết bị này hỗ trợ giao thức MQTT, HTTP và có thể lập trình dễ dàng thông qua Arduino IDE hoặc MicroPython. M5Stack gateway phù hợp cho các ứng dụng prototyping và phát triển nhanh. Hai dòng sản phẩm tiêu biểu là CoreS3-Lite và CoreMP135.

M5Stack CoreS3-Lite Thiết bị nhỏ gọn của M5Stack với khả năng hoạt động như một mini-gateway hoặc HMI trong hệ thống IoT quy mô nhỏ. Nhờ tích hợp sẵn WiFi/BLE, màn hình cảm ứng, cảm biến và các giao tiếp mở rộng, CoreS3-Lite có thể vừa thu thập dữ liệu, hiển thị trạng thái, vừa giao tiếp với nhiều thiết bị ngoại vi thông qua các module.

- MCU **ESP32-S3**, lõi kép Xtensa LX7 240 MHz, hỗ trợ **WiFi 2.4 GHz** và **Bluetooth Low Energy (BLE)**.
- Bộ nhớ **Flash 16 MB**, **PSRAM 8 MB**, đủ cho ứng dụng có giao diện đồ họa, kết nối mạng và xử lý dữ liệu cảm biến.
- Màn hình **IPS 2.0"** cảm ứng điện dung, độ phân giải 320×240, phù hợp làm giao diện HMI mini (nút bấm, slider, biểu đồ,...).
- Tích hợp **camera 0.3 MP**, cảm biến ánh sáng/tiêm cận **LTR-553**, **IMU 6 trục** và **từ kế 3 trục**, cho phép xây dựng các ứng dụng nhận diện cơ bản, phát hiện chuyển động, tương tác theo hướng nhìn,...
- Khối âm thanh gồm **codec ES7210**, **ampli I2S** và **loa 1 W** cùng **micro kép**, hỗ trợ các ứng dụng trợ lý giọng nói, phát cảnh báo âm thanh,...
- **Hỗ trợ GPIO/I²C/UART** thông qua:
 - Cổng mở rộng dạng HY2.0/Grove trên thân thiết bị để kết nối trực tiếp cảm biến, relay, module I/O.
 - Bus phía sau **M5-Bus** cho phép *stack* thêm các module M5Stack như **LoRa**, **Zigbee**, **Ethernet**, **RS485/RS232**, **CAN**, **mở rộng GPIO/ADC**,... giúp mở rộng giao thức phần cứng mà không cần thiết kế lại mạch chính.
- Hỗ trợ các giao thức ứng dụng như **MQTT**, **HTTP/REST** qua firmware phát triển trên **ESP-IDF**, **Arduino** hoặc **UiFlow**, cho phép hoạt động như một mini-gateway gửi dữ liệu lên server/cloud.
- Phù hợp cho các ứng dụng **home automation**, bảng điều khiển HMI, logger dữ liệu môi trường hay thiết bị hiển thị trạng thái tại chỗ.

M5Stack CoreMP135 Gateway/HMI IoT mini dựa trên vi xử lý **STM32MP135** mạnh mẽ, có khả năng chạy Linux, hướng đến môi trường công nghiệp và các ứng dụng edge computing cần nhiều giao tiếp trường và giao thức công nghiệp.

- CPU **STM32MP135DAE7** (Arm Cortex-A7 @ 1 GHz), có MMU nên **chạy Linux** (Buildroot, Debian, Yocto,...).

- Tích hợp **2× Gigabit Ethernet**, **2× USB 2.0-A**, **1× USB-C OTG**, khe **microSD** và cổng **PWR485** (nguồn 9–24 V + RS485), phù hợp làm gateway công nghiệp.
- WiFi/BLE có thể bổ sung thông qua **module mở rộng M5Stack** hoặc USB dongle, giúp linh hoạt trong việc xây dựng gateway IoT có cả Ethernet và kết nối không dây.
- Màn hình cảm ứng **IPS 2.0"** cùng loa 1 W dùng làm HMI: hiển thị trạng thái thiết bị, tham số hệ thống, nút điều khiển,...
- **Hỗ trợ giao tiếp công nghiệp và mở rộng phần cứng** thông qua:
 - 2× giao tiếp **CAN-FD** và **RS485** tích hợp sẵn.
 - Bus phía sau **M5-Bus** và các cổng Grove cho phép gắn thêm các module **LoRa**, **Zigbee**, **Ethernet bổ sung**, **RS485**, **CAN**, **I/O mở rộng**,... giúp CoreMP135 hoạt động như một gateway trung tâm kết nối nhiều chuẩn trường khác nhau.
- Bộ nhớ **DDR3L SDRAM** và lưu trữ **eMMC** kết hợp **microSD** (kèm thẻ microSD cài sẵn Debian) cho phép chạy các dịch vụ edge như gateway, xử lý dữ liệu, logging, web UI cấu hình,...
- Thích hợp cho **edge computing** quy mô nhỏ, gateway điều khiển công nghiệp, HMI thông minh, hoặc làm bộ điều khiển trung tâm trong tủ điện với khả năng mở rộng giao thức phần cứng qua các module M5Stack.

RAK7391 WisGate Connect

Gateway IoT/LoRaWAN linh hoạt dựa trên **Raspberry Pi Compute Module 4**, phù hợp cho ứng dụng edge và công nghiệp.

- **CPU:** Raspberry Pi Compute Module 4 (Broadcom BCM2711, 4× Cortex-A72 1.5 GHz, 1–8 GB RAM tùy cấu hình CM4).
- **Kết nối mạng:** 1× cổng Ethernet 1 Gbps (hỗ trợ PoE tùy chọn) và 1× cổng Ethernet 2.5 Gbps; HDMI 2.0, 1× USB 2.0, 2× USB 3.0.
- **Kết nối không dây:** **WiFi 5 2.4/5 GHz** và **BLE 5.0** (tùy phiên bản CM4 có WiFi/BLE); có thể gắn thêm module không dây qua mPCIe hoặc M.2.
- **Mở rộng:** 3× khe **mPCIe** (module LoRaWAN concentrator, LTE, WiFi 6,...), 1× **M.2 B-Key**, 2× khe **WisBlock IO** và header **Raspberry Pi HAT** 40-pin.
- **Nguồn:** dải **10–28 VDC** qua jack DC, terminal Phoenix hoặc **PoE IEEE 802.3at/bt**; tích hợp giám sát điện áp, nhiệt độ PCB, RTC kèm pin và bộ siêu tụ để duy trì hệ thống khi mất điện ngắn hạn.
- **Hệ điều hành:** **RAKPiOS** (fork từ Raspberry Pi OS) với driver tích hợp sẵn, tăng cường bảo mật, hỗ trợ **Docker** và triển khai nhanh các dịch vụ IoT.
- **Ứng dụng:** gateway LoRaWAN đa kênh/đa băng tần, gateway công nghiệp/edge dùng WisBlock (Modbus, 0–5 V, 4–20 mA, I/O,...), nền tảng phát triển sản phẩm mới.

SRT-IMX8P industrial gateway

Gateway công nghiệp hiệu năng cao của AAEON, phù hợp cho ứng dụng edge/AI tại biên và môi trường công nghiệp khắc nghiệt.

- **CPU:** NXP i.MX8M Plus Quad-Core Cortex-A53 1.6 GHz (tùy cấu hình có NPU tích hợp cho AI tại biên).
- **Bộ nhớ:** LPDDR4 2 GB (tùy chọn 4 GB) và eMMC 16 GB (tùy chọn 32 GB).
- **Kết nối:**
 - 2× Gigabit Ethernet.
 - 2× USB 3.0 Type-A.
 - 2× cổng nối tiếp RS-232/422/485 dạng Phoenix.
 - 2× kênh CAN-FD.
 - 1× HDMI 2.0a cho hiển thị.
- **Mở rộng:** 2× khe mini-PCIe (cho module 4G/LTE, WiFi,...), khe microSD, khe SIM và đầu nối board-to-board để mở rộng thêm PCIe, USB 3.0, I²C, GPIO,...
- **Nguồn:** dải DC 9–36 V, công suất điển hình khoảng 9.36 W, hỗ trợ làm việc trong dải nhiệt -20 °C đến 70 °C.
- **Bảo mật:** tích hợp TPM 2.0, hỗ trợ Secure Boot và các chứng chỉ công nghiệp như CE/FCC Class A, IEC61000-6-2/6-4, IEC61131-2,...
- **Hệ điều hành:** hỗ trợ Debian 11 (mặc định), Android 13, Windows 10 IoT và Yocto, thích hợp cho nhiều kịch bản triển khai khác nhau.
- **Ứng dụng:** nhà máy thông minh, gateway AI tại biên, hệ thống tự động hóa công nghiệp và các bài toán AIoT đòi hỏi độ tin cậy cao.

3.3.6 Xu hướng phát triển hiện tại

Hiện nay, xu hướng phát triển gateway IoT hướng tới:

- **Modularity:** Các gateway được thiết kế theo dạng mô-đun để có thể tùy chỉnh tính năng theo nhu cầu cụ thể, giúp tối ưu chi phí.
- **Configurability:** Quy trình cấu hình được đơn giản hóa, thân thiện với người dùng, giúp rút ngắn thời gian triển khai.
- **Edge intelligence:** Tích hợp thêm các chức năng xử lý dữ liệu cao cấp, học máy (machine learning) tại biên mạng.
- **Interoperability:** Cải thiện khả năng tích hợp giữa các nền tảng và tiêu chuẩn khác nhau.
- **Security:** Tăng cường các tính năng bảo mật, đặc biệt là trong các ứng dụng công nghiệp và quan trọng.

3.4 Cơ sở lý thuyết về Thiết kế phần cứng

3.4.1 Thiết kế High-Speed PCB

3.4.1.1 Khái niệm High-Speed PCB Design

High-Speed PCB Design là quá trình thiết kế mạch in trong đó tín hiệu số có tốc độ chuyển mức rất nhanh (thời gian cạnh lên/xuống nhỏ), khiến các đường mạch phải được xem như một đường truyền thay vì dây nối lý tưởng. Khi độ trễ lan truyền trên đường mạch xấp xỉ hoặc lớn hơn khoảng 20% thời gian cạnh lên/xuống của tín hiệu, các hiệu ứng truyền sóng như phản xạ, nhiễu xuyên kênh và bức xạ điện từ trở nên đáng kể, nên mạch phải tuân theo các nguyên tắc thiết kế high-speed.

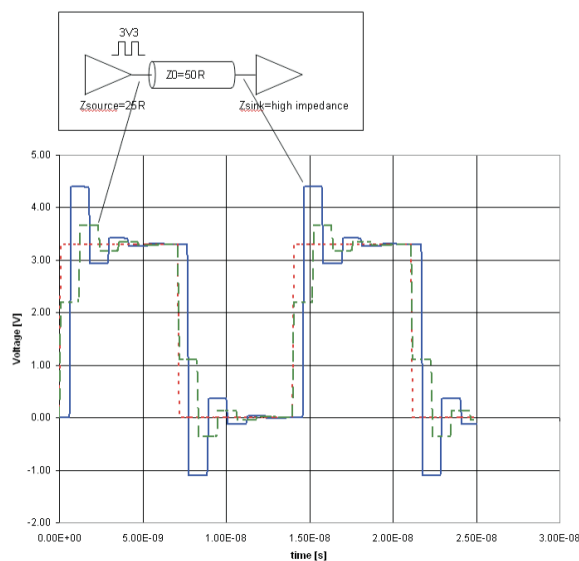
Ranh giới giữa mạch low-speed và high-speed không được xác định bằng một tần số cố định mà chủ yếu dựa trên edge rate của tín hiệu. Các hệ thống dùng giao thức tốc độ cao như USB, Ethernet, HDMI, PCIe, v.v. hầu hết đều nằm trong phạm vi thiết kế high-speed.

3.4.1.2 Tính toàn vẹn tín hiệu (Signal Integrity) và đường hồi dòng (Return Path)

Signal Integrity (SI) Signal Integrity là mức độ mà tín hiệu đến đầu nhận vẫn giữ được biên độ, dạng xung và thời điểm chuyển mức như dự kiến. Mất toàn vẹn tín hiệu thể hiện qua hiện tượng méo dạng sóng, overshoot/undershoot, rung (ringing), jitter và gây ra vấn đề lỗi dữ liệu.

Các nguyên nhân chính gây suy giảm SI gồm:

- Không khớp trở kháng trên đường truyền.
- Nhiễu xuyên kênh (crosstalk) giữa các đường mạch gần nhau.
- Bố trí return path không liên tục, tạo hồi tiếp dòng lớn và tăng cảm kháng.
- Ảnh hưởng của đặc tính vật liệu PCB như tổn hao điện môi, hiệu ứng bề mặt của dòng điện và hiện tượng tán sắc, gây suy hao biên độ và biến dạng dạng sóng trên các đường truyền dài.

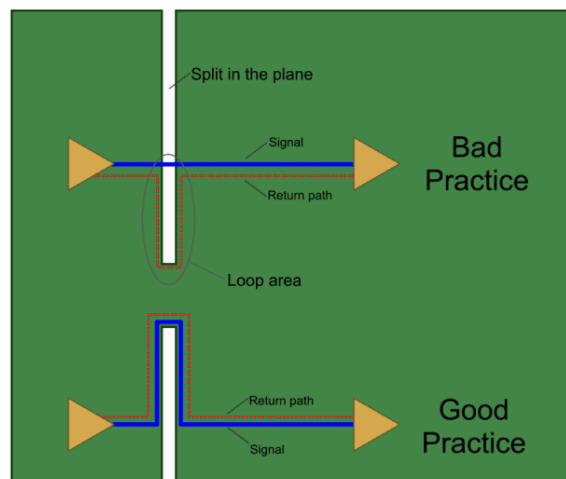


Hình 3.4: Minh họa hiện tượng ringing do không khớp trở kháng

Return path và vai trò của mặt phẳng tham chiếu Mỗi tín hiệu tốc độ cao luôn gắn liền với một dòng trở về tương ứng, thường phân bố trên mặt phẳng tham chiếu như ground plane hoặc power plane nằm kề sát lớp tín hiệu. Ở miền tần số cao, dòng trở về có xu hướng tập trung và di chuyển theo đường có tổng trở nhỏ nhất, thông thường là chạy song song và ngay phía dưới đường mạch tín hiệu nhằm giảm diện tích hồi tiếp dòng và cảm kháng. Khi mặt phẳng tham chiếu bị chia cắt, phân mảnh hoặc tồn tại khe hở, đường trở về của dòng tín hiệu sẽ bị gián đoạn và buộc phải đi vòng xa hơn, dẫn đến gia tăng cảm kháng, tăng nhiễu xuyên kênh và phát xạ nhiễu điện từ.

Do đó, trong thiết kế high-speed PCB, việc duy trì một return path liên tục và gần kề với đường tín hiệu là rất quan trọng để đảm bảo tính toàn vẹn tín hiệu. Một số nguyên tắc cơ bản bao gồm:

- Định tuyến các tín hiệu tốc độ cao trên những lớp liền kề với một mặt phẳng tham chiếu liên tục để đảm bảo đường trở về ngắn và ổn định.
- Tránh cho đường tín hiệu băng qua các khe hở hoặc vùng bị cắt trên mặt phẳng tham chiếu; trong trường hợp bắt buộc phải chuyển lớp, cần bố trí các via mass hoặc via hồi dòng để duy trì tính liên tục của đường trở về.



Hình 3.5: Minh họa return path liên tục dưới trace trên mặt phẳng tham chiếu

Hiện tượng phản xạ tín hiệu Khi tín hiệu lan truyền trên đường truyền có trở kháng đặc trưng Z_0 và gặp một điểm có trở kháng tải Z_L , một phần năng lượng bị phản xạ ngược về nguồn, gây méo dạng sóng và có thể tạo rung. Hệ số phản xạ được cho bởi:

$$\Gamma = \frac{Z_L - Z_0}{Z_L + Z_0}$$

Giá trị Γ càng xa 0, biên độ sóng phản xạ càng lớn. Điều này là lý do cốt lõi phải kiểm soát và thực hiện phối hợp trở kháng trong thiết kế high-speed.

Trong các hệ thống số tốc độ cao hiện đại, phần lớn các linh kiện giao tiếp như PHY Ethernet, bộ thu phát USB, HDMI/DisplayPort, DDR PHY, các transceiver MIPI hay các bộ chuyển đổi ADC/DAC tốc độ cao đều được thiết kế với trở kháng vào và ra danh định, tương thích với các giá trị trở kháng chuẩn của đường truyền trên PCB và cáp kết nối. Các mức trở kháng này

đã được tiêu chuẩn hóa nhằm đảm bảo khả năng truyền dẫn ổn định và tính tương thích giữa các thiết bị. Những giá trị thường gặp bao gồm 50 Ω cho các đường tín hiệu single-ended trong nhiều ứng dụng RF và high-speed, 90 Ω vì sai cho các giao thức như USB, HDMI, MIPI, và 100 Ω vì sai cho Ethernet cũng như nhiều giao thức truyền vi sai khác.

Khi các đường mạch trên PCB không được thiết kế để có trở kháng đặc trưng phù hợp với giá trị mà linh kiện và chuẩn giao thức yêu cầu, sự không khớp trở kháng sẽ xuất hiện tại các điểm chuyển tiếp giữa chip, đường truyền và tải. Hiện tượng này làm tăng hệ số phản xạ Γ , dẫn đến các vấn đề như overshoot, ringing, méo dạng sườn tín hiệu, suy giảm biên độ và sai lệch thời điểm lấy mẫu tại phía thu. Do đó, trong thiết kế PCB tốc độ cao, các đường truyền tín hiệu luôn được thiết kế theo nguyên tắc kiểm soát trở kháng (controlled impedance) với các giá trị chuẩn như 50 Ω , 90 Ω , 100 Ω , ... nhằm khai thác tối đa đặc tính trở kháng đã được chuẩn hóa.

3.4.1.3 Kiểm soát trở kháng

Trở kháng đặc trưng và các chuẩn thông dụng Trở kháng đặc trưng Z_0 của đường truyền trên PCB là một đại lượng phân bố liên tục dọc theo chiều dài đường mạch, phản ánh mối quan hệ giữa điện áp và dòng điện của sóng lan truyền trên đường dẫn. Giá trị này phụ thuộc trực tiếp vào hình học của trace như chiều rộng, độ dày đồng, khoảng cách tới mặt phẳng tham chiếu, cấu hình đường truyền (microstrip hoặc stripline), cũng như các thông số điện môi của stack-up PCB. Để tín hiệu tốc độ cao lan truyền ổn định và hạn chế phản xạ, trở kháng đặc trưng cần được duy trì đồng đều trên toàn bộ chiều dài đường truyền, bao gồm cả các đoạn chuyển tiếp như via hoặc vùng breakout linh kiện.

Trong thực tế, các giá trị trở kháng mục tiêu đã được chuẩn hóa theo từng giao thức truyền thông và loại đường dẫn nhằm đảm bảo khả năng tương thích giữa linh kiện phát-thu, đường mạch PCB và cáp kết nối. Một số chuẩn trở kháng phổ biến thường được áp dụng trong thiết kế PCB tốc độ cao được liệt kê trong Bảng 3.1.

Giao thức/Chuẩn	Loại đường truyền	Trở kháng mục tiêu
RF, tín hiệu high-speed tổng quát	Single-ended	50 Ω
USB 2.0/3.x, HDMI	Differential	90 Ω
Ethernet (10/100/1000/10G)	Differential	100 Ω
PCIe, DDR3/DDR4	Differential	100 Ω
CAN bus, RS-485	Differential	120 Ω
LVDS	Differential	100 Ω
SATA	Differential	100 Ω

Bảng 3.1: Các giá trị trở kháng đặc trưng theo các chuẩn giao thức truyền thông phổ biến.

Mỗi chuẩn giao thức đều quy định chặt chẽ giá trị trở kháng danh định và dung sai cho phép nhằm đảm bảo đặc tính truyền dẫn và khả năng liên thông giữa các thiết bị. Chẳng hạn, các PHY Ethernet thường yêu cầu cặp đường vi sai có trở kháng 100 Ω với dung sai khoảng $\pm 10\%$, trong khi các bộ điều khiển USB được thiết kế để hoạt động tối ưu với đường truyền vi sai 90 Ω từ phía phát đến phía thu. Việc tuân thủ đúng các giá trị trở kháng này là điều kiện tiên quyết để giảm phản xạ, hạn chế méo tín hiệu và đảm bảo độ tin cậy của hệ thống truyền thông tốc độ cao.

Các yếu tố quyết định trở kháng của đường dây Trở kháng đặc trưng Z_0 của đường truyền microstrip (trace đặt trên bề mặt PCB) và stripline (trace nằm kẹp giữa hai mặt phẳng tham chiếu) được xác định từ các bài toán trường điện từ phân bố, trong đó điện trường và từ trường lan truyền trong môi trường điện môi xung quanh trace. Mặc dù các công thức chính xác tương đối phức tạp, giá trị Z_0 có thể được phân tích định tính thông qua một số yếu tố chi phối chính sau đây:

1. Hằng số điện môi ϵ_r : Là tham số đặc trưng của vật liệu PCB, có quan hệ tỷ lệ nghịch với trở kháng đặc trưng. Vật liệu FR4 thông dụng có ϵ_r vào khoảng 4.2–4.6, trong khi các vật liệu chuyên dụng cho tốc độ cao như Rogers RO4000 có ϵ_r thấp hơn, khoảng 3.3–3.5, cho phép đạt cùng giá trị Z_0 với trace rộng hơn và suy hao thấp hơn.
2. Độ dày điện môi H : Là khoảng cách từ trace đến mặt phẳng tham chiếu gần nhất, có quan hệ tỷ lệ thuận với Z_0 . Với cấu trúc microstrip, H là khoảng cách từ trace lớp trên đến ground plane phía dưới; với stripline, trace nằm giữa hai plane nên có hai khoảng cách điện môi gần như đối xứng.
3. Chiều rộng trace W : Có ảnh hưởng mạnh và tỷ lệ nghịch với Z_0 . Khi chiều rộng trace giảm, mật độ điện trường tăng lên và dẫn đến trở kháng đặc trưng cao hơn; ngược lại, trace rộng hơn sẽ làm giảm Z_0 .
4. Độ dày lớp đồng T : Ảnh hưởng ở mức thứ yếu so với W và H , thường có quan hệ tỷ lệ nghịch nhẹ với Z_0 . Trong thực tế, độ dày đồng phổ biến là 1 oz (khoảng $35 \mu\text{m}$) hoặc 0.5 oz (khoảng $17 \mu\text{m}$).
5. Cấu hình mặt phẳng tham chiếu: Microstrip có ưu điểm dễ chế tạo và dễ định tuyến nhưng chịu ảnh hưởng nhiều hơn từ môi trường bên ngoài và lớp solder mask; stripline được che chắn tốt hơn bởi các plane xung quanh, cho trở kháng ổn định và EMI thấp hơn, nhưng việc định tuyến và kiểm soát stack-up phức tạp hơn.
6. Đối với cặp tín hiệu vi sai: Ngoài các yếu tố hình học của từng trace, khoảng cách giữa hai trace S đóng vai trò quyết định đến mức độ ghép điện dung và điện cảm giữa chúng. Trở kháng vi sai Z_{diff} thường được biểu diễn gần đúng theo quan hệ

$$Z_{\text{diff}} \approx 2Z_0(1 - k),$$

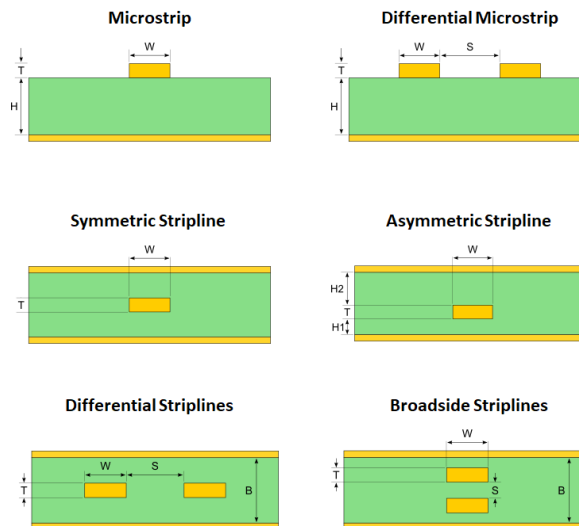
trong đó k là hệ số ghép, phụ thuộc vào khoảng cách S và cấu trúc stack-up.

Trong thực tế thiết kế, các tham số hình học và vật liệu hiếm khi được tính toán hoàn toàn bằng tay mà thường được tối ưu hóa thông qua các công cụ field solver tích hợp trong phần mềm thiết kế PCB hoặc công cụ chuyên dụng. Các công cụ này cho phép mô phỏng chính xác trở kháng đặc trưng dựa trên stack-up cụ thể của nhà sản xuất PCB, từ đó đảm bảo sai số nằm trong dung sai cho phép của chuẩn giao tiếp.

Đối với đường microstrip single-ended, một công thức gần đúng thường được sử dụng trong giai đoạn ước lượng ban đầu là:

$$Z_0 \approx \frac{87}{\sqrt{\epsilon_r + 1.41}} \ln \left(\frac{5.98H}{0.8W + T} \right) \quad (\text{Ohm}),$$

trong đó ϵ_r là hằng số điện môi của vật liệu, H là độ dày điện môi, W là chiều rộng trace và T là độ dày lớp đồng. Công thức này cho phép đánh giá nhanh xu hướng ảnh hưởng của các tham số hình học trước khi tiến hành mô phỏng chi tiết.



Hình 3.6: Minh họa một số cách định tuyến trace trên PCB ảnh hưởng trở kháng

Lý do phải kiểm soát và phối hợp trở kháng Kiểm soát trở kháng là quá trình thiết kế và chế tạo PCB nhằm đảm bảo trở kháng đặc trưng Z_0 của đường truyền thực tế nằm trong một dải dung sai xác định quanh giá trị mục tiêu, thường ở mức $\pm 10\%$ hoặc chặt chẽ hơn là $\pm 5\%$, và được duy trì tương đối đồng đều trên toàn bộ chiều dài đường mạch. Đối với các tín hiệu tốc độ cao, việc kiểm soát này là điều kiện cần để các giả định thiết kế ở mức hệ thống và chuẩn giao tiếp được thỏa mãn trong quá trình vận hành thực tế.

Các mục tiêu chính của việc kiểm soát và phối hợp trở kháng bao gồm:

- Giảm thiểu phản xạ tín hiệu: Khi Z_0 của đường truyền được duy trì ổn định và phù hợp với trở kháng của nguồn phát và tải, hệ số phản xạ Γ tiệm cận về 0. Nhờ đó, các hiện tượng như overshoot, ringing và méo dạng tín hiệu được hạn chế, giúp cải thiện biên độ tín hiệu hiệu dụng và chất lượng eye diagram.
- Tối ưu hóa quá trình truyền năng lượng tín hiệu: Trong điều kiện trở kháng nguồn, đường truyền và tải được khớp với nhau, năng lượng của xung tín hiệu được truyền đến tải một cách hiệu quả hơn, giảm suy hao do phản xạ và cải thiện độ ổn định của mức logic tại đầu thu.
- Đảm bảo tương thích với các chuẩn giao thức tốc độ cao: Nhiều chuẩn truyền thông như USB, Ethernet hay PCIe được xây dựng dựa trên các giả định trở kháng chuẩn hóa. Việc kiểm soát trở kháng đúng giúp duy trì bit error rate thấp, eye diagram mở và đảm bảo các yêu cầu về timing margin theo đặc tả của từng giao thức.

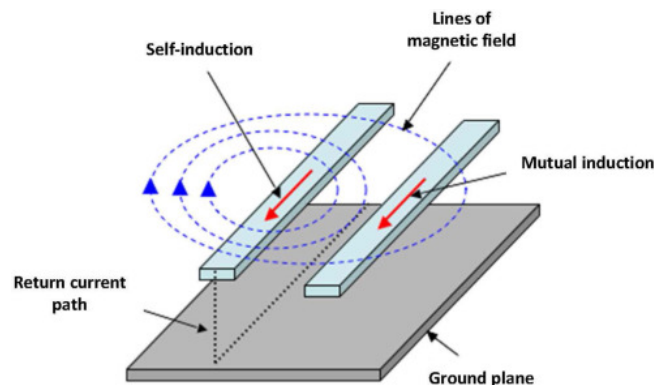
Trong quá trình sản xuất, nhà chế tạo PCB có thể tinh chỉnh các thông số công nghệ như chiều rộng và khoảng cách trace, cấu trúc stack-up cũng như vật liệu điện môi để đảm bảo trở kháng thực tế của đường truyền nằm trong dải sai số cho phép so với giá trị thiết kế. Việc phối hợp chặt chẽ giữa khâu thiết kế và chế tạo là yếu tố quan trọng để đạt được hiệu quả kiểm soát trở kháng trong các hệ thống PCB tốc độ cao.

3.4.1.4 Quy tắc bố trí để giảm nhiễu, crosstalk và cải thiện EMI

Giảm nhiễu xuyên kênh (crosstalk) Nhiễu xuyên kênh là hiện tượng nhiễu không mong muốn phát sinh do sự ghép điện dung và ghép cảm ứng giữa các đường tín hiệu đặt gần nhau

trên PCB. Trong các hệ thống tốc độ cao, crosstalk có thể làm méo dạng xung, gây jitter, làm thu hẹp eye diagram và trong trường hợp nghiêm trọng có thể dẫn đến lỗi dữ liệu trên các đường tín hiệu lân cận. Mức độ crosstalk phụ thuộc vào khoảng cách giữa các trace, chiều dài đoạn chạy song song, cấu trúc stack-up và chất lượng của đường trở về dòng tín hiệu. Một số biện pháp thường được áp dụng để hạn chế crosstalk bao gồm:

- Tăng khoảng cách giữa các đường tín hiệu, đặc biệt là giữa các tín hiệu tốc độ cao độc lập; một quy tắc kinh nghiệm phổ biến là duy trì khoảng cách tối thiểu lớn hơn hoặc bằng ba lần chiều cao điện môi đến mặt phẳng tham chiếu hoặc ba lần chiều rộng trace.
- Đối với các cặp tín hiệu vi sai, duy trì hai trace chạy song song, sát nhau và có khoảng cách không đổi nhằm tăng cường ghép vi sai, đồng thời làm suy giảm thành phần nhiễu chế độ chung.
- Hạn chế các đoạn chạy song song dài giữa tín hiệu nhạy cảm và các tín hiệu có mức nhiễu cao; trong trường hợp các đường buộc phải giao nhau, ưu tiên bố trí vuông góc giữa các lớp khác nhau để giảm ghép điện từ.
- Sử dụng mặt phẳng tham chiếu liên tục và đảm bảo đường trở về của dòng tín hiệu ngắn, ổn định nhằm giảm ghép cảm ứng giữa các đường truyền.



Hình 3.7: Minh họa hiện tượng nhiễu xuyên kênh giữa các đường truyền đặt gần nhau

Cải thiện khả năng EMI/EMC Nhiễu điện từ và khả năng tương thích điện từ là các vấn đề thường gặp trong thiết kế PCB tốc độ cao, do các cạnh tín hiệu nhanh tạo ra phổ tần rộng và dễ phát xạ năng lượng điện từ ra môi trường xung quanh. Layout và routing không hợp lý có thể làm tăng đáng kể mức EMI và ảnh hưởng đến hoạt động của các khối mạch khác trong hệ thống. Để cải thiện đặc tính EMI/EMC, một số nguyên tắc thiết kế high-speed thường được áp dụng như sau:

- Giảm diện tích hồi tiếp dòng bằng cách định tuyến tín hiệu ngay trên hoặc sát mặt phẳng tham chiếu liên tục, đồng thời bổ sung via mass khi tín hiệu phải chuyển lớp để duy trì đường trở về liên tục.
- Tránh định tuyến các đường tín hiệu tốc độ cao chạy dọc theo mép PCB trong khoảng dài và không được che chắn; khi cần giảm phát xạ, ưu tiên sử dụng các lớp trong với cấu trúc stripline.
- Hạn chế số lượng via không cần thiết trên các đường high-speed, do via có thể tạo ra gián đoạn trở kháng; đối với PCB dày hoặc tín hiệu tốc độ rất cao, kỹ thuật backdrill có thể được áp dụng để giảm stub của via.

- Phân vùng rõ ràng giữa các miền digital, analog và RF; bố trí các tụ decoupling, tụ bypass và mạch lọc gần chân cấp nguồn của các IC nhạy cảm để giảm nhiễu dẫn.

Một số nguyên tắc layout và routing tiêu biểu cho mạch tốc độ cao Thiết kế layout và routing cho PCB tốc độ cao cần được xem xét ngay từ giai đoạn đầu của quá trình thiết kế, với các ràng buộc rõ ràng về tín hiệu, lớp mạch và cấu trúc stack-up. Một số nguyên tắc tiêu biểu bao gồm:

- Xây dựng cấu trúc stack-up ngay từ đầu, bố trí các lớp tín hiệu xen kẽ với các lớp plane và xác định rõ những lớp dành riêng cho tín hiệu tốc độ cao.
- Áp dụng quy tắc matching độ dài cho các cặp vi sai và các bus song song nhằm giảm lệch thời gian truyền (skew) giữa các đường tín hiệu.
- Tuân thủ nghiêm ngặt các ràng buộc về chiều rộng trace, khoảng cách, lớp định tuyến và cấu trúc via theo yêu cầu của từng giao thức truyền thông cụ thể.
- Tránh các góc gãy 90 độ trên đường tín hiệu tốc độ cao; thay vào đó sử dụng hai góc 45 độ hoặc đường bo cong để giảm gián đoạn trở kháng và hạn chế phát xạ nhiễu.

3.4.2 Power Distribution Network – PDN

Mạng phân phối điện (Power Distribution Network – PDN) bao gồm tất cả các kết nối từ Voltage Regulator Module (VRM) đến pad nguồn của chip và lớp kim loại trên die, đảm nhận việc phân phối dòng điện và đường hồi tiếp (return path). PDN bao gồm VRM, tụ khối (bulk decoupling capacitors), via, trace, plane trên PCB, tụ gắn trên board, kết nối package, wire bond / bóng hàn C4, và các kết nối trên chip.

Việc kiểm soát PDN là cực kỳ quan trọng đối với các SoC hiệu năng cao, đặc trưng bởi dòng điện tiêu thụ lớn, điện áp thấp và tần số hoạt động cao. Trở kháng phức tạp của PDN phải được kiểm soát tại pad nguồn của chip. Do dòng tiêu thụ của SoC thường mang dạng sóng vuông (square wave) thay vì tuyến tính, nên sẽ xuất hiện độ gợn (ripple) điện áp do transistor chuyển mạch theo tần số hoạt động. Vì vậy, trở kháng PDN cần được duy trì dưới giá trị trở kháng mục tiêu Z_{target} :

$$Z_{\text{target}} = \frac{V_{dd} \times \Delta V_{\text{target}}}{I_{\text{max}}} \quad (3.1)$$

Trong đó:

- V_{dd} : Điện áp cung cấp (Volt).
- ΔV_{target} : Điện áp ripple mục tiêu trên rail nguồn (%).
- I_{max} : Dòng đỉnh lớn nhất trong trường hợp xấu nhất (Amps).

Kiểm soát trở kháng PDN được phân theo các dải tần số khác nhau, mỗi dải chịu ảnh hưởng bởi các thành phần và yêu cầu thiết kế cụ thể:

- **Dải DC đến kHz:**
 - Kiểm soát bởi VRM, một số VRM hiện đại phản hồi tới 10–100 kHz.

- Trở kháng tần số thấp được xác định bởi đặc tính của bộ điều chỉnh điện áp (regulator).

- **Dải kHz đến MHz:**

- Kiểm soát bởi tụ khối (bulk capacitors).
- Giá trị tụ được tính dựa trên tần số f mà VRM không còn cung cấp trở kháng thấp:

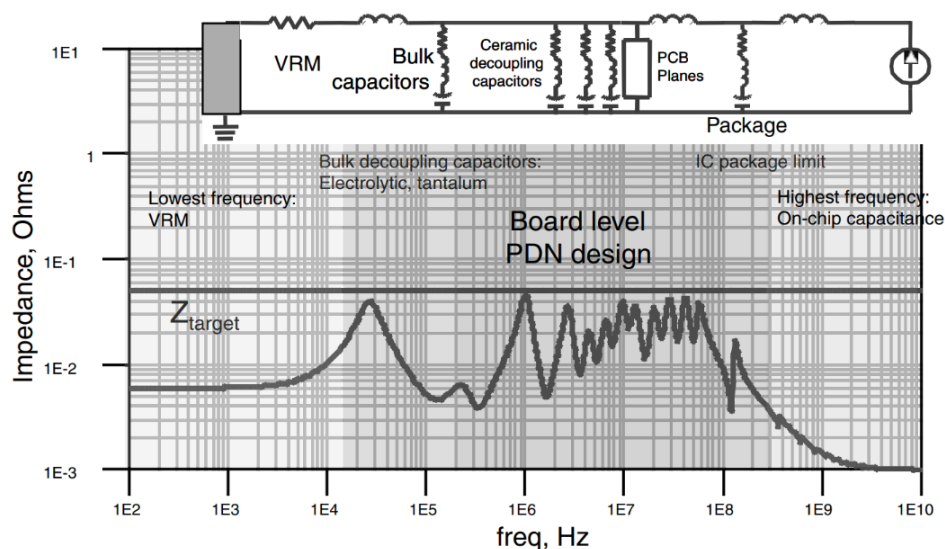
$$C = \frac{1}{2\pi f \times Z_{\text{target}}} \quad (3.2)$$

- **Dải MHz đến 100 MHz:**

- Kiểm soát bởi tụ ceramic decoupling và PCB plane.
- Chọn lựa tụ decoupling phù hợp và số lượng đủ.
- Khoảng cách via đến pad tụ tối thiểu; đường trả về (return path) ngắn.
- Sử dụng tụ decoupling thấp ESL / cao ESR.
- Khoảng cách giữa lớp power/ground mỏng.
- Trace bề mặt rộng, ngắn từ tụ decoupling đến pin nguồn IC.
- Via đường kính lớn, khoảng cách giữa via nguồn và GND là tối thiểu.

- **Trên 100 MHz:**

- Kiểm soát bởi package IC và tụ trên chip (on-chip capacitance).
- Vượt ngoài khả năng điều khiển từ bên ngoài, được nhà cung cấp IC quản lý thông qua thiết kế package.



Hình 3.8: Các phần của PDN được phân tách theo dải tần số mà chúng ảnh hưởng

Chương 4

Phân tích và Thiết kế

4.1 Phân tích yêu cầu

Mục tiêu cốt lõi là thiết kế một IoT Gateway giải quyết bài toán linh hoạt và khả cấu hình nhằm tối ưu chi phí và thời gian triển khai hệ thống cho đa dạng các nhu cầu sử dụng.

4.1.1 Yêu cầu chức năng

Hệ thống Gateway IoT cần đáp ứng các nhóm chức năng chính sau đây nhằm đảm bảo tính linh hoạt và khả năng xử lý dữ liệu tin cậy:

- **Tính mô-đun hóa phần cứng:**

- Thiết kế theo kiến trúc mô-đun, hỗ trợ kết nối linh hoạt và tương thích với đa dạng các loại mô-đun.
- Cung cấp cơ chế thay thế, nâng cấp mô-đun tùy theo mục đích ứng dụng mà không yêu cầu tái thiết kế bo mạch chủ (Baseboard).
- Hỗ trợ các chuẩn giao tiếp ngoại vi phổ thông: UART, SPI, I2C, USB, ...

- **Khả năng thu thập và xử lý dữ liệu:**

- Tiếp nhận dữ liệu từ cảm biến hoặc thiết bị đầu cuối thông qua các giao thức truyền thông tiêu chuẩn.
- Hỗ trợ thu thập đồng thời từ nhiều nhóm thiết bị hoạt động trên các nền tảng khác nhau: Zigbee, LoRa TDMA, CAN, UART/RS-485, ...
- Triển khai các tiến trình xử lý riêng biệt cho từng giao thức (*Zigbee_Handler*, *LoRa_Handler*, *CAN_Handler*, *Data_Communication_Handler*) trên nhân xử lý.
- Quy trình xử lý dữ liệu tại mỗi mô-đun bao gồm: Trích xuất dữ liệu thô; kiểm chứng định dạng và Checksum; loại bỏ khung tin lỗi; gắn nhãn nguồn dữ liệu và đóng dấu thời gian (Time-stamping).
- Chuẩn hóa dữ liệu sau kiểm tra về một định dạng khung nội bộ thống nhất.
- Sử dụng hàng đợi (Queue) để đệm dữ liệu, đảm bảo duy trì dữ liệu khi mất kết nối WAN trong một khoảng thời gian xác định.

- Thiết lập kênh truyền thông nội bộ nhằm đồng bộ hóa nội dung khung dữ liệu và thông tin trạng thái.
- Tiến trình *Internet_Communication_Handler* trên MCU WAN chịu trách nhiệm đóng gói và chuyển giao dữ liệu lên máy chủ thông qua MQTT, HTTP, hoặc CoAP theo cấu hình tùy chọn.

- **Khả năng khả cấu hình:**

- Cung cấp ứng dụng cấu hình trên PC để tinh chỉnh toàn bộ thông số vận hành mà không cần can thiệp vào mã nguồn (Firmware).
- Tích hợp tính năng tự động quét và nhận diện Gateway đang kết nối qua giao tiếp USB/UART.
- Cho phép truy xuất toàn bộ trạng thái cấu hình hiện tại (danh sách mô-đun hiện hữu, thông số mạng WAN, tham số MQTT, ...).
- Giao diện cấu hình trực quan dưới dạng biểu mẫu (Form), tối ưu hóa cho kỹ thuật viên vận hành.
- Hỗ trợ hiệu chỉnh chi tiết: Lựa chọn danh mục mô-đun thực tế; thiết lập tham số lớp vật lý (Baudrate, địa chỉ, ID mạng) cho Zigbee/LoRa/CAN/UART; cấu hình hạ tầng mạng (Wi-Fi/LTE/PPP) và thông số máy chủ MQTT (Host, Port, Credentials, Topics, cấu hình bảo mật TLS); điều chỉnh chu kỳ lấy mẫu, chính sách lưu đệm và cơ chế thử lại (Retry) khi mất kết nối.
- Thực thi lệnh ghi cấu hình xuống Gateway thông qua giao thức truyền tin nội bộ đã định nghĩa.

4.1.2 Yêu cầu phi chức năng

IoT Gateway phải thỏa mãn các tiêu chuẩn vận hành để đáp ứng điều kiện triển khai thực tế:

- **Độ tin cậy và tính ổn định:**

- Đảm bảo khả năng vận hành liên tục 24/7 trong thời gian dài.
- Tích hợp cơ chế giám sát phần cứng và phần mềm (Watchdog Timer) để tự động phục hồi khi xảy ra hiện tượng treo hệ thống hoặc xung đột tiến trình.
- Chủ động phát hiện và xử lý có kiểm soát các lỗi ngoại vi (mất Internet, lỗi cảm biến, lỗi mô-đun giao tiếp) thông qua hệ thống bản ghi (Log) và chuyển đổi về trạng thái an toàn.
- Duy trì tính toàn vẹn của dữ liệu bằng bộ đệm nội bộ, hỗ trợ lưu trữ và gửi bù dữ liệu ngay khi kết nối mạng được khôi phục.
- Thiết kế phần cứng có khả năng chống chịu tác động môi trường: bảo vệ chống tĩnh điện (ESD), lọc nhiễu điện từ (EMI).
- Hỗ trợ cập nhật phần mềm từ xa (FOTA) an toàn cho nhân xử lý với cơ chế kiểm tra tính toàn vẹn, đảm bảo không ghi đè lên Firmware hiện tại khi chưa xác thực thành công bản cập nhật.

- **Nguồn điện và năng lượng:**

- Hệ thống nguồn yêu cầu độ ổn định cao, tích hợp các mạch bảo vệ quá áp (OVP), bảo vệ ngắn mạch (SCP) và chống ngược cực.
- Tối ưu hóa mức tiêu thụ năng lượng thông qua các chế độ tiết kiệm điện (Sleep mode) của vi điều khiển, phù hợp cho các ứng dụng vận hành bằng pin hoặc năng lượng tái tạo.

- **Hiệu năng:**

- Năng lực xử lý trung tâm đảm bảo độ trễ thấp trong việc thu thập và chuyển giao dữ liệu từ tất cả các mô-đun kết nối đồng thời.
- Tốc độ phản hồi giao diện cấu hình và thời gian khởi động hệ thống tối ưu, đảm bảo không gây tắc nghẽn hàng đợi dữ liệu khi mật độ thiết bị đầu cuối tăng cao.

- **Tính khả dụng và dễ sử dụng:**

- Thiết kế cơ khí hỗ trợ tháo lắp mô-đun nhanh chóng, giảm thiểu yêu cầu về công cụ chuyên dụng.
- Giao diện phần mềm thân thiện, lược bỏ các thao tác lập trình phức tạp, cho phép người dùng cấu hình thông qua các tùy chọn có sẵn.
- Hệ thống chỉ thị trực quan qua đèn LED (trạng thái nguồn, kết nối LAN/WAN, trạng thái Server, lỗi hệ thống). Duy trì nhật ký sự kiện chi tiết và cung cấp bộ tài liệu hướng dẫn vận hành, xử lý sự cố (Troubleshooting) toàn diện.

4.1.3 Các ràng buộc thiết kế

- **Ràng buộc về chi phí:** Cần đảm bảo chi phí sản xuất và triển khai gateway trên thị trường không quá cao, tối ưu hóa lựa chọn phần cứng và phần mềm sao cho tiết kiệm mà vẫn đáp ứng được yêu cầu hệ thống.
- **Ràng buộc về công nghệ:** Cần phải sử dụng các công nghệ phổ biến và đáng tin cậy, đảm bảo khả năng tương thích với các thiết bị hiện có, cũng như dễ dàng tích hợp vào các nền tảng cloud hoặc hệ thống IoT.
- **Ràng buộc về hiệu suất:** Gateway phải đảm bảo tốc độ truyền dữ liệu, khả năng xử lý song song và đáp ứng các yêu cầu về độ trễ thấp trong quá trình thu thập và truyền tải dữ liệu.
- **Ràng buộc về khả năng mở rộng:** Hệ thống phải được thiết kế sao cho dễ dàng mở rộng hoặc thay thế các module mà không làm gián đoạn hoạt động của hệ thống, hỗ trợ thêm giao thức hoặc phần cứng khi cần thiết.

4.2 Giải pháp đề xuất

Nhằm hiện thực hóa mục tiêu xây dựng một IoT Gateway có tính linh hoạt cao, khả năng cấu hình linh hoạt, đồng thời tối ưu hóa chi phí sản xuất và rút ngắn thời gian triển khai cho các kịch bản ứng dụng đa dạng, giải pháp đề xuất là tích hợp giữa kiến trúc phần cứng mô-đun và hệ thống xử lý phân tán.

4.2.1 Giải pháp phần cứng

Giải pháp phần cứng tập trung vào việc xây dựng một hệ sinh thái mô-đun hóa toàn diện, cho phép tối ưu hóa khả năng tích hợp, bảo trì và nâng cấp hệ thống mà không yêu cầu thay đổi thiết kế cốt lõi.

- **Thiết kế Bo mạch chủ (Baseboard) chuẩn hóa:** Xây dựng Baseboard đóng vai trò là nền tảng trung tâm, tích hợp các khe cắm với giao tiếp vật lý và giao thức được chuẩn hóa, tạo điều kiện kết nối đồng bộ với các loại Module Adapter khác nhau.
- **Phát triển hệ thống Module Adapter linh hoạt:** Các Module Adapter được thiết kế để đóng vai trò là cầu nối trung gian, có nhiệm vụ chuyển đổi và chuẩn hóa các chuẩn giao tiếp của các mô-đun thương mại hiện có trên thị trường về giao thức nội bộ thống nhất của Gateway.
- **Tối ưu hóa hiệu năng và chi phí bằng giải pháp Vi điều khiển (MCU):** Thay vì sử dụng các bộ vi xử lý (CPU) hiệu năng cao vốn làm tăng đáng kể giá thành linh kiện và chi phí gia công PCB (do yêu cầu về mạch in nhiều lớp phức tạp), nhóm quyết định sử dụng dòng vi điều khiển (MCU) làm nhân xử lý chính dựa trên các tiêu chí:
 - Cân bằng giữa giá thành thấp và hiệu năng xử lý đủ mạnh mẽ, sở hữu tài nguyên phần cứng đủ nhiều và hỗ trợ đa dạng các giao thức ngoại vi.
 - Hệ sinh thái thư viện mã nguồn mở dồi dào và cộng đồng hỗ trợ lớn, giúp đẩy nhanh tốc độ phát triển và tối ưu hóa mã nguồn (Firmware).
 - Đặc tính tiêu thụ năng lượng thấp, đáp ứng tiêu chuẩn khắt khe của các thiết bị IoT vận hành liên tục hoặc sử dụng nguồn năng lượng hạn chế.
- **Kiến trúc xử lý Dual-MCU:** Để giải quyết giới hạn về tài nguyên bộ nhớ khi phải tích hợp đồng thời nhiều thư viện giao thức phức tạp, nhóm đề xuất kiến trúc sử dụng hai MCU chuyên biệt hóa nhiệm vụ:
 - **WAN-MCU (Master):** Chịu trách nhiệm quản lý luồng dữ liệu mạng diện rộng (WAN), điều phối toàn bộ hoạt động của hệ thống và quản trị các tác vụ cấp cao.
 - **LAN-MCU (Slave):** Tập trung xử lý các giao tiếp mạng nội bộ (LAN), quản lý việc thu thập dữ liệu từ các mô-đun ngoại vi và thiết bị đầu cuối.

Hai khối xử lý này liên kết chặt chẽ thông qua giao thức truyền thông tốc độ cao và hệ thống tín hiệu điều khiển logic, đảm bảo tính đồng bộ và độ trễ thấp trong quá trình trao đổi thông tin.

4.2.2 Giải pháp phần mềm

Giải pháp phần mềm được xây dựng nhằm đáp ứng các yêu cầu cốt lõi của hệ thống IoT Gateway bao gồm: tính module hóa (Modularity), khả năng cấu hình linh hoạt (Configurability), độ ổn định – tin cậy cao, khả năng mở rộng và dễ bảo trì.

Phần mềm phải tận dụng hiệu quả kiến trúc **Dual-MCU Master-Slave**, phân tách rõ ràng trách nhiệm giữa hai vi điều khiển để tối ưu tài nguyên phần cứng, đồng thời bảo đảm luồng dữ liệu thông suốt từ thiết bị cảm biến tới nền tảng đám mây.

4.2.2.1 Kiến trúc phần mềm tổng thể

Phần mềm hệ thống được thiết kế theo kiến trúc phân lớp (Layered Architecture) và module hóa, áp dụng nhất quán cho cả hai MCU LAN và MCU WAN. Mỗi firmware được chia thành 4 lớp chính:

- Driver Layer
- Board Support Package (BSP)
- Middleware Layer
- Application Layer

Việc phân lớp rõ ràng giúp:

- Tách biệt logic ứng dụng và phụ thuộc phần cứng
- Dễ dàng mở rộng, thay thế module
- Giảm rủi ro ảnh hưởng dây chuyền khi thay đổi một thành phần

4.2.2.2 Giải pháp phần mềm cho Master WAN-MCU

MCU WAN giữ vai trò Application Master, quản lý WAN, Cloud và điều phối toàn hệ thống.

Application Layer – MCU WAN: Các thành phần chính:

- Main Control: điều phối hệ thống, quản lý trạng thái gateway.
- Internet Communication Handler:
 - Ethernet Handler Task
 - Wi-Fi Handler Task
 - LTE / NB-IoT Handler Task
- Server Communication Handler:
 - MQTT Handler Task
 - HTTP Handler Task (mở rộng)
 - CoAP Handler Task (mở rộng)
- MCU LAN Communication Handler: quản lý kênh Quad SPI, UART và GPIO với MCU LAN.
- FOTA & Bridge Bootloader: cập nhật firmware cho cả hai MCU.

Dữ liệu từ MCU LAN được MCU WAN đóng gói và gửi lên Cloud theo cấu hình (MQTT/HTTP/CoAP).

Middleware Layer – MCU WAN:

- ESP IDF MQTT
- Ethernet / Wi-Fi / LTE / NB-IoT Handler
- ESP OTA Handler

- FreeRTOS

BSP & Driver – MCU WAN:

- BSP: SIM7600, Ethernet Module, MCU LAN Communication, RTC, IO Expander.
- Driver: UART, SPI, USB, I2C, MAC, GPIO, Wi-Fi Driver.

4.2.2.3 Giải pháp phần mềm cho MCU LAN (Slave LAN-MCU)

MCU LAN chịu trách nhiệm chính cho Sensing Domain và LAN Network Domain, tập trung vào thu thập, tiền xử lý và lưu trữ dữ liệu.

Application Layer – MCU LAN: Tầng Application của MCU LAN bao gồm các khối chính:

- Main Control: điều phối hoạt động tổng thể của MCU LAN theo lệnh từ MCU WAN.
- LAN Network Handler: quản lý các mạng cảm biến nội bộ.
- Các Handler Task theo giao thức:
 - LoRa Handler Task
 - Zigbee Handler Task
 - Bluetooth Handler Task
 - Z-Wave Handler Task
 - Thread Handler Task
 - CAN Handler Task
 - Modbus / RS-485 Handler Task (nếu được cấu hình)

Mỗi Handler Task hoạt động như một FreeRTOS task độc lập, đảm bảo:

- Thu thập dữ liệu song song từ nhiều mạng khác nhau
- Tránh blocking giữa các giao thức
- Dễ dàng bật/tắt theo cấu hình mà không cần sửa firmware

Chức năng chung của mỗi Handler Task:

- Đọc dữ liệu thô từ phần cứng
- Kiểm tra định dạng, checksum, loại bỏ khung lỗi
- Gắn nhãn nguồn dữ liệu và timestamp
- Chuẩn hóa dữ liệu về định dạng khung nội bộ
- Đưa dữ liệu vào hàng đợi (queue) để gửi sang MCU WAN

Middleware Layer – MCU LAN: Middleware đóng vai trò trừu tượng hóa giao thức và thư viện nền:

- Zigbee / LoRa / Thread / Z-Wave / Bluetooth Handler
- Data Storage Handler (lưu dữ liệu tạm thời vào SD Card)
- ESP OTA Handler (hỗ trợ cập nhật firmware)
- FreeRTOS Kernel

Lớp này giúp Application không phụ thuộc trực tiếp vào BSP hoặc Driver.

BSP & Driver – MCU LAN:

- BSP quản lý giao tiếp với các module vật lý: Zigbee, LoRa, Thread, RS-485, CAN, SD Card, IO Expander.
- Driver Layer cung cấp driver chuẩn: UART, SPI, I2C, SDIO, GPIO, ADC, USB.

4.2.2.4 Cơ chế cấu hình và cập nhật

- Toàn bộ tham số gateway được lưu trong non-volatile memory.
- Configuration Tools trên PC giao tiếp với gateway qua USB/UART.
- Cho phép:
 - Đọc cấu hình hiện tại
 - Chỉnh sửa module, giao thức, WAN, MQTT
 - Ghi cấu hình mà không cần build lại firmware
- Hỗ trợ FOTA an toàn:
 - Kiểm tra checksum
 - Cập nhật từng MCU
 - Không ghi đè firmware đang chạy

4.2.2.5 Đánh giá giải pháp phần mềm

Giải pháp phần mềm đáp ứng:

- Modularity: bật/tắt module qua cấu hình
- Configurability: cấu hình không cần sửa firmware
- Reliability: watchdog, queue, retry, buffer
- Interoperability: hỗ trợ đa giao thức LAN/WAN
- Scalability: dễ mở rộng thêm protocol hoặc cloud service

4.3 Đặc tả sản phẩm

Sản phẩm IoT Gateway dạng mô-đun được thiết kế nhằm tối ưu hóa tính linh hoạt và khả năng cấu hình cho các kịch bản ứng dụng IoT đa dạng:

4.3.1 Thông số phần cứng và Hiệu năng xử lý

Hệ thống áp dụng kiến trúc Dual-MCU Master-Slave để phân tách nhiệm vụ giữa miền mạng diện rộng (WAN) và miền thu thập dữ liệu nội bộ (LAN).

- **Vi điều khiển chính (Master WAN-MCU):** ESP32-S3-WROOM-1U-N16R8.
 - CPU: Xtensa® lõi kép 32-bit LX7, xung nhịp lên đến 240 MHz.
 - Bộ nhớ: 16 MB Flash và 8 MB PSRAM tích hợp.
 - Kết nối không dây: Tích hợp Wi-Fi 2.4 GHz và Bluetooth Low Energy (BLE).
 - Vai trò: Điều phối hệ thống, quản lý kết nối Cloud và thực hiện cập nhật FOTA cho cả hai MCU.
- **Vi điều khiển phụ (Slave LAN-MCU):** ESP32-S3-WROOM-1U-N16R8.
 - Vai trò: Quản lý các giao thức mạng nội bộ, thu thập dữ liệu cảm biến và lưu trữ cục bộ.
- **Giao tiếp nội bộ (Inter-MCU):**
 - Quad SPI: Kênh dữ liệu tốc độ cao để trao đổi khung tin dữ liệu và yêu cầu.
 - UART: Kênh chuyên biệt dành cho luồng nạp Firmware liên chip (Passthrough Flashing).
 - GPIO: Các đường tín hiệu điều khiển và ngắt (Boot, Reset, Interrupt) để đồng bộ trạng thái.

4.3.2 Khả năng kết nối và Mở rộng mô-đun

- **Khe cắm mở rộng:** Gồm 03 khe cắm chuẩn hóa (01 WAN Module, 02 LAN Module) hỗ trợ thay thế linh hoạt.
- **Giao diện tại khe cắm:** Hỗ trợ đầy đủ các giao tiếp UART, SPI, I2C, USB 2.0 và các chân GPIO chức năng.
- **Các mô-đun Adapter hỗ trợ:**
 - WAN: SIM7600G-H (LTE 4G/NB-IoT), Ethernet. . .
 - LAN: Zigbee, LoRa, RS-485, CAN Bus, và mô-đun đa giao thức Bluetooth/Thread. . .
- **Lưu trữ cục bộ:** Khe cắm thẻ nhớ Micro SD hỗ trợ giao tiếp SDIO 4-bit phục vụ việc ghi nhật ký dữ liệu ngoại tuyến.

4.3.3 Đặc tả Hệ thống nguồn và Bảo vệ

Mạng phân phối nguồn (PDN) được thiết kế phân tầng để đảm bảo tính toàn vẹn năng lượng cho toàn hệ thống.

- **Nguồn đầu vào:** DC 12V, phù hợp với hạ tầng công nghiệp.
- **Các mức điện áp hệ thống:** Cung cấp đồng thời các ray nguồn +5V0_SYS, +5V0, +3V3, và +1V8 phục vụ các khối chức năng và mô-đun cắm thêm.
- **Cơ chế Bảo vệ phần cứng:**
 - Electronic Fuse (eFuse): Sử dụng eFuse để bảo vệ quá dòng và quá áp, giới hạn dòng đỉnh tại có điều khiển.
 - Chống tĩnh điện (ESD): Trang bị TVS Diode tại đầu vào và TVS chuyên dụng cho các đường dữ liệu.
 - Lọc nhiễu: Hệ thống tụ Decoupling đa tầng đặt sát chân linh kiện để triệt tiêu nhiễu cao tần.

4.3.4 Giao diện người dùng và Đặc điểm cơ khí

- **Hiển thị:** Màn hình OLED 0.96 inch (Driver SH1107), giao tiếp I2C để thông báo trạng thái.
- **Chỉ thị trạng thái:** Hệ thống đèn LED báo hiệu trạng thái nguồn, kết nối LAN/WAN và trạng thái máy chủ.
- **Quản lý thời gian:** IC RTC kèm pin dự phòng CR2032 giúp duy trì thời gian thực khi mất điện.
- **Tương tác vật lý:** Nút nhấn Reset và nút nhấn điều khiển nguồn tích hợp.

4.3.5 Đặc tả Phần mềm

- **Hệ điều hành nhúng:** Chạy trên nền tảng FreeRTOS, hỗ trợ lập lịch đa nhiệm và xử lý song song.
- **Giao thức truyền dữ liệu:** Hỗ trợ MQTT, HTTP, và CoAP theo cấu hình linh hoạt.
- **Khả năng cấu hình:** Sử dụng công cụ Configuration Tools trên PC để thiết lập tham số (chu kỳ gửi, thông số mạng, lựa chọn mô-đun) qua USB/UART mà không cần nạp lại Firmware.
- **Tính an toàn và ổn định:** Tích hợp Watchdog Timer tự động phục hồi khi có sự cố treo hệ thống và cơ chế lưu đệm hàng đợi (Queue) khi mất kết nối mạng.

4.4 Khảo sát và Thử nghiệm

4.4.1 Khảo sát các mô hình kiến trúc Gateway IoT phổ biến

Qua khảo sát các gateway IoT thương mại và các nền tảng triển khai thực tế, kiến trúc gateway hiện nay thường rơi vào ba nhóm chính:

1. Gateway Linux/SoC (CPU hiệu năng cao)

- Loại này dùng SoC/CPU (ARM Cortex-A...) chạy Linux, ưu điểm là tài nguyên lớn, dễ tích hợp nhiều giao thức, hỗ trợ tốt các framework cloud/edge (MQTT broker cục bộ, container, phân tích dữ liệu...).
- Tuy nhiên, nhược điểm là chi phí BOM và PCB cao, tiêu thụ năng lượng lớn, thời gian khởi động dài và độ phức tạp phần mềm cao. Với các bài toán chỉ cần thu thập – tiền xử lý – đẩy dữ liệu lên server, nền tảng Linux thường dư thừa tài nguyên, khó đạt mục tiêu tối ưu chi phí và thời gian triển khai.

2. Gateway một MCU (Single-MCU Gateway)

- Ưu điểm nổi bật là rẻ, gọn, tiết kiệm điện. Tuy vậy, khi cần hỗ trợ đồng thời nhiều giao thức LAN (Zigbee/LoRa/CAN/RS-485...) và WAN (Wi-Fi/LTE/Ethernet) cộng thêm giao thức ứng dụng (MQTT/HTTP/CoAP), một MCU đơn dễ gặp giới hạn tài nguyên: CPU/RAM/Flash, đồng thời dễ xảy ra blocking giữa các tác vụ I/O và tác vụ mạng. Điều này làm giảm tính ổn định và làm hệ thống khó mở rộng.

3. Gateway đa MCU nhưng thiếu chuẩn hóa phân vai

- Một số giải pháp tách nhiều MCU nhưng phân chia nhiệm vụ chưa rõ (thường do phát sinh từ yêu cầu phần cứng). Khi đó, giao tiếp giữa các MCU thiếu chuẩn hóa, phần mềm phụ thuộc chặt vào phần cứng, khó bảo trì và khó mở rộng theo hướng module hóa.

Từ khảo sát trên có thể thấy: để đạt được yêu cầu đa giao thức + ổn định + tiết kiệm chi phí, cần một kiến trúc phân vai rõ ràng theo miền chức năng. Đây là cơ sở để đề án chọn kiến trúc Dual-MCU Master-Slave:

- MCU LAN chuyên thu thập/tiền xử lý và quản lý mạng cảm biến nội bộ.
- MCU WAN chuyên kết nối Internet, giao tiếp server và điều phối toàn hệ thống.

4.4.2 Khảo sát mức độ module hóa phần cứng và khả năng mở rộng giao thức

Về phần cứng, nhiều gateway hiện nay tích hợp cố định các module (Zigbee/LoRa/LTE/ETH) lên board chính. Cách này tiện cho sản xuất nhưng hạn chế lớn ở chỗ:

- Không linh hoạt theo từng bài toán triển khai (dùng ít vẫn phải mua đủ).
- Khó thay thế/nâng cấp theo nhu cầu thực tế.
- Khi thay đổi yêu cầu ứng dụng thường phải thay cả thiết bị.

Một số gateway sử dụng các chuẩn module như Mini-PCIe/M.2 để thay LTE/5G... giúp nâng cao tính thay thế. Tuy nhiên, chi phí module cao và thiết kế phần cứng phức tạp; đồng thời các module cảm biến/fieldbus phổ biến (RS-485, CAN, LoRa, Zigbee dạng rời) vẫn không đồng nhất chuẩn cơ khí – chân – giao thức.

Từ thực tế đó, hướng tiếp cận phù hợp là chuẩn hóa giao diện (UART/SPI/I2C/USB/GPIO) thay vì phụ thuộc vào chuẩn module vật lý. Vì vậy, giải pháp Module Adapter trong đề án giúp:

- Dùng được module thị trường đa dạng (không theo chuẩn thống nhất).

- Chuẩn hóa kết nối về một chuẩn gateway nội bộ.
- Dễ tháo lắp, thay thế theo nhu cầu (đúng mục tiêu Modularity).

4.4.3 Khảo sát kiến trúc phần mềm gateway: monolithic vs phân lớp/module

Về phần mềm, các gateway có thể chia làm hai xu hướng:

1. Monolithic (một khối chương trình lớn)

- Các module giao thức, quản lý cấu hình, xử lý dữ liệu và truyền cloud được “trộn” chung. Nhược điểm là:
 - Khó bảo trì, khó kiểm thử theo module.
 - Khi thêm giao thức mới phải sửa nhiều phần.
 - Dễ tạo lỗi dây chuyền và khó mở rộng.

2. Phân lớp và module hóa (Layered + Modular)

- Phần mềm tách lớp rõ: Driver → BSP → Middleware → Application; mỗi giao thức được đóng gói thành module/handler riêng. Kiểu này phù hợp với hệ thống cần mở rộng giao thức và phù hợp với xu hướng Modularity/Interoperability.

Đối chiếu với 2 sơ đồ firmware đã xây dựng:

- Trên MCU LAN, tầng Application tổ chức thành các Handler Task (Zigbee, LoRa, BLE, Thread, Z-Wave, CAN, RS-485...) chạy dưới FreeRTOS; tầng Middleware cung cấp các handler tương ứng và khối lưu trữ (Data Storage) + OTA.
- Trên MCU WAN, tầng Application tách rõ Internet Communication Handler (Ethernet, Wi-Fi, LTE, NB-IoT), Server Communication Handler (MQTT, HTTP, CoAP) và MCU LAN Comm Handler để nhận dữ liệu từ MCU LAN và đẩy lên server.

Qua khảo sát, kiến trúc phân lớp + handler theo giao thức là phù hợp để đảm bảo:

- Dễ mở rộng giao thức
- Chạy song song, giảm blocking
- Dễ cấu hình bật/tắt module theo yêu cầu triển khai

4.4.4 Khảo sát cơ chế cấu hình và cập nhật firmware trong triển khai thực tế

Một điểm “nghẽn” thường gặp của gateway là cấu hình và bảo trì. Nhiều thiết bị hiện nay vẫn cấu hình theo kiểu “cứng” trong firmware: thay đổi module/giao thức/WAN/MQTT phải chỉnh code và nạp lại firmware. Cách này:

- Tốn thời gian triển khai
- Phụ thuộc kỹ sư firmware
- Không phù hợp cho kỹ thuật viên vận hành tại hiện trường

Do đó, xu hướng phù hợp hơn là cấu hình động bằng công cụ (Configuration Tools) qua USB/UART để:

- Quét thiết bị, đọc cấu hình hiện tại
- Chỉnh tham số theo form
- Ghi cấu hình xuống gateway mà không sửa firmware

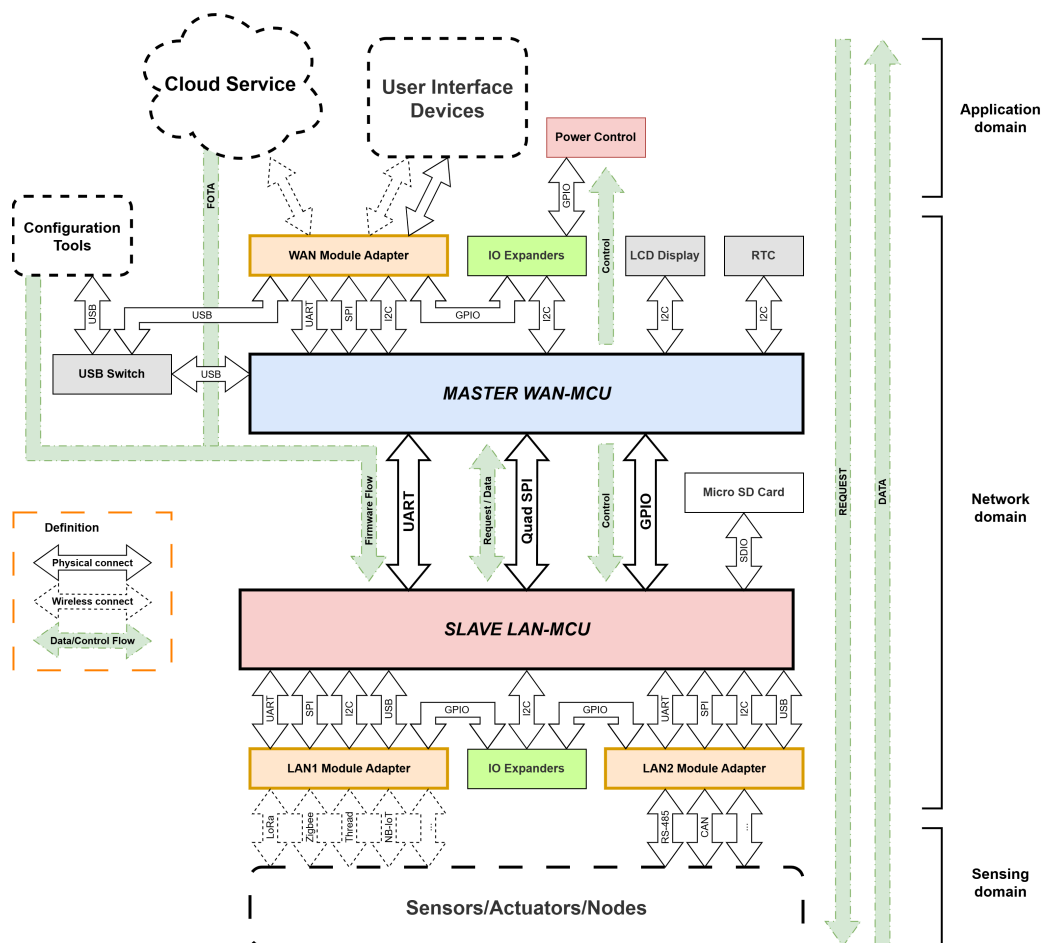
Về cập nhật firmware, khảo sát cũng cho thấy FOTA nếu không có cơ chế an toàn sẽ gây rủi ro “brick” thiết bị khi mất điện/mất mạng. Vì vậy, giải pháp cần có:

- Kiểm tra tính toàn vẹn (checksum)
- Cơ chế cập nhật có kiểm soát (không ghi đè firmware đang chạy khi chưa xác thực)
- Phân luồng cập nhật cho cả WAN-MCU và LAN-MCU (vì hệ thống dual MCU)

4.5 Thiết kế

4.5.1 Thiết kế Kiến trúc Hệ thống (System Architecture Design)

4.5.1.1 Sơ đồ khối Hệ thống



Hình 4.1: Sơ đồ khối Hệ thống

4.5.1.2 Mô tả thành phần hệ thống

Hệ thống được thiết kế theo mô hình Dual-MCU Master-Slave, chia thành 3 miền chức năng chính: Application Domain (Ứng dụng), Network Domain (Mạng lưới xử lý), và Sensing Domain (Cảm biến).

Baseboard: Là board chính của hệ thống, chịu trách nhiệm cung cấp nguồn điện ổn định, giao tiếp giữa các thành phần, kết nối với Module Adapter, và chứa các vi điều khiển chính.

Khối Xử lý trung tâm: Hệ thống sử dụng kiến trúc xử lý song song với hai vi điều khiển chuyên biệt nhằm tối ưu hóa hiệu năng và tài nguyên phần cứng:

- **MASTER WAN-MCU**

- Vai trò: Là vi điều khiển chính, chịu trách nhiệm xử lý các tác vụ thuộc Lớp ứng dụng (Application Layer) và điều phối hoạt động cả hệ thống.
- Chức năng: Xử lý các tác vụ Lớp ứng dụng thông qua WAN Module Adapter, quản lý giao diện người dùng và kết nối đám mây, điều phối luồng dữ liệu chính, và điều phối hoạt động của SLAVE LAN-MCU. MASTER WAN-MCU chịu trách nhiệm cập nhật firmware từ thiết bị cấu hình (Configuration Tools) và từ đám mây (FOTA) cho cả hệ thống.

- **SLAVE LAN-MCU**

- Vai trò: Chịu trách nhiệm tương tác trực tiếp với lớp Cảm biến (Sensing Domain), quản lý các kết nối mạng nội bộ thông qua LAN Module Adapter, và lưu trữ dữ liệu.
- Chức năng: Quản lý các giao thức truyền thông nội bộ và mạng cảm biến, thu thập dữ liệu thô, thực hiện tiền xử lý và lưu trữ cục bộ (Micro SD Card) trước khi gửi lên MASTER WAN-MCU.

Giao tiếp giữa hai Vi xử lý: Để đảm bảo đồng bộ dữ liệu giữa MASTER WAN-MCU và SLAVE LAN-MCU, hệ thống sử dụng ba đường giao tiếp riêng biệt:

- **Quad SPI (Request/Data):** Kênh truyền dữ liệu tốc độ cao, dùng để Slave gửi dữ liệu cảm biến lên Master và Master gửi yêu cầu xuống Slave.
- **UART (Firmware Flow):** Kênh chuyên biệt dùng để nạp firmware cho SLAVE LAN-MCU thông qua MASTER WAN-MCU
- **GPIO (Control):** Các đường tín hiệu điều khiển (Boot, Reset, Interrupt) để đảm bảo đồng bộ trạng thái hoạt động.

Khối Module Adapter Đây là thành phần cho phép tháo lắp và thay thế linh hoạt các module, được chuẩn hóa kết nối với Baseboard gồm các giao thức: UART, SPI, I2C, USB, và các GPIO.

- **WAN Module Adapter**

- Kết nối với MASTER WAN-MCU.

- Hỗ trợ các module giao tiếp với Cloud hoặc thiết bị người dùng (User Interface Devices).

- **LAN1 Module Adapter**

- Kết nối với SLAVE LAN-MCU.
- Hỗ trợ kết nối các module kết nối không dây như LoRa, Zigbee, BLE, Thread, NB-IoT, ...

- **LAN2 Module Adapter**

- Kết nối với SLAVE LAN-MCU.
- Hỗ trợ kết nối các module kết nối có dây như RS-485, CAN, ...

Thiết bị giao diện người dùng (User Interface Devices): Các thiết bị đầu cuối giúp người quản trị tương tác với hệ thống thông qua các giao thức mạng không dây như Wi-Fi hay có dây như Ethernet.

Cloud Service: Nền tảng đám mây cung cấp dịch vụ lưu trữ, phân tích dữ liệu và quản lý tập trung cho hệ thống IoT.

Configuration Tools: Một chương trình máy tính để cấu hình cho gateway thực hiện những chức năng cho các module được kết nối.

Luồng dữ liệu:

- **Firmware/FOTA Flow:** Firmware được nạp vào thiết bị thông qua Configuration Tools bằng kênh USB, hoặc từ đám mây (FOTA) qua MASTER WAN-MCU, sau đó được chuyển tiếp đến SLAVE LAN-MCU qua kênh UART chuyên biệt.
- **Request/Data Flow:** Request được gửi từ Miền Ứng dụng (Application Domain) qua MASTER WAN-MCU đến SLAVE LAN-MCU để lấy dữ liệu cảm biến. Dữ liệu cảm biến sau khi được thu thập và tiền xử lý tại SLAVE LAN-MCU, hoặc đang được lưu trữ sẽ được gửi ngược lại qua MASTER WAN-MCU lên Cloud Service. Các request và dữ liệu này được truyền qua kênh Quad SPI tốc độ cao kết nối giữa hai MCU.
- **Control Flow:** Luồng tín điều khiển từ MASTER WAN-MCU để điều khiển trạng thái hoạt động của SLAVE LAN-MCU, và điều khiển hệ thống nguồn điện cho gateway thông qua các đường GPIO.

4.5.1.3 Thiết kế Module hóa

Nguyên lý: Thiết kế module hóa cho phép người dùng dễ dàng thay thế, nâng cấp hoặc mở rộng các chức năng của gateway mà không cần thay đổi toàn bộ hệ thống. Mỗi module được thiết kế theo chuẩn kết nối chung, giúp đảm bảo tính tương thích và linh hoạt trong việc lựa chọn các module phù hợp với nhu cầu sử dụng cụ thể.

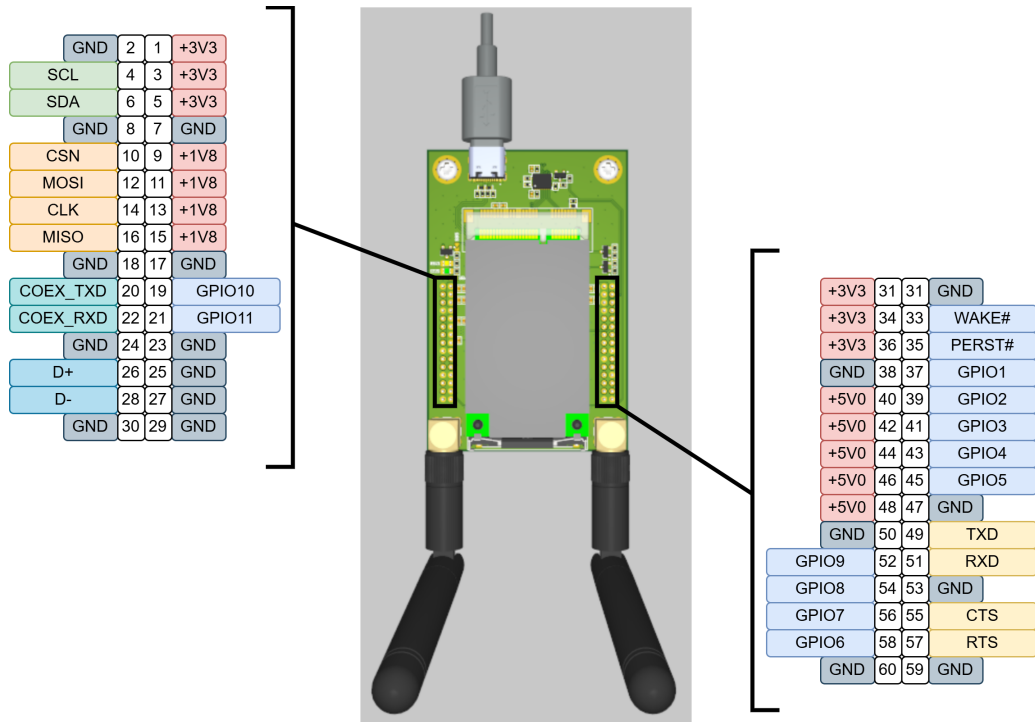
Thiết kế: Đối với các module có sẵn trên thị trường, các module này thông thường sẽ không theo một tiêu chuẩn cụ thể kể cả các module dạng Mini PCIe hay M.2, do đó việc thiết kế

các Module Adapter đóng vai trò quan trọng trong việc chuẩn hóa kết nối giữa các module và Baseboard.

Chuẩn hóa giao diện: Mỗi Module Adapter được thiết kế với các giao diện chuẩn bao gồm UART, SPI, I2C, USB, và GPIO được định nghĩa sơ đồ chân:

Group	Pin Name	Pin Number	Description
Power	+1V8	9, 11, 13, 15	Power Supply +1.8V
	+3V3	1, 3, 5, 32, 34, 36	Power Supply +3.3V
	+5V0	40, 42, 44, 46, 48	Power Supply +5.0V
	GND	2, 7, 8, 17, 18, 23, 24, 25, 27, 29, 30, 31, 38, 47, 50, 53, 59, 60	Ground
USB	D+	26	USB Data Positive
	D-	28	USB Data Negative
I2C	SDA	6	I2C Serial Data
	SCL	4	I2C Serial Clock
UART	TXD	49	UART0 Transmit Data
	RXD	51	UART0 Receive Data
	RTS	57	UART0 Request to Send
	CTS	55	UART0 Clear to Send
COEX_UART	COEX_TXD	20	Coexistence UART TXD
	COEX_RXD	22	Coexistence UART RXD
SPI	MOSI	12	SPI Master Out Slave In
	MISO	16	SPI Master In Slave Out
	CLK	14	SPI Serial Clock
	CSN	10	SPI Chip Select
GPIO	WAKE#	33	Wake Signal
	PERST#	35	Reset Signal
	GPIO1	37	General Purpose I/O 1
	GPIO2	39	General Purpose I/O 2
	GPIO3	41	General Purpose I/O 3
	GPIO4	43	General Purpose I/O 4
	GPIO5	45	General Purpose I/O 5
	GPIO6	58	General Purpose I/O 6
	GPIO7	56	General Purpose I/O 7
	GPIO8	54	General Purpose I/O 8
	GPIO9	52	General Purpose I/O 9
	GPIO10	19	General Purpose I/O 10
	GPIO11	21	General Purpose I/O 11

Bảng 4.1: Pinout Module Adapter



Hình 4.2: Sơ đồ chân Module Adapter

4.5.1.4 Xử lý dữ liệu

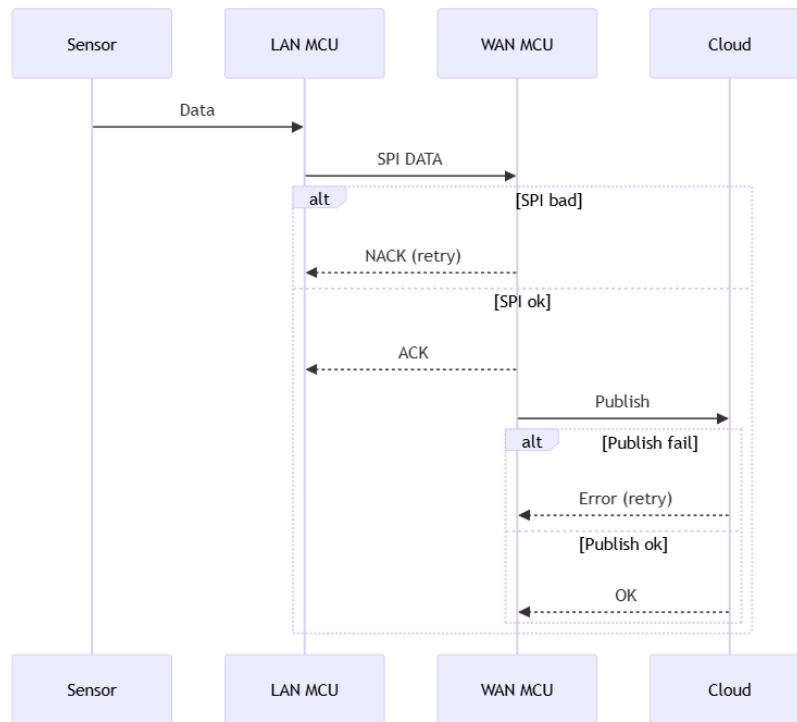
Phần xử lý dữ liệu của gateway được thiết kế theo kiến trúc **Dual-MCU Master-Slave**, trong đó LAN-MCU chịu trách nhiệm tương tác trực tiếp với thiết bị đầu cuối (thu thập/đẩy lệnh xuống), còn WAN-MCU đảm nhiệm kết nối Internet và giao tiếp Cloud. Toàn bộ luồng dữ liệu được tổ chức theo hai hướng chính: Uplink (Sensor → Cloud) và Downlink (Cloud → Sensor), đồng thời bảo đảm độ tin cậy bằng cơ chế CRC/sequence + ACK/NACK + retry trên kênh SPI và cơ chế retry/backoff khi publish lên Cloud.

Luồng Uplink: Sensor → LAN-MCU → WAN-MCU → Cloud Ở chiều uplink, dữ liệu từ cảm biến/thiết bị đầu cuối được LAN-MCU tiếp nhận, sau đó thực hiện các bước xử lý cơ bản như kiểm tra tính hợp lệ, phân tích (parse) và chuẩn hóa (normalize) về định dạng dữ liệu nội bộ trước khi chuyển sang WAN-MCU qua SPI.

Trên kênh SPI, gói dữ liệu luôn đi kèm các trường kiểm tra như CRC và sequence. WAN-MCU xác thực gói nhận được:

- Nếu gói lỗi (CRC sai hoặc sai thứ tự), WAN-MCU trả NACK, LAN-MCU thực hiện gửi lại (retry).
- Nếu gói hợp lệ, WAN-MCU trả ACK để LAN-MCU xác nhận hoàn tất giao dịch.

Sau khi nhận ACK, WAN-MCU thực hiện gửi dữ liệu lên Cloud theo cơ chế publish. Trong trường hợp publish thất bại, WAN-MCU áp dụng retry (có thể kèm backoff) để tránh gửi dồn liên tục gây nghẽn mạng hoặc quá tải broker.



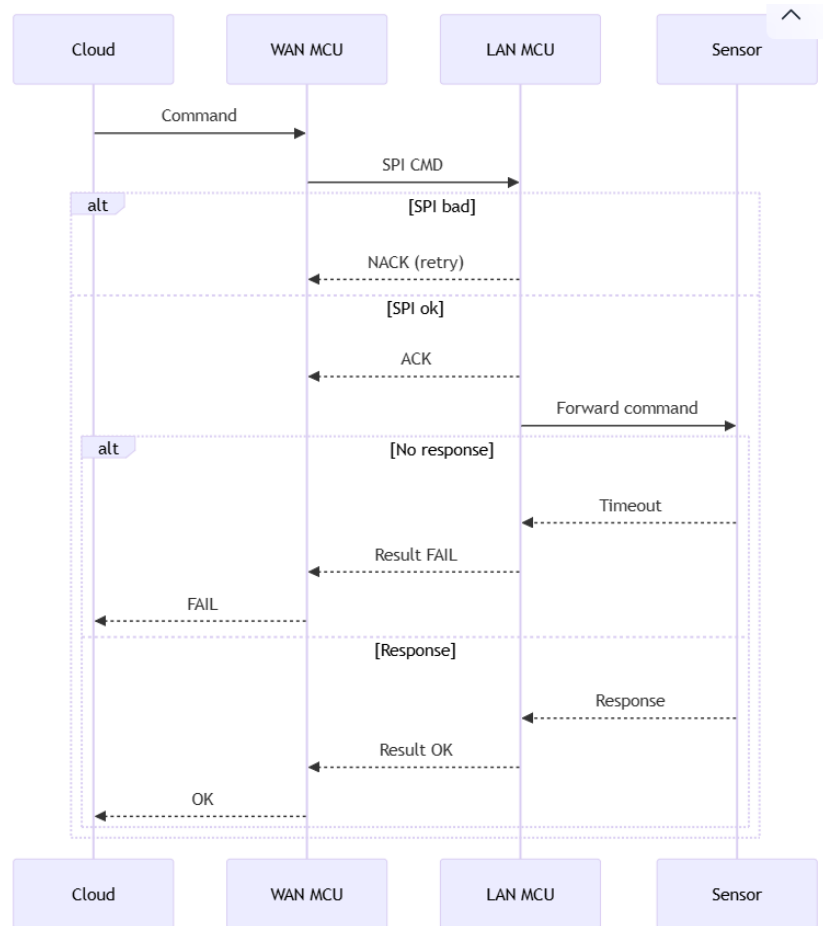
Hình 4.3: Sequence diagram: luồng xử lý dữ liệu end-to-end uplink

Luồng Downlink: Cloud → WAN-MCU → LAN-MCU → Sensor Ở chiều downlink, Cloud gửi lệnh/ yêu cầu xuống gateway thông qua WAN-MCU. WAN-MCU chuyển lệnh xuống LAN-MCU bằng SPI (theo cơ chế gói có CRC/sequence tương tự uplink). LAN-MCU nhận lệnh và:

- Nếu gói lỗi → trả NACK để WAN-MCU gửi lại.
- Nếu gói đúng → trả ACK, sau đó forward lệnh tới thiết bị đầu cuối theo giao thức tương ứng.

Sau khi gửi lệnh xuống sensor, LAN-MCU đợi phản hồi:

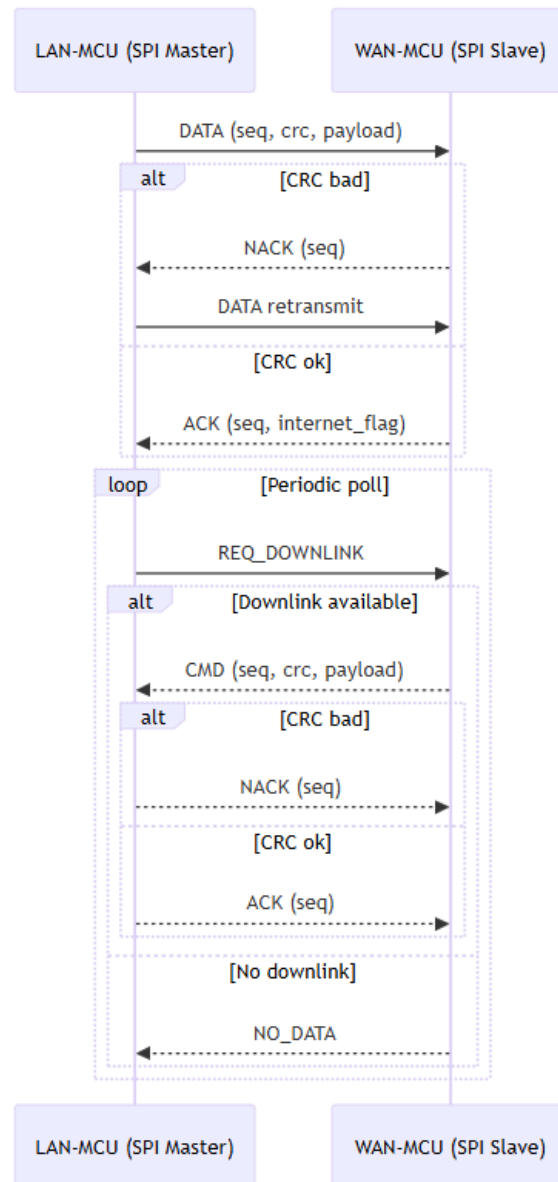
- Nếu thiết bị không phản hồi (timeout) → LAN-MCU báo FAIL lên WAN-MCU, WAN-MCU phản hồi FAIL về Cloud.
- Nếu thiết bị phản hồi → LAN-MCU báo OK lên WAN-MCU và WAN-MCU phản hồi OK về Cloud.



Hình 4.4: Sequence diagram: luồng xử lý dữ liệu end-to-end downlink

Cơ chế giao tiếp LAN-MCU ↔ WAN-MCU qua SPI (đảm bảo tin cậy) Kênh SPI giữa hai MCU đóng vai trò xương sống cho luồng dữ liệu. Thiết kế giao tiếp sử dụng các nguyên tắc:

- **Kiểm tra toàn vẹn:** mỗi gói có CRC, phát hiện lỗi đường truyền.
- **Chống mất/trùng gói:** sequence giúp nhận biết gói lặp hoặc sai thứ tự.
- **Bắt tay xác nhận:** ACK/NACK cho từng gói, kết hợp retry khi thất bại.
- **Downlink theo cơ chế polling/pull:** LAN-MCU (master) thực hiện REQ_DOWNLINK theo chu kỳ; WAN-MCU trả về CMD khi có dữ liệu hoặc NO_DATA khi trống.

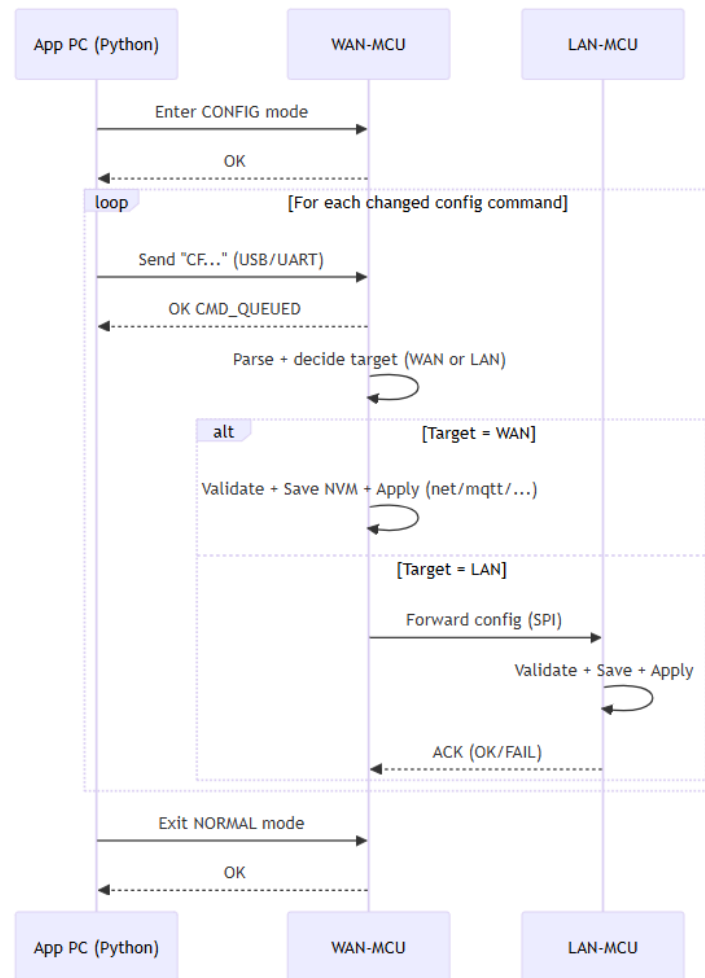


Hình 4.5: Sequence diagram: giao tiếp LAN-MCU ↔ WAN-MCU qua SPI

Cơ chế cập nhật cấu hình cho Gateway từ App Config PC Cơ chế cấu hình từ ứng dụng PC cho phép thay đổi tham số mà không cần sửa firmware, trực tiếp tác động đến pipeline dữ liệu như:

- Chu kỳ gửi dữ liệu, chính sách retry/backoff
- Thông số MQTT (host/port/topic/QoS nếu có), lựa chọn kênh Internet
- Bật/tắt luồng downlink hoặc định tuyến lệnh
- Các tham số giao tiếp thuộc LAN (phải chuyển tiếp sang LAN-MCU)

Quy trình cập nhật cấu hình gồm 3 thành phần chính: App PC → WAN-MCU → (tùy trường hợp) LAN-MCU. WAN-MCU đóng vai trò “router cấu hình”: nếu cấu hình thuộc WAN thì tự lưu và áp dụng; nếu thuộc LAN thì forward qua SPI và nhận ACK/FAIL từ LAN-MCU.

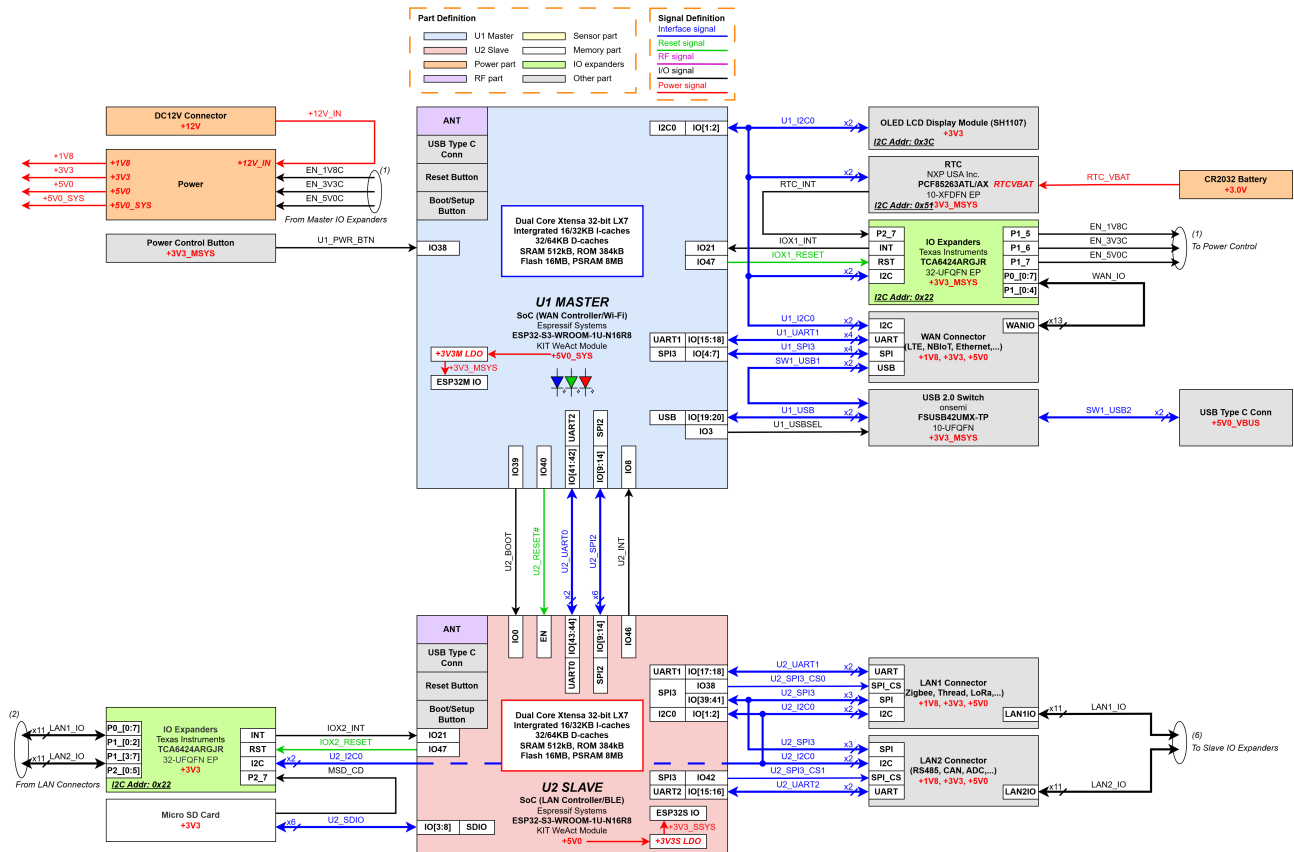


Hình 4.6: Sequence diagram: cập nhật cấu hình từ App PC

4.5.2 Thiết kế phần cứng PCB (PCB Hardware Design)

4.5.2.1 Thiết kế sơ đồ khối phần cứng

Sơ đồ khối phần cứng của hệ thống được xây dựng nhằm mô tả tổng thể kiến trúc và mối liên hệ giữa các khối chức năng chính, bao gồm khối xử lý trung tâm, các khối ngoại vi và hệ thống cấp nguồn. Mục tiêu của thiết kế sơ đồ khối là đảm bảo tính rõ ràng, dễ hiểu và khả năng mở rộng trong tương lai.



Hình 4.7: Sơ đồ khối hệ thống

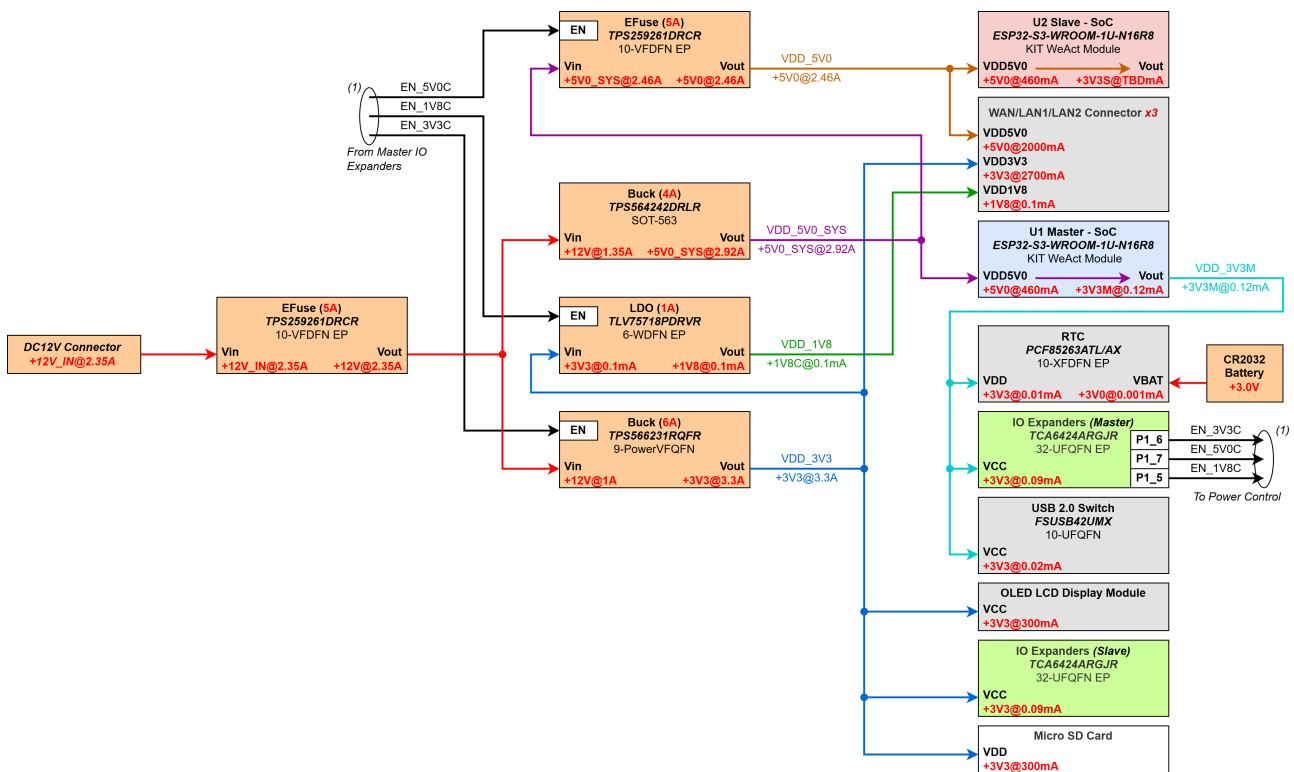
Sơ đồ khối hệ thống phần cứng: Hệ thống phần cứng được thiết kế theo kiến trúc dual-MCU Master-Slave nhằm phân tách chức năng xử lý giữa hai tầng mạng LAN và WAN, qua đó giảm tải cho từng vi điều khiển. Mỗi MCU đảm nhiệm việc giao tiếp và điều khiển các module thuộc miền mạng tương ứng thông qua các tập lệnh và ngoại vi tích hợp sẵn, giúp hệ thống đạt được khả năng cấu hình linh hoạt và nâng cao độ ổn định trong quá trình vận hành. Nền tảng phần cứng sử dụng module SoC ESP32-S3-WROOM-1U-N16R8, đáp ứng tốt các yêu cầu về năng lực xử lý, kết nối và khả năng mở rộng cho các ứng dụng IoT Gateway.

ESP32-S3 được trang bị vi xử lý lõi kép Xtensa® 32-bit LX7 với xung nhịp tối đa 240 MHz, tích hợp sẵn Wi-Fi và Bluetooth Low Energy (BLE). Vi điều khiển hỗ trợ đầy đủ các chuẩn giao tiếp phổ biến như UART, SPI, I²C, USB và SDIO, cho phép kết nối linh hoạt với các module ngoại vi. Bộ nhớ tích hợp gồm 16 MB Flash và 8 MB PSRAM, đáp ứng nhu cầu lưu trữ firmware, cập nhật OTA cũng như xử lý dữ liệu đệm dung lượng lớn trong quá trình vận hành.

Bên cạnh khối xử lý trung tâm, hệ thống còn tích hợp nhiều khối chức năng nhằm tăng cường

khả năng quản lý, giám sát và mở rộng phần cứng, bao gồm:

- **Mở rộng GPIO:** Sử dụng IC TCA6424ARGJR để điều khiển tín hiệu Enable của các khối nguồn, đồng thời quản lý các tín hiệu reset, interrupt và GPIO của các module mở rộng.
- **Quản lý thời gian thực (RTC):** Hệ thống tích hợp IC RTC PCF85263ATL, được cấp nguồn dự phòng bằng pin CR2032 (3.0 V), cho phép duy trì thông tin thời gian ngay khi nguồn cấp chính của hệ thống bị mất.
- **Hiển thị trạng thái:** Màn hình OLED kích thước 0.96 inch sử dụng driver SH1107, giao tiếp qua chuẩn I²C, dùng để hiển thị các thông tin trạng thái và tình trạng hoạt động của hệ thống.
- **Lưu trữ cục bộ:** MCU Slave được trang bị khe cắm thẻ nhớ microSD giao tiếp theo chuẩn SDIO 4-bit, cho phép ghi log và lưu trữ dữ liệu cục bộ trong quá trình vận hành. Trong trường hợp kết nối mạng bị gián đoạn, thẻ nhớ microSD đồng thời đóng vai trò bộ đệm dữ liệu, hỗ trợ cơ chế lưu trữ tạm thời và gửi lại khi kết nối được khôi phục.
- **Nút nhấn nguồn:** Nút nhấn được sử dụng để gửi lệnh điều khiển bật/tắt nguồn các khối chức năng thông qua MCU Master.
- **Cổng USB:** ESP32-S3 hỗ trợ giao tiếp USB phục vụ nạp firmware hoặc hoạt động ở chế độ USB Host. Hệ thống sử dụng IC chuyển mạch USB 2.0 FSUSB42UMX để lựa chọn giữa đường USB sử dụng cho việc nạp code/cấu hình thiết bị và đường USB kết nối với module WAN.



Hình 4.8: Sơ đồ hệ thống nguồn

Sơ đồ khối nguồn: Nguồn đầu vào của hệ thống sử dụng điện áp DC 12 V, phù hợp với các bộ nguồn công nghiệp và adapter thông dụng. Ngay tại ngõ vào, nguồn được bảo vệ bằng

eFuse nhằm hạn chế các sự cố quá dòng, ngắn mạch và đảm bảo an toàn cho toàn bộ hệ thống. Từ nguồn DC 12 V, các khối chuyển đổi DC/DC và LDO được sử dụng để tạo ra các rail nguồn chính, bao gồm:

- **+5V0_SYS:** Được chuyển đổi trực tiếp từ nguồn DC 12 V, dùng để cấp trực tiếp cho U1 Master và nhánh nguồn qua eFuse của nguồn +5V0.
- **+3V3M:** Là nguồn chuyển đổi từ rail +5V0_SYS bằng LDO nội bộ của module Master, dùng để cấp nguồn cho IO Expander (Master), RTC và USB Switch.
- **+5V0:** Là rail nguồn rẽ nhánh từ +5V0_SYS thông qua eFuse, dùng để cấp cho các khối chức năng và module sử dụng điện áp 5 V. Việc bật/tắt rail nguồn này được điều khiển bởi MCU Master.
- **+3V3:** Được tạo trực tiếp từ nguồn DC 12 V, cung cấp cho phần lớn các khối chức năng của hệ thống, bao gồm module LAN/WAN, mạch logic điều khiển và các ngoại vi. Rail nguồn này cũng được quản lý bởi MCU Master.
- **+1V8:** Được chuyển đổi từ rail +3V3 và quản lý bởi MCU Master, dùng để cấp nguồn cho các module mở rộng yêu cầu điện áp 1.8 V.

Đối với các module gắn ngoài thông qua Adapter module, các đường nguồn 5 V, 3.3 V và 1.8 V đều được điều khiển bằng tín hiệu enable từ MCU Master. Cách tổ chức này cho phép hệ thống:

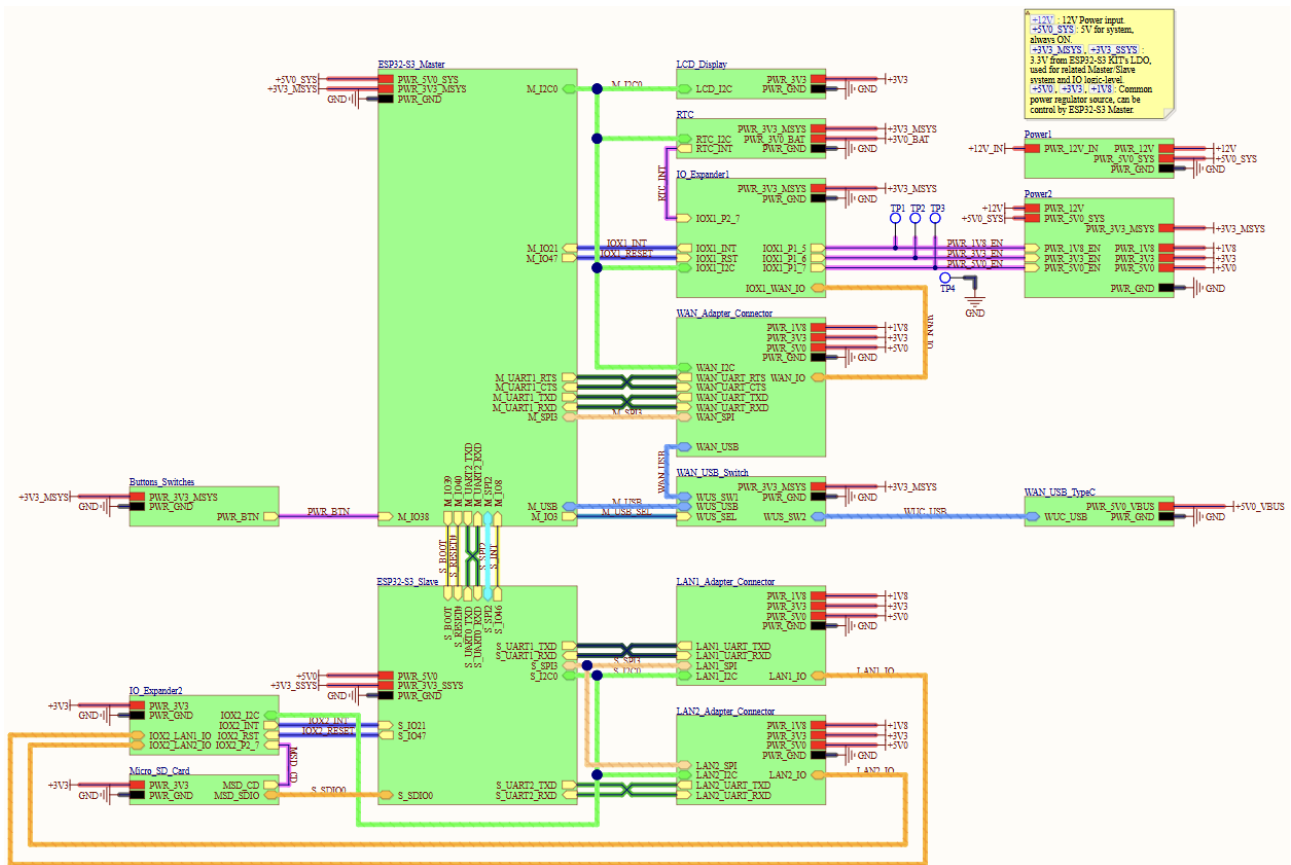
- Cho phép chủ động bật/tắt nguồn từng module LAN/WAN tùy theo cấu hình và kích bản sử dụng.
- Nhanh chóng cô lập và ngắt nguồn khi phát hiện sự cố từ các module bên ngoài.
- Giảm tiêu thụ năng lượng trong các chế độ hoạt động không liên tục hoặc tiết kiệm năng lượng.

4.5.2.2 Thiết kế Mạch nguyên lý (Schematic Design)

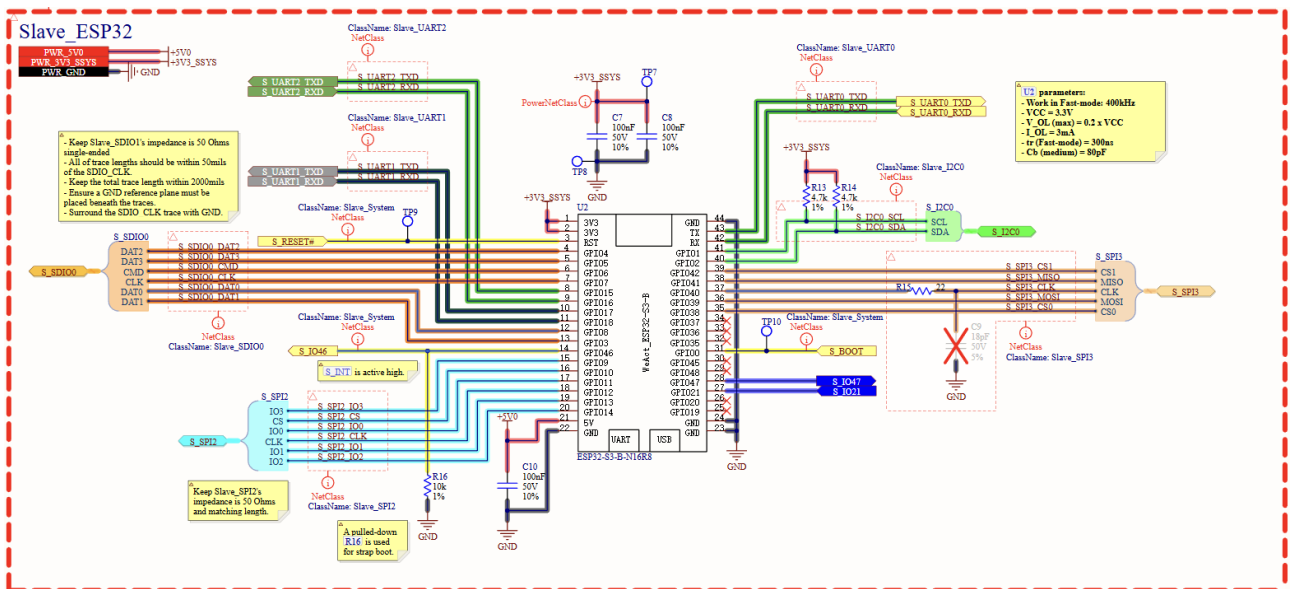
Phần này trình bày các nội dung kỹ thuật của dự án thông qua thiết kế sơ đồ nguyên lý. Thiết kế sơ đồ nguyên lý mô tả chi tiết các kết nối điện giữa các thành phần, nhằm đảm bảo hệ thống hoạt động đúng chức năng, đồng thời đáp ứng các yêu cầu về hiệu năng và các ràng buộc trong thiết kế.

Thiết kế Mạch xử lý trung tâm Khối xử lý trung tâm của hệ thống được thiết kế theo kiến trúc dual-MCU, sử dụng hai module ESP32-S3-WROOM-1U-N16R8 đảm nhiệm vai trò Master và Slave.

Nguyên lý được thiết kế dựa trên các khuyến nghị trong tài liệu [ESP32-S3 Hardware Design Guidelines - Espressif](#). Đối với các đường tín hiệu clock, một điện trở nối tiếp giá trị 22 Ω được bố trí trên đường tín hiệu nhằm giảm hiện tượng phản xạ, hạn chế nhiễu và cải thiện độ toàn vẹn tín hiệu.



Hình 4.9: Sơ đồ nguyên lý hệ thống



Hình 4.10: Sơ đồ nguyên lý ESP32 Slave

Giá trị điện trở kéo lên của bus I²C được xác định dựa trên đặc tính điện của đường truyền và các yêu cầu về thời gian đáp ứng của giao thức. Theo phương pháp tính toán được tham khảo từ tài liệu [I²C Bus Pull-Up Resistor Calculation](#) của Texas Instruments, miền giá trị hợp lệ của điện trở kéo lên R_{PU} được xác định bởi hai công thức sau:

$$R_{PU,max} = \frac{t_r}{0.8473 \cdot C_b} \quad (4.1)$$

$$R_{PU,min} = \frac{V_{CC} - V_{OL(max)}}{I_{OL}} \quad (4.2)$$

Trong đó:

- R_{PU} : điện trở kéo lên của các đường SDA và SCL (đơn vị Ω).
- C_b : điện dung tổng của bus I²C, bao gồm điện dung ký sinh của đường mạch và điện dung ngõ vào của các thiết bị trên bus (F).
- t_r : thời gian cạnh lên của tín hiệu I²C; đối với chế độ Fast Mode (400 kHz), t_r thường được giới hạn ở mức 300 ns.
- V_{CC} : điện áp cấp cho bus I²C (V).
- $V_{OL(max)}$: mức điện áp logic thấp tối đa cho phép khi thiết bị nhận mức 0 (V), thường lấy theo datasheet (xấp xỉ $0.2 V_{CC}$).
- I_{OL} : dòng kéo xuống cực đại của thiết bị khi xuất mức logic thấp (A).

Từ hai ràng buộc trên, điện trở kéo lên được lựa chọn sao cho thỏa mãn điều kiện:

$$R_{PU,min} \leq R_{PU} \leq R_{PU,max}$$

Cổng USB của module ESP32-S3 Master được kết nối ra ngoài thông qua cáp kết nối tới một IC USB Switch. Giải pháp này cho phép sử dụng linh hoạt đường tín hiệu USB giữa hai chức năng: nạp firmware, debug và cấu hình thiết bị; đồng thời kết nối và giao tiếp với module WAN hỗ trợ giao thức USB 2.0.

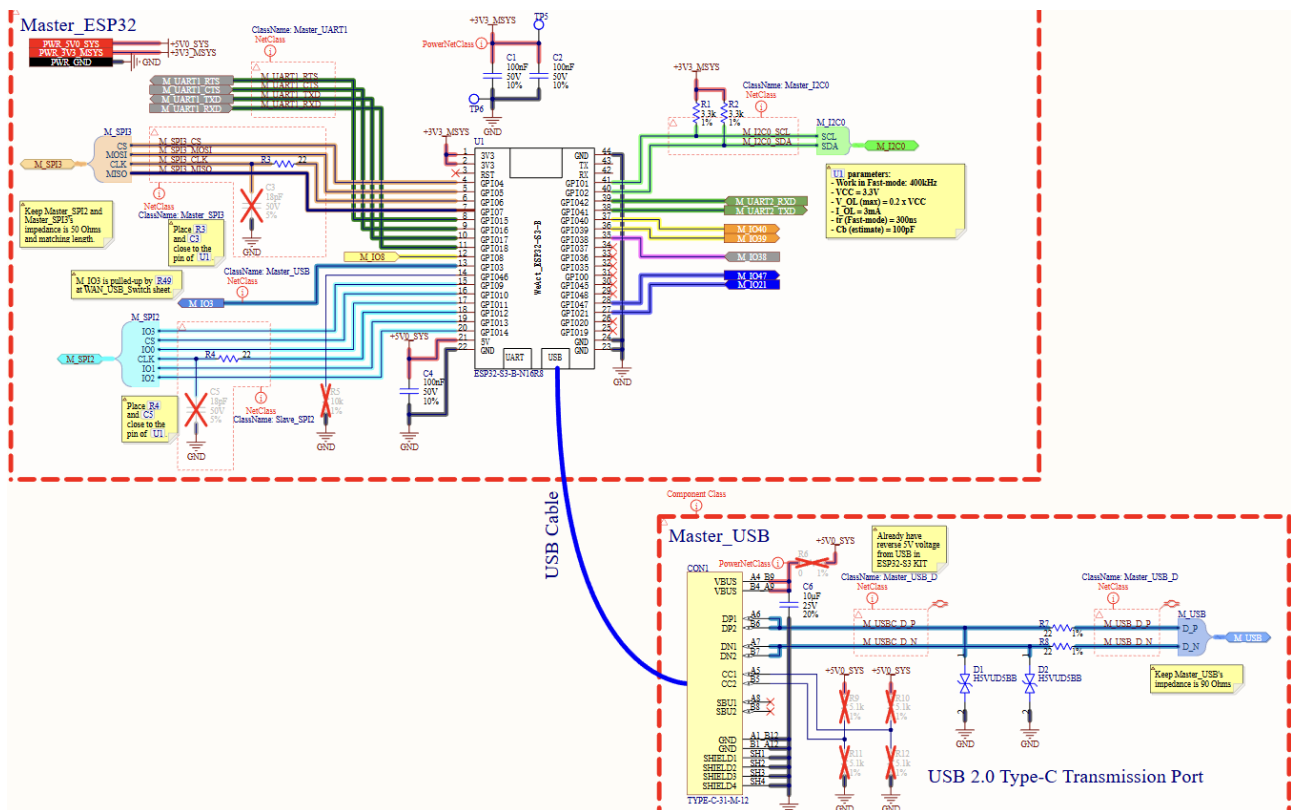
Hai đường dữ liệu USB Data (D^+ và D^-) được trang bị TVS diode bidirectional để chống lại các xung ESD, có các tham số đặc tính phù hợp để không làm ảnh hưởng nghiêm trọng đến tính toàn vẹn của tín hiệu tốc độ cao. TVS diode H5VUD5BB có các thông số đặc trưng:

- Peak Reverse Working Voltage (VRWM): 5 V
- Junction Capacitance (CJ): rất thấp, khoảng 0.3 pF tại 1 MHz

Giá trị điện dung ký sinh rất thấp (0.3 pF) giúp đảm bảo tín hiệu dữ liệu USB duy trì tốc độ truyền chuẩn Full-Speed 12 Mbit/s, giảm thiểu suy hao biên độ hay méo dạng sóng.

Ngoài ra, trên mỗi đường D^+ và D^- còn được bố trí thêm điện trở giảm chấn (series damping resistor) 22 Ω . Các điện trở này luôn được khuyến nghị trong các tài liệu thiết kế USB 2.0, với tác dụng:

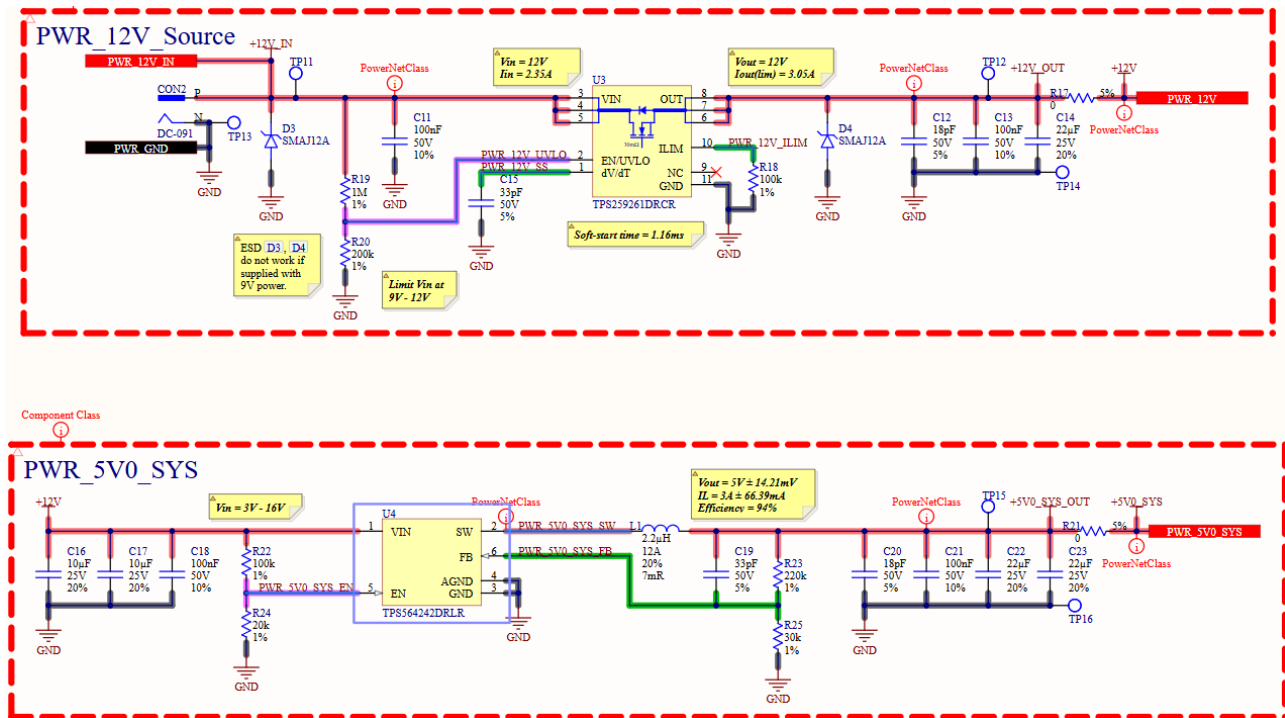
- Giảm hiện tượng overshooting và ringing trên cạnh xung
- Cải thiện Signal Integrity
- Giảm phát xạ EMI (Electromagnetic Interference)



Hình 4.11: Sơ đồ nguyên lý ESP32 Master

Thiết kế Mạch nguồn Tại connector nguồn đầu vào DC 12V, mạch được trang bị một TVS Diode SMAJ12A. Linh kiện này có nhiệm vụ hấp thu và triệt tiêu năng lượng xung điện trong trường hợp xuất hiện các xung ESD ngắn hạn, giúp bảo vệ mạch nguồn và các linh kiện phía sau khỏi hư hỏng do phóng tĩnh điện hoặc nhiễu xung cao áp từ môi trường.

Lớp bảo vệ kế tiếp là electronic Fuse TPS259261DRCR, giúp bảo vệ nguồn đầu vào khỏi các vấn đề quá áp và quá dòng. Theo như tính toán tiêu thụ tải dòng của hệ thống, eFuse tại nguồn đầu vào được giới hạn dòng tối đa ở giá trị 3.05 A. Các giá trị linh kiện khác được thiết kế dựa trên khuyến nghị trong datasheet của linh kiện.

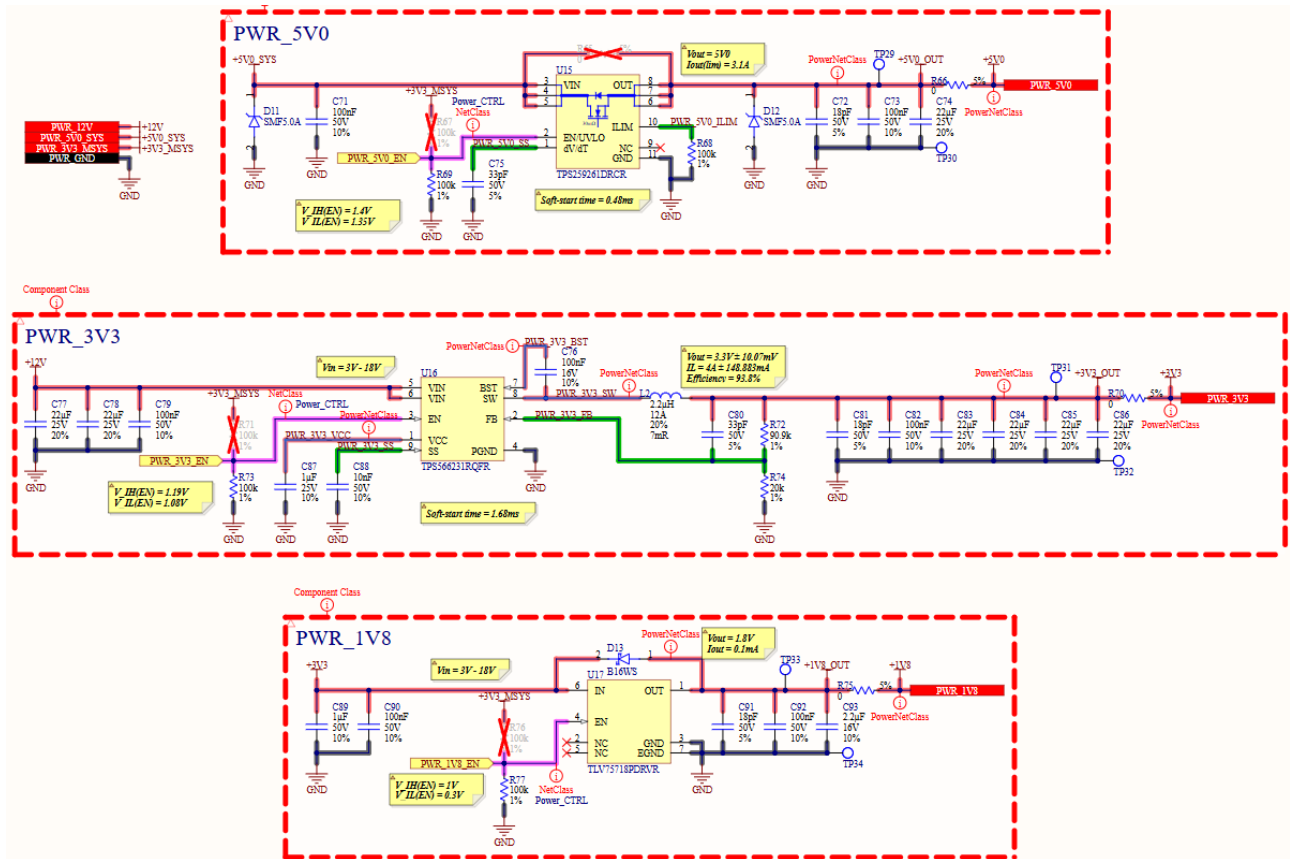


Hình 4.12: Sơ đồ nguyên lý mạch nguồn 1

Các mạch nguồn PWR_5V0_SYS và PWR_3V3 là các nguồn switching được thiết kế và mô phỏng dựa trên nền tảng mô phỏng WEBENCH của TI. Nền tảng này hỗ trợ người thiết kế lựa chọn và tính toán các linh kiện cần thiết để xây dựng nguồn theo đúng đặc tả mong muốn. Các tụ điện đầu vào được lựa chọn theo khuyến nghị, với mức điện áp chịu đựng được chọn lớn hơn gấp đôi mức điện áp đặt trên tụ. Dòng DC của cuộn cảm cũng được lựa chọn lớn hơn khoảng 30–50% so với giá trị tính toán nhằm đảm bảo nguồn hoạt động ổn định trong điều kiện tải thay đổi.

Các đường dẫn thiết kế cho mạch nguồn trên nền tảng WEBENCH TI:

- **PWR_5V0_SYS:** TPS564242DRLR 12V–12V to 5.00V @ 4A
- **PWR_3V3:** TPS566231RQFR 12V–12V to 3.30V @ 5A



Hình 4.13: Sơ đồ nguyên lý mạch nguồn 2

Nguồn PWR_5V0_SYS sẽ mặc định được cấp cho MCU master để điều khiển toàn bộ hệ thống ở giai đoạn khởi động ban đầu. Đồng thời, các khối IO Expanders của Master, RTC và USB switch cũng sẽ được mặc định cấp nguồn ngay từ đầu.

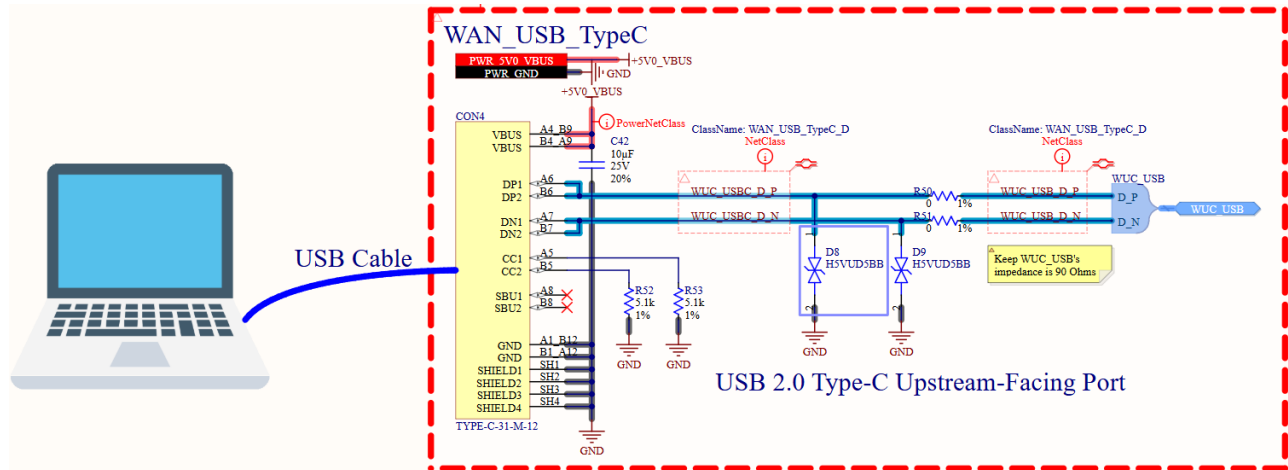
Các khối nguồn PWR_5V0 (rẽ nhánh từ PWR_5V0_SYS và có thể điều khiển thông qua eFuse), PWR_3V3 và PWR_1V8 được thiết kế mặc định ở trạng thái power off bằng điện trở pull-down tại các chân Enable. Sau khi hoàn tất cấu hình hệ thống, Master MCU sẽ thực hiện bật nguồn cho các khối tương ứng theo nhu cầu vận hành; các khối không sử dụng sẽ được giữ nguyên trạng thái power off nhằm tối ưu năng lượng.

Thiết kế Mạch USB Master MCU ESP32-S3 hỗ trợ một đường giao tiếp vật lý USB, cho phép thực hiện chức năng nạp firmware và debug thông qua khối phần cứng JTAG, đồng thời cũng có khả năng hoạt động ở chế độ USB Host. Để đáp ứng yêu cầu sử dụng linh hoạt hai chức năng này trên cùng một cổng vật lý, thiết kế sử dụng USB Switch FSUSB42UMX-TP.

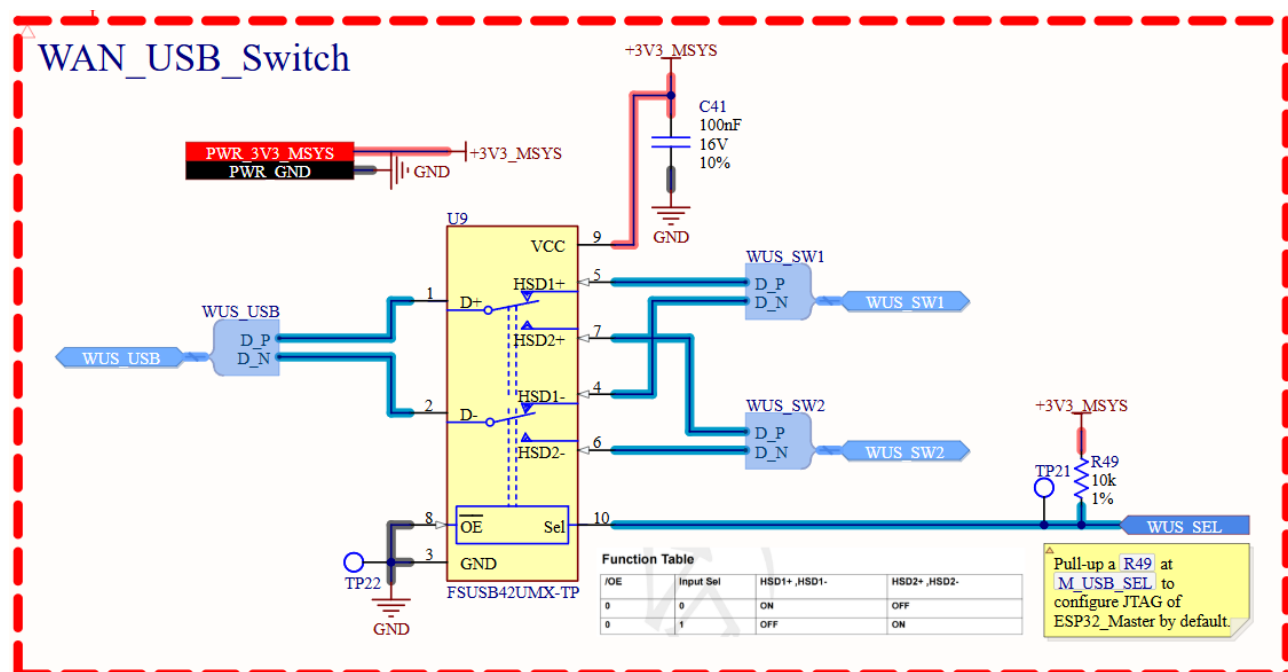
IC USB Switch cho phép chuyển mạch đường tín hiệu USB giữa hai khối chức năng khác nhau. Trong thiết kế này, IC được cấu hình mặc định nối đường USB ra cổng nạp firmware/debug bằng cách sử dụng điện trở pull-up tại chân SEL của IC. Cấu hình này đảm bảo quá trình nạp firmware và debug có thể được thực hiện ngay khi hệ thống chưa có sẵn chương trình lúc ban đầu, đồng thời vẫn giữ khả năng mở rộng để sử dụng chức năng USB Host khi cần thiết thông qua điều khiển từ MCU.

Cổng USB nạp firmware/debug cũng được trang bị thêm các TVS diode nhằm bảo vệ đường tín hiệu khỏi các xung phóng tĩnh điện (ESD) từ môi trường bên ngoài, giúp nâng cao độ tin

cây và độ bền của hệ thống. Ngoài ra, thiết kế còn dự phòng một cặp điện trở nối tiếp trên các đường dữ liệu USB để sử dụng trong trường hợp cần bổ sung điện trở giảm chấn (damping resistor) trong tương lai, nhằm tối ưu tính toàn vẹn tín hiệu khi có yêu cầu thay đổi về bố trí mạch hoặc điều kiện vận hành.

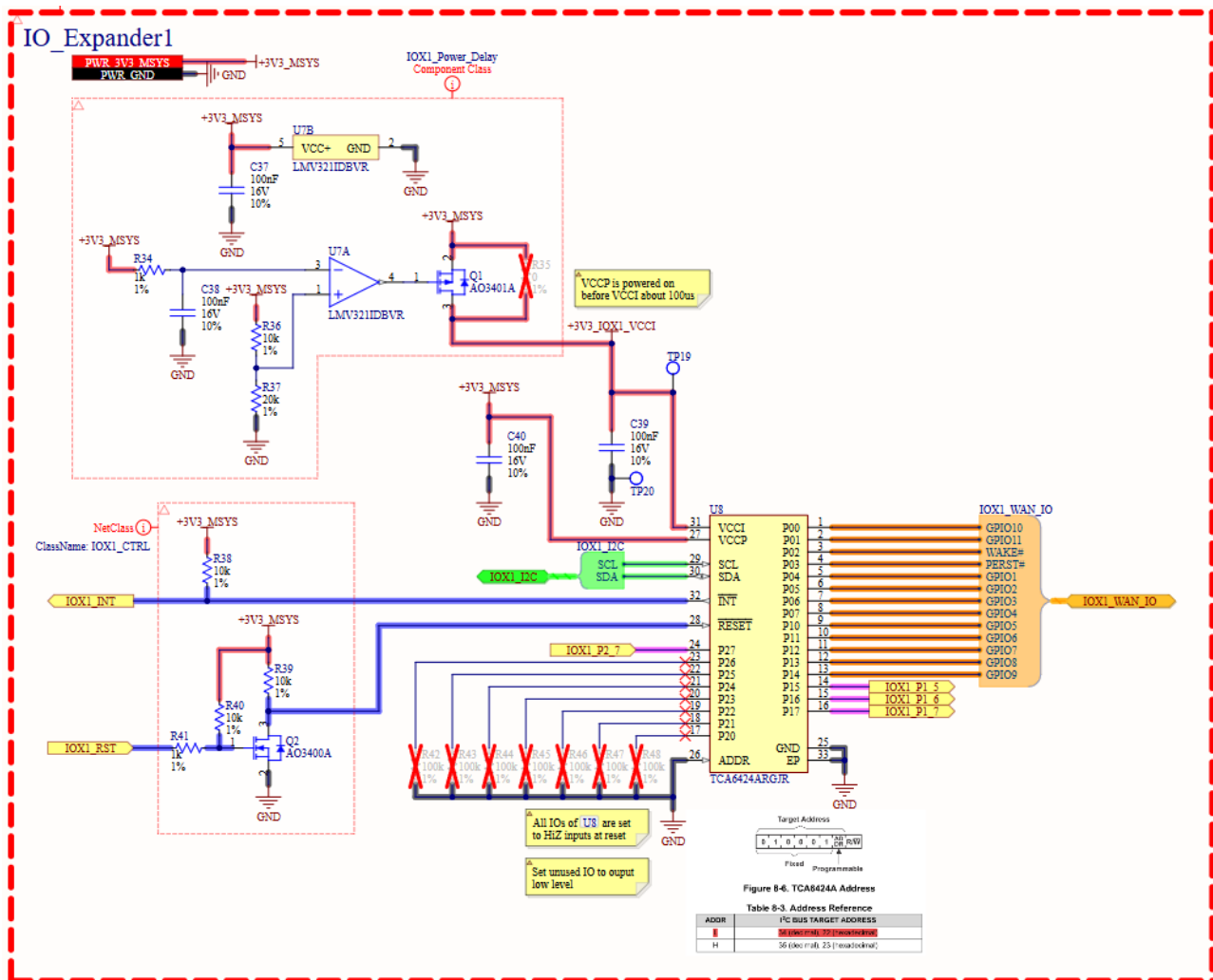


Hình 4.14: Sơ đồ nguyên lý mạch USB



Hình 4.15: Sơ đồ nguyên lý mạch USB Switch

Thiết kế Mạch IO Expander Mạch IO Expander sử dụng IC TCA6424ARGJR để mở rộng số lượng chân GPIO cho Master và Slave MCU. Do số lượng chân GPIO tích hợp sẵn trên MCU không đáp ứng đủ nhu cầu kết nối, IC này được sử dụng như một giải pháp thay thế hiệu quả thông qua giao thức I²C. IC TCA6424ARGJR cung cấp tổng cộng 24 chân GPIO, được chia thành ba cổng (Port 0, Port 1, Port 2), mỗi cổng gồm 8 chân. Các chân này có thể được cấu hình linh hoạt làm ngõ vào hoặc ngõ ra tùy theo yêu cầu ứng dụng.



Hình 4.16: Sơ đồ nguyên lý mạch IO Expanders (Master)

Theo khuyến nghị trong datasheet của IC IO Expander, trình tự cấp nguồn cần đảm bảo điện áp VCCP được tăng trước khi tăng điện áp VCCI, với thời gian ramp-up nằm trong khoảng 0.01–100 ms. Mục tiêu của yêu cầu này là đảm bảo nguồn VCCP được cấp trước, sau đó tối thiểu 0.01 ms mới cấp nguồn cho VCCI nhằm tránh hiện tượng đường dữ liệu SDA bị kẹt ở mức logic thấp trong quá trình khởi động. Mạch delay khởi động nguồn được thiết kế tại chân VCCI của IO Expander, bao gồm ba thành phần chính:

- **MOSFET kênh P:** dùng để đóng/ngắt nguồn cấp cho V_{CCI} .
- **Bộ so sánh OpAmp:** dùng để so sánh điện áp tham chiếu và điện áp tăng dần từ mạch RC.
- **Mạch RC delay:** tạo độ trễ thời gian cho điện áp tại đầu vào bộ so sánh.

9.2 Power Supply Recommendation

In the event of a glitch or data corruption, TCA6424A can be reset to its default conditions by using the power-on reset feature. Power-on reset requires that the device go through a power cycle to be completely reset. This reset also happens when the device is powered on for the first time in an application.

Ramping up the device V_{CCP} before V_{CCI} is recommended to prevent SDA from potentially being stuck LOW.

The two types of power-on reset are shown in Figure 9-4 and Figure 9-5.

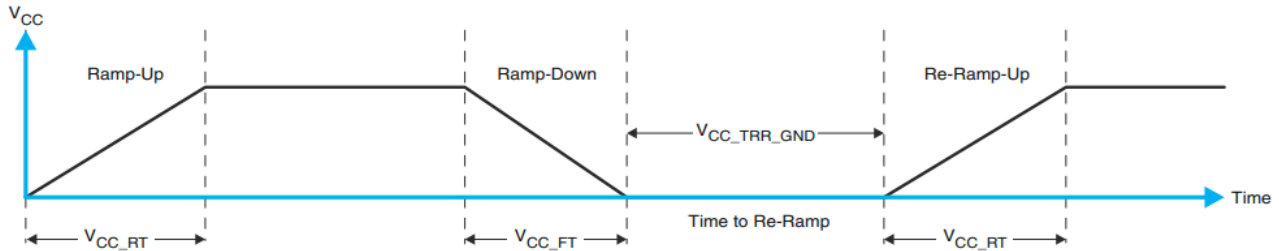


Figure 9-4. V_{CC} is Lowered Below 0.2 V or 0 V and Then Ramped Up to V_{CC}

Table 9-1. Recommended Supply Sequencing and Rates⁽¹⁾

PARAMETER		MIN	TYP	MAX	UNIT
t_{VCC_FT}	Fall rate	See Figure 9-4		1	100
t_{VCC_RT}	Rise rate	See Figure 9-4		0.01	100

Hình 4.17: Khuyến nghị cấp nguồn cho IC TCA6424ARGJR từ datasheet

Tại đầu vào dương của bộ so sánh là một cầu chia áp nhằm tạo điện áp tham chiếu. Điện áp tham chiếu này có giá trị xấp xỉ 2.2 V. Tại đầu vào âm của bộ so sánh là mạch RC delay, trong đó điện áp trên tụ tăng dần theo thời gian khi mạch được cấp nguồn. Khi điện áp trên tụ vượt qua mức tham chiếu 2.2 V, đầu ra của bộ so sánh sẽ chuyển từ mức thấp sang mức cao, kích hoạt MOSFET kênh P đóng và cấp nguồn cho chân VCCI của IO Expander.

Thời gian để điện áp trên tụ đạt đến mức điện áp tham chiếu được xác định theo công thức:

$$t = -\tau \ln \left(1 - \frac{V_C}{V_S} \right)$$

Trong đó:

- t : thời gian trễ (s),
- $\tau = RC$: hằng số thời gian của mạch RC (s),
- V_C : điện áp trên tụ điện tại thời điểm t (V),
- V_S : điện áp nguồn cấp cho mạch RC (V).

Với các giá trị được lựa chọn trong thiết kế, trong đó $\tau = RC = 0.0001$ s, điện áp nguồn $V_S = 3.3$ V và điện áp trên tụ tại thời điểm so sánh $V_C = 2.2$ V, thời gian trễ tính toán được là $t \approx 0.11$ ms. Giá trị này đáp ứng dải thời gian ramp-up từ 0.01 đến 100 ms theo khuyến nghị trong datasheet của nhà sản xuất.

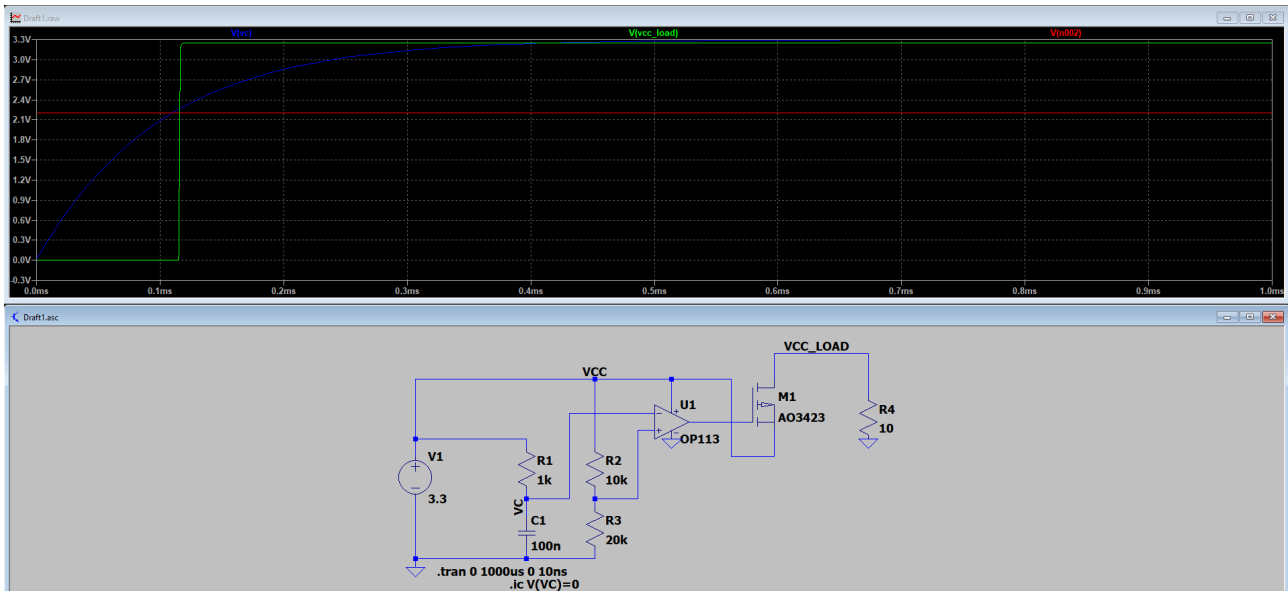
Cơ chế hoạt động của mạch được mô tả theo trình tự thời gian như sau:

- **Tại thời điểm $t = 0$:** mạch IO Expander vừa được cấp nguồn. Điện áp tại đầu vào âm của bộ so sánh bắt đầu tăng dần từ 0 V lên mức 3.3 V, trong khi điện áp tại đầu vào

đương được giữ cố định ở mức 2.2 V. Op-amp cho mức điện áp đầu ra cao (xấp xỉ 3.3 V), khiến MOSFET kênh P ở trạng thái đóng, nguồn VCCI chưa được cấp.

- **Tại thời điểm $t \approx 0.11\text{ ms}$:** điện áp tại đầu vào âm vượt qua mức 2.2 V, làm đầu ra op-amp bị kéo xuống mức thấp (xấp xỉ 0 V). MOSFET kênh P chuyển sang trạng thái dẫn, cho phép nguồn VCCI được cấp cho IO Expander.

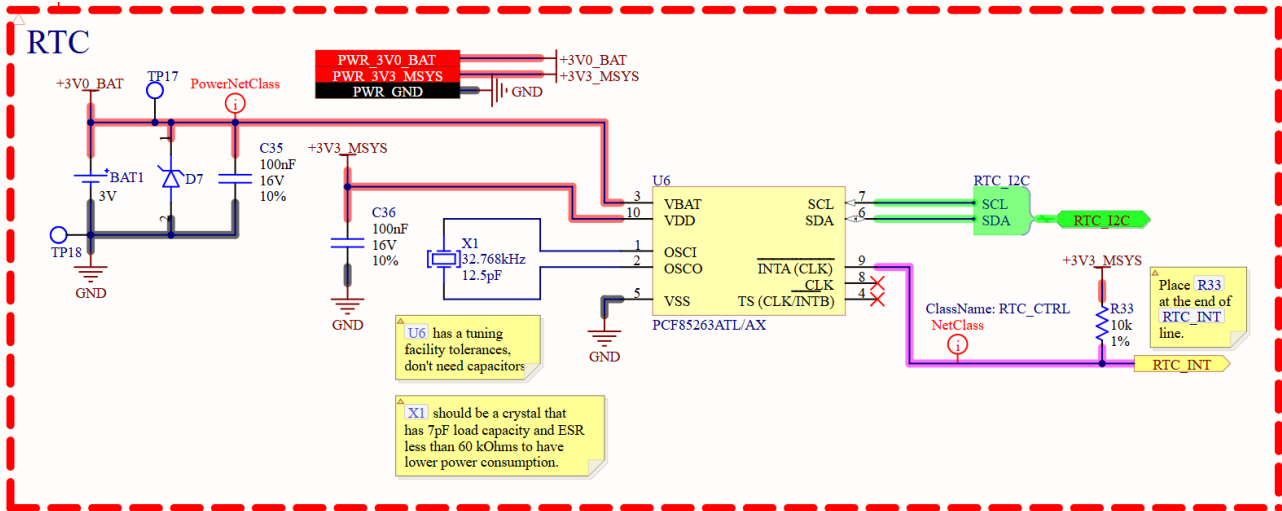
Mạch được mô phỏng trên phần mềm LTspice, kết quả mô phỏng được trình bày trong Hình 4.18.



Hình 4.18: Kết quả mô phỏng mạch delay khởi động nguồn VCCI cho IO Expander

Thiết kế Mạch RTC Khối RTC của hệ thống sử dụng IC PCF85263ATL/AX, giao tiếp với MCU thông qua chuẩn I²C. Nguồn VDD của RTC được cấp từ rail +3V3_MSYS, trong khi nguồn VBAT được cấp từ pin RTC backup 3V. Đường VBAT được trang bị một TVS diode nhằm bảo vệ mạch khỏi các xung ESD gây ra trong quá trình tháo lắp pin RTC.

IC RTC sử dụng thạch anh ngoài tần số 32.768 kHz được kết nối trực tiếp vào hai chân OSC1 và OSC0. Theo khuyến nghị của nhà sản xuất, IC PCF85263ATL/AX đã tích hợp khả năng hiệu chỉnh dao động (tuning facility), do đó không cần sử dụng thêm tụ tải ngoài cho thạch anh. Thạch anh được lựa chọn có điện dung tải danh định khoảng 7 pF và giá trị ESR nhỏ hơn 60 kΩ, nhằm đảm bảo mức tiêu thụ năng lượng thấp và độ ổn định dao động cao cho RTC.

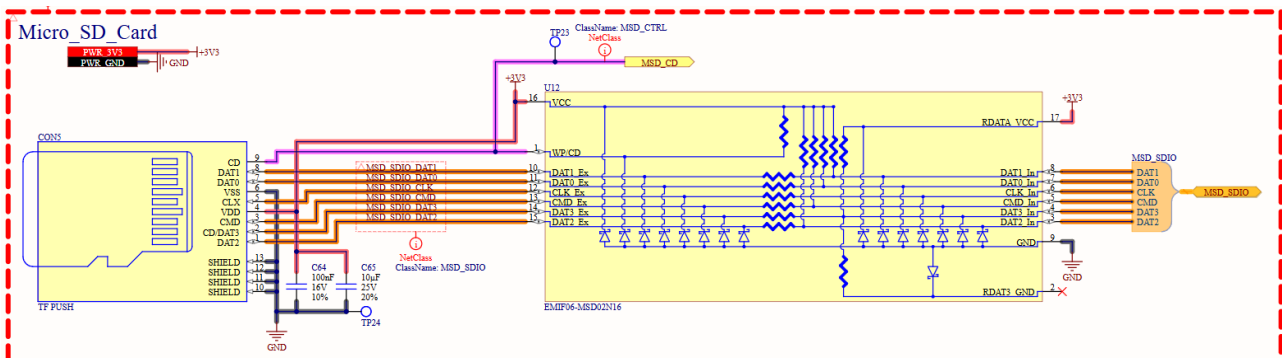


Hình 4.19: Sơ đồ nguyên lý mạch RTC

Thiết kế Mạch Micro SD Card Thẻ Micro SD được kết nối với Slave MCU thông qua giao thức SDIO 4-bit. Cấu hình giao tiếp này cho phép đạt được tốc độ truyền dữ liệu cao, phù hợp với yêu cầu lưu trữ và truy xuất dữ liệu của hệ thống.

Theo khuyến nghị trong tài liệu [ESP32-S3 Hardware Design Guidelines - Espressif](#), trên các đường tín hiệu SDIO bao gồm data, clock và cmd cần được bố trí các điện trở pull-up, đồng thời nên bổ sung các điện trở giảm chấn mắc nối tiếp nhằm cải thiện chất lượng tín hiệu. Việc bổ sung các linh kiện này giúp hạn chế hiện tượng ringing, overshoot trên các cạnh xung và nâng cao tính toàn vẹn tín hiệu khi thẻ Micro SD hoạt động ở tần số cao.

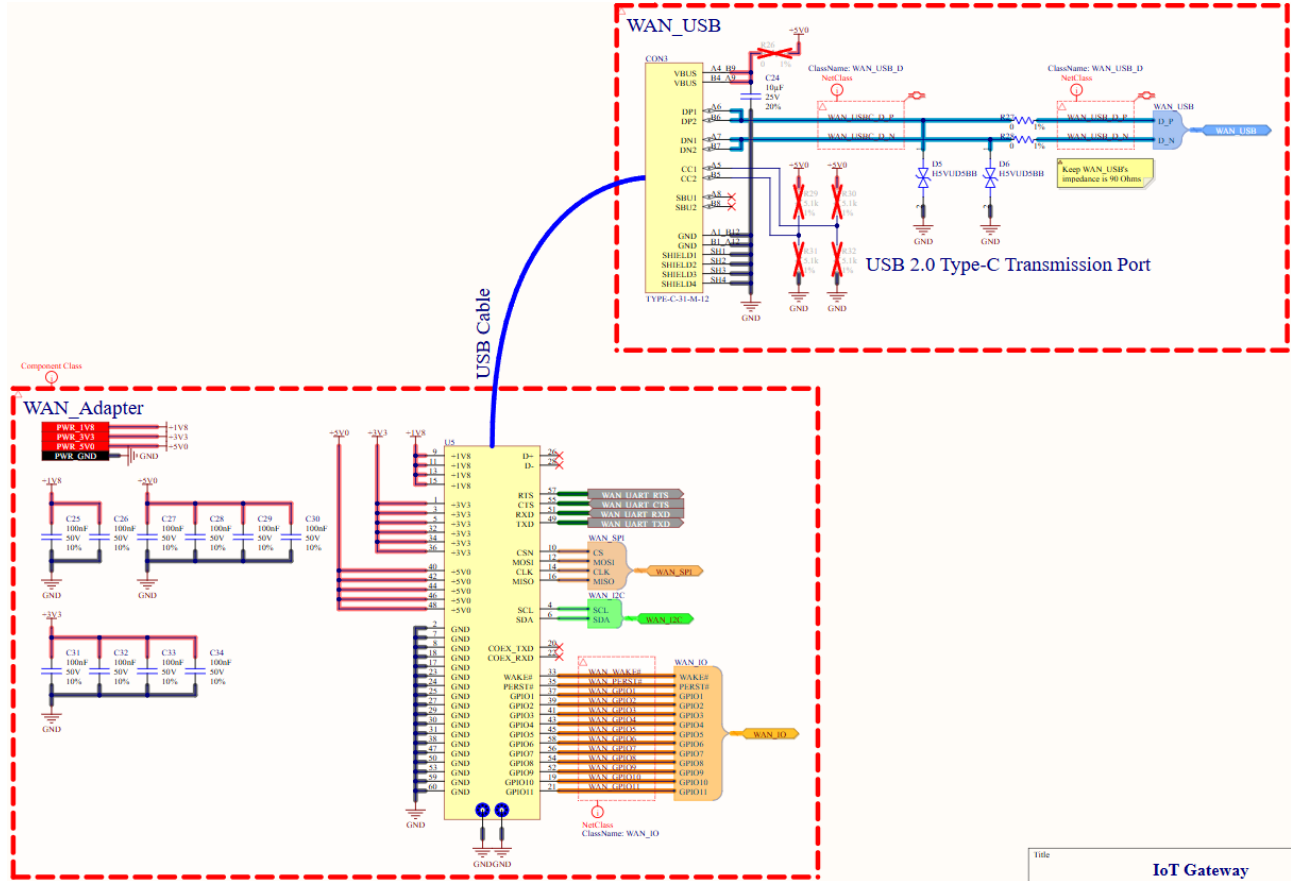
Trong thiết kế, mạch sử dụng IC EMIF06-MSD02N16 được đặt ngay tại đầu vào của cổng kết nối thẻ Micro SD. IC này tích hợp sẵn các diode TVS để bảo vệ chống phóng tĩnh điện, các điện trở kéo lên với giá trị xấp xỉ 80–100 kΩ, cùng các điện trở giảm chấn mắc nối tiếp có giá trị khoảng 36–54 Ω trên toàn bộ các đường tín hiệu SDIO của thẻ Micro SD. Việc sử dụng IC tích hợp giúp đơn giản hóa sơ đồ mạch, giảm số lượng linh kiện rời, đồng thời đảm bảo tuân thủ đầy đủ các khuyến nghị về bảo vệ và chất lượng tín hiệu trong quá trình thiết kế phần cứng.



Hình 4.20: Sơ đồ nguyên lý mạch Micro SD Card

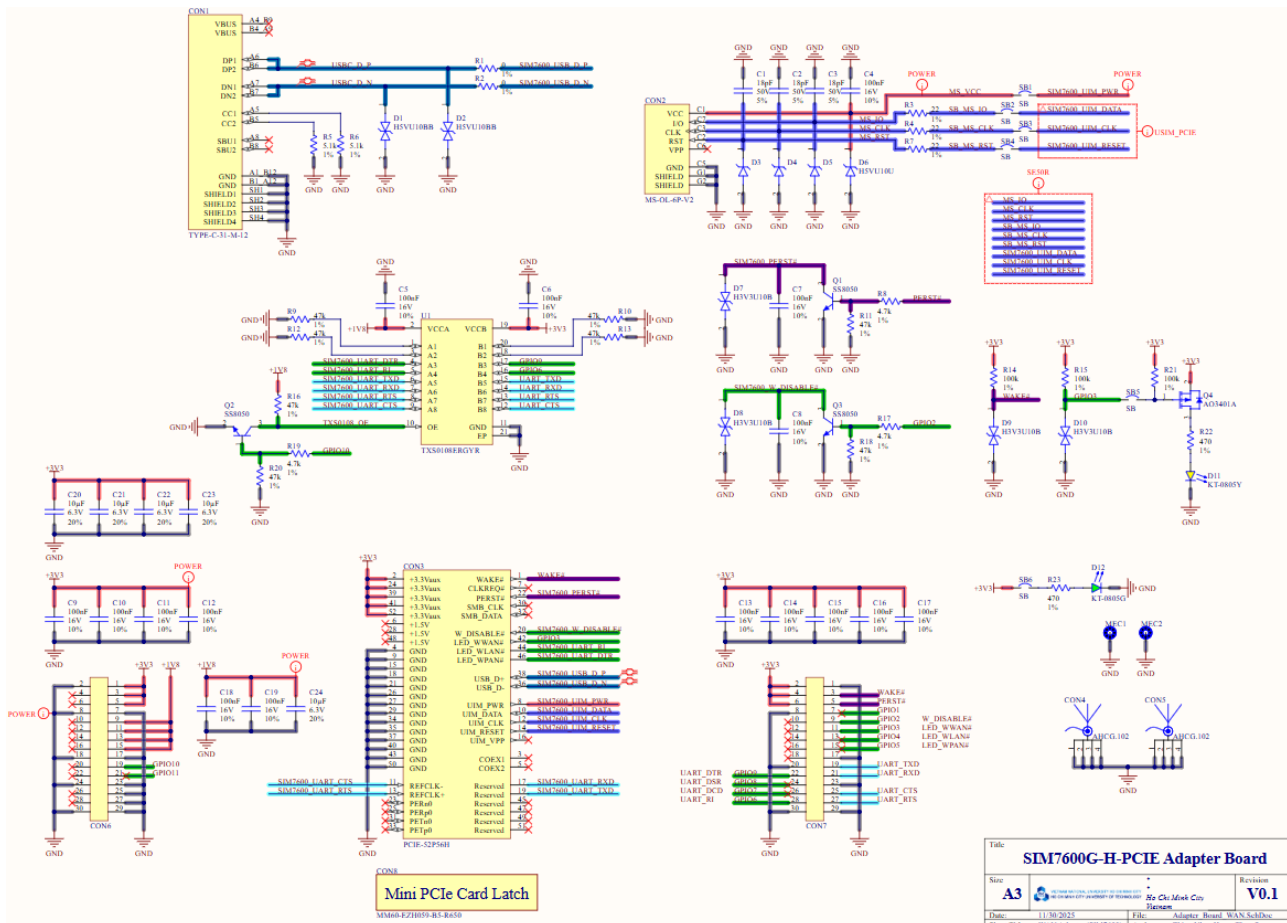
Thiết kế Mạch Module Adapter Baseboard được thiết kế để kết nối với các module mạng WAN và LAN thông qua một module adapter trung gian. Module adapter này được chuẩn hóa

chân kết nối theo các giao thức phổ biến như UART, I²C, SPI, USB cùng các chân GPIO hỗ trợ, nhằm đảm bảo khả năng tương thích linh hoạt giữa các module WAN/LAN và baseboard. Trên module adapter WAN được trang bị thêm một cổng USB và kết nối về baseboard thông qua cáp USB, cho phép thực hiện giao tiếp trực tiếp giữa Master MCU và module WAN thông qua giao thức USB 2.0.



Hình 4.21: Sơ đồ nguyên lý mạch WAN

Ở phiên bản Prototype, nhóm thiết kế triển khai mạch adapter WAN dành cho module SIM7600G-H chuẩn mPCIe. Mạch adapter này được thiết kế các khối chức năng cần thiết để giao tiếp với baseboard, bao gồm cấp nguồn, giao tiếp dữ liệu và các tín hiệu điều khiển, với các tham khảo trực tiếp từ tài liệu thiết kế phần cứng của module SIM7600G-H. Việc sử dụng adapter trung gian cho phép tách biệt phần thiết kế baseboard và phần đặc thù của module WAN.

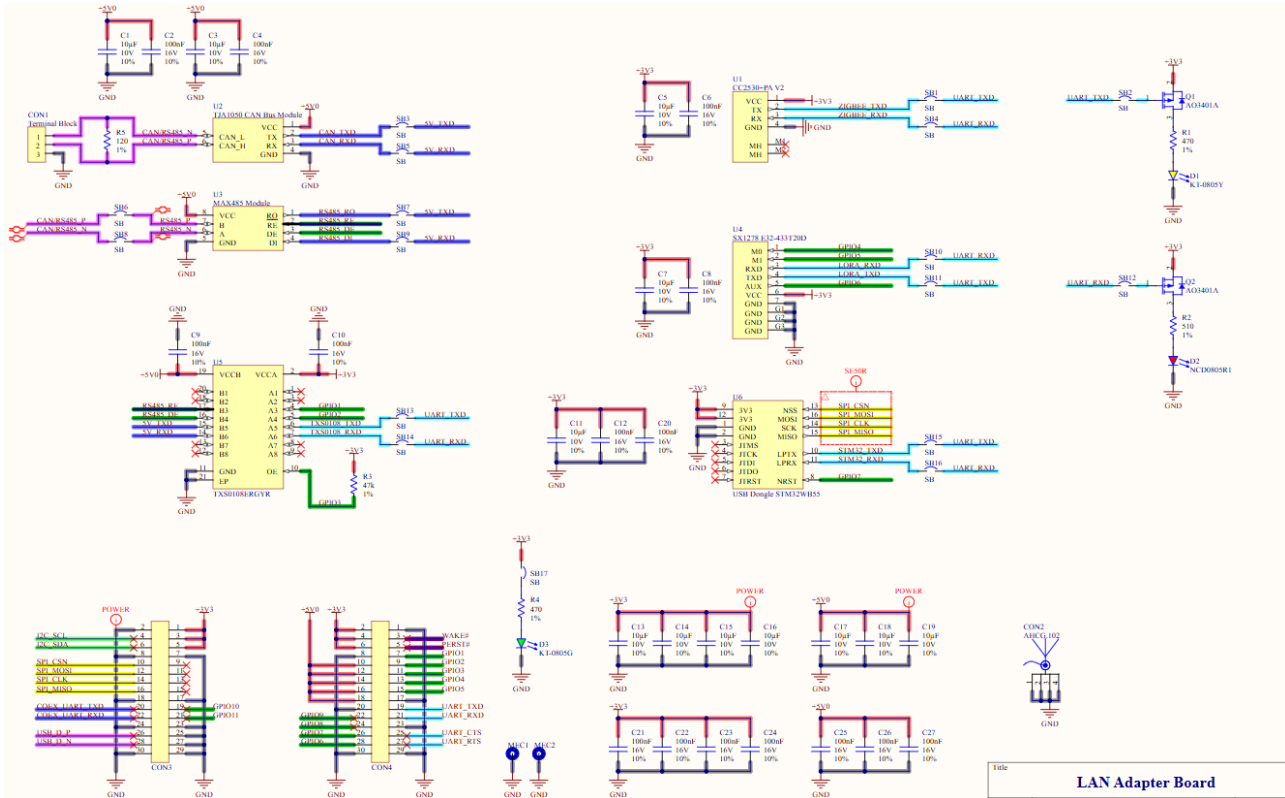


Hình 4.22: Sơ đồ nguyên lý mạch WAN Adapter

Bên cạnh đó, mạch adapter LAN được thiết kế nhằm hỗ trợ linh hoạt nhiều module truyền thông có sẵn trên thị trường. Các module được xem xét và hỗ trợ trong thiết kế prototype bao gồm:

- Module CAN Bus TJA1050, sử dụng nguồn cấp 5 V và giao tiếp UART.
- Module chuyển đổi MAX485 TTL to RS485, sử dụng nguồn cấp 5 V và giao tiếp UART.
- Module Zigbee UART RF Transmitter and Receiver CC2530+PA V2, sử dụng nguồn cấp 3.3 V và giao tiếp UART.
- Module RF LoRa SX1278 E32-433T20D, hoạt động ở băng tần 433 MHz, sử dụng nguồn cấp 3.3 V và giao tiếp UART.
- USB Dongle STM32WB55, hỗ trợ các giao thức không dây Bluetooth 5.4, Thread 1.3 và Zigbee 3.0, sử dụng nguồn cấp 3.3 V. Đây là module do nhóm tự nghiên cứu và phát triển, hỗ trợ giao tiếp UART và SPI.

Mạch adapter LAN được thiết kế để có thể hỗ trợ đồng thời tối đa năm module khác nhau. Việc lựa chọn module hoạt động được thực hiện thông qua các jumper nối tắt, cho phép định tuyến đường UART từ baseboard tới module tương ứng. Cách thiết kế này giúp tăng tính linh hoạt trong quá trình thử nghiệm, đồng thời giảm chi phí và thời gian phát triển khi xây dựng phiên bản prototype cho mạch LAN Adapter.



Hình 4.23: Sơ đồ nguyên lý mạch LAN Adapter

4.5.2.3 Thiết kế Layout (Layout Design)

Thiết kế PCB Stack-up và Kiểm soát trở kháng (Impedance Control)

1. Thiết kế Stack-up:

- Lựa chọn cấu trúc lớp:** Hệ thống bao gồm nhiều khối chức năng với các kênh tín hiệu tốc độ cao và đa dạng ray nguồn (power rails). Để đáp ứng các yêu cầu nghiêm ngặt về tính toàn vẹn tín hiệu (Signal Integrity - SI), tính toàn vẹn nguồn (Power Integrity - PI) và tương thích điện từ (EMC), thiết kế sử dụng cấu trúc PCB 6 lớp (6-layer stack-up) với sự phân bổ chức năng như sau:

Lớp	Chức năng chính	Mô tả chi tiết
L1	Top Signal & Power	Định tuyến tín hiệu tốc độ cao (High-speed), linh kiện SMD và các đường nguồn cục bộ.
L2	Ground Plane	Mặt phẳng đất tham chiếu (Reference Plane) liên tục cho tín hiệu ở L1, giúp giảm diện tích vòng lặp và chấn nhiễu.
L3	Power Plane	Mặt phẳng nguồn chính, phân phối năng lượng cho toàn bộ hệ thống.
L4	Signal (Low-speed)	Định tuyến các tín hiệu tốc độ thấp, tín hiệu điều khiển hoặc các đường mạch không yêu cầu kiểm soát trở kháng khắt khe.
L5	Ground Plane	Mặt phẳng đất tham chiếu liên tục cho tín hiệu ở L6.

Lớp	Chức năng chính	Mô tả chi tiết
L6	Bottom Signal & Power	Định tuyến tín hiệu tốc độ cao, linh kiện mặt dưới và đường nguồn phụ trợ.

- **Phân tích hiệu quả thiết kế:** Cấu trúc stack-up này cung cấp sự cân bằng tối ưu giữa hiệu năng và chi phí sản xuất:
 - **Toàn vẹn nguồn (PI):** Việc bố trí mặt phẳng đất (L2) và mặt phẳng nguồn (L3) liền kề nhau với khoảng cách điện môi mỏng tạo ra một tụ điện phẳng ký sinh (Inter-plane capacitance). Tụ điện này đóng vai trò quan trọng trong việc lọc nhiễu tần số cao và giảm trở kháng của mạng phân phối nguồn (PDN) một cách tự nhiên.
 - **Chống nhiễu xuyên âm (Crosstalk):** Các lớp tín hiệu tốc độ cao (L1 và L6) đều có mặt phẳng tham chiếu đất liền kề (L2 và L5). Cấu trúc đường truyền Microstrip này giúp khép kín trường điện từ, hạn chế bức xạ nhiễu và giảm thiểu xuyên nhiễu (Crosstalk) giữa các đường tín hiệu và đường nguồn switching có tần số đóng cắt lớn.

#	Name	Material	Type	Weight	Thickness	Dk	Df
	Top Overlay		Overlay				
	Top Solder	Solder Resist	Solder Mask		0.03048mm	3.8	
1	Top Signal	CF-004	Signal	1oz	0.035mm		
	Top Pregreg	PP-017	Pregreg		0.0994mm	4.36	0.02
2	GND TopRef	CF-003	Signal	1/2oz	0.0152mm		
	Top Core	Core-001	Core		0.55mm	4.1	0.02
3	Power	CF-003	Signal	1/2oz	0.0152mm		
	Center Pregreg	PP-016	Pregreg		0.1088mm	4.36	0.02
4	Inner Signal	CF-003	Signal	1/2oz	0.0152mm		
	Bottom Core	Core-001	Core		0.55mm	4.1	0.02
5	GND BotRef	CF-003	Signal	1/2oz	0.0152mm		
	Bottom Pregreg	PP-017	Pregreg		0.0994mm	4.36	0.02
6	Bottom Signal	CF-004	Signal	1oz	0.035mm		
	Bottom Solder	Solder Resist	Solder Mask		0.03048mm	3.8	
	Bottom Overlay		Overlay				

Hình 4.24: Layer Stack-up Management

- **So sánh với định tuyến Stripline lớp trong (Inner Layer Routing):** Một số thiết kế tham khảo ưu tiên định tuyến tín hiệu tốc độ cao ở các lớp trong (ví dụ Layer 4) để tận dụng cấu trúc Stripline (được bao bọc bởi hai mặt phẳng tham chiếu), giúp tín hiệu lan truyền trong môi trường điện môi đồng nhất và chống nhiễu EMI tốt hơn so với Microstrip (tiếp xúc với không khí). Tuy nhiên, việc áp dụng Stripline trong thiết kế này đòi hỏi Layer 3 phải chuyển thành Ground Plane để làm tham chiếu, dẫn đến thiếu hụt không gian cho mặt phẳng nguồn (Power Plane). Xét thấy các giao tiếp trong hệ thống (như SPI, SDIO, USB) có tần số hoạt động và chiều dài đường dây nằm trong phạm vi mà cấu trúc Microstrip (trên L1/L6) hoàn toàn có thể đáp ứng tốt các yêu cầu về băng thông và trở kháng nếu tuân thủ chặt chẽ quy tắc layout. Do đó, phương án hiện tại giúp tối ưu hóa không gian layout mà vẫn đảm bảo hiệu năng kỹ thuật.

2. Kiểm soát trở kháng (Impedance Control):

- Để đảm bảo chất lượng tín hiệu và giảm thiểu hiện tượng phản xạ (reflection) trên đường truyền, việc phối hợp trở kháng là bắt buộc đối với các đường tín hiệu tốc độ cao. Các tiêu chuẩn trở kháng đặc tính (Characteristic Impedance) áp dụng trong thiết kế bao gồm:
 - **Tín hiệu đơn cực (Single-ended):** Mục tiêu $50\Omega \pm 10\%$. Áp dụng cho các giao tiếp SDIO, SPI, I2C. Cấu trúc sử dụng là Coplanar Waveguide with Ground (CPWG).
 - **Tín hiệu vi sai (Differential pair):** Mục tiêu $90\Omega \pm 10\%$. Áp dụng cho cặp dây dữ liệu USB (D+/D-). Cấu trúc sử dụng là Coplanar Differential-pair.
- Các thông số hình học của đường mạch (bề rộng, khoảng cách cách ly) được tính toán dựa trên độ dày lớp đồng, hằng số điện môi (ϵ_r) và chiều cao lớp cách điện (Prepreg/Core) của nhà sản xuất PCB (JLCPCB). Chi tiết kích thước và cấu trúc stack-up được thể hiện trong các hình dưới đây:

Impedance (Ω)	Type	Signal Layer	Top Ref	Bottom Ref	Trace Width	Trace Spacing	Impedance trace to copper
50	Coplanar Single Ended	L1	/	L2	0.1483	/	0.2000

Hình 4.25: Thông số kích thước cho đường tín hiệu Coplanar Single-ended 50Ω (Đơn vị: mm)

Impedance (Ω)	Type	Signal Layer	Top Ref	Bottom Ref	Trace Width	Trace Spacing	Impedance trace to copper
90	Coplanar Differential Pair	L1	/	L2	0.1547	0.2000	0.2000

Hình 4.26: Thông số kích thước cho cặp tín hiệu vi sai (Coplanar Differential-pair) 90Ω (Đơn vị: mm)

Layer	Material	Thickness (mil)	Thickness (mm)
L1	Outer Copper Weight1oz	1.38	0.0350
Prepreg	3313 RC57% 4.2mil	3.91	0.0994
L2	Inner Copper Weight	0.60	0.0152
Core	0.075mm H/HOZ without copper	2.95	0.0750
L3	Inner Copper Weight	0.60	0.0152
Prepreg	3313 RC57% 4.2mil	3.61	0.0918
L4	Inner Copper Weight	0.60	0.0152
Core	0.075mm H/HOZ without copper	2.95	0.0750
L5	Inner Copper Weight	0.60	0.0152
Prepreg	3313 RC57% 4.2mil	3.91	0.0994
L6	Outer Copper Weight1oz	1.38	0.0350

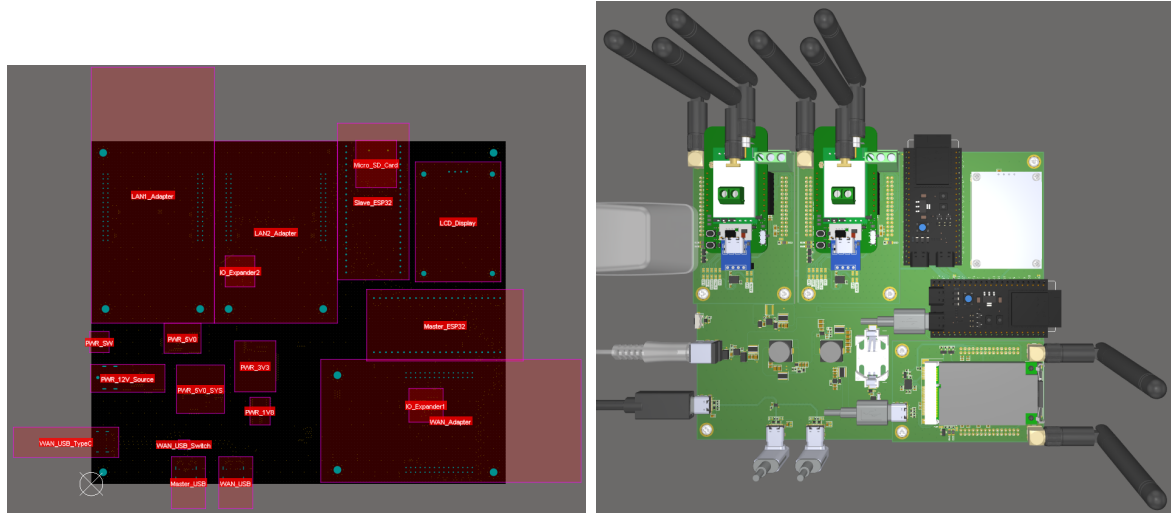
Hình 4.27: Cấu trúc Stack-up và thông số vật liệu tiêu chuẩn của JLCPCB

Quy hoạch và Bố trí linh kiện (Component Placement & Floor-planning): Việc bố trí linh kiện (Floor-planning) là bước quyết định đến hiệu năng chống nhiễu (EMC) và tính ổn định của hệ thống. Dựa trên sơ đồ nguyên lý và các ràng buộc cơ khí, chiến lược quy hoạch được thực hiện theo các nguyên tắc sau:

1. Phân vùng chức năng (Functional Partitioning):

- Bo mạch được chia thành các vùng chức năng vật lý riêng biệt dựa trên mức năng lượng và loại tín hiệu để giảm thiểu nhiễu giao thoa. Cụ thể bao gồm:
 - **Vùng Nguồn (Power Zone):** Bố trí cục bộ gần đầu vào DC để cô lập nhiễu chuyển mạch.

- **Vùng Xử lý trung tâm (Digital Core):** Chứa MCU Master và Slave, đặt tại trung tâm để tối ưu hóa đường đi tín hiệu.
- **Vùng Giao tiếp Modular (Interface Zone):** Bố trí dọc theo các cạnh bo mạch để thuận tiện cho thao tác cắm rút module.
- **Vùng RF (RF Zone):** Dành riêng cho các module không dây và anten.



Hình 4.28: Sơ đồ quy hoạch phân vùng chức năng trên PCB (Floor-planning)

2. Tối ưu hóa Luồng tín hiệu (Signal Flow Optimization):

- **Vị trí vi xử lý:** Hai MCU được đặt sát nhau và đặt ở vị trí phù hợp so với các Module Adapter xung quanh. Điều này giúp cân bằng và giảm thiểu chiều dài đường mạch đến các cổng kết nối xung quanh, đặc biệt quan trọng với các bus giao tiếp tốc độ cao như Quad-SPI và các đường điều khiển tới Module Adapter.
- **Tín hiệu tốc độ cao:** Các linh kiện liên quan đến giao tiếp tốc độ cao (như USB Switch, ESD protection) được đặt thẳng hàng và sát nhất có thể với cổng kết nối vật lý. Mục tiêu là giảm thiểu chiều dài đường dây (Trace length), từ đó giảm cảm kháng ký sinh và suy hao tín hiệu.

3. Kiểm soát Nhiễu điện từ (EMI Management):

- **Cách ly nguồn nhiễu:** Các thành phần gây nhiễu mạnh như mạch nguồn xung (Buck Converter), cuộn cảm, và diode chuyển mạch được quy hoạch xa khỏi các khu vực nhạy cảm như khối Analog, mạch tạo dao động (Crystal Oscillator) và khối RF.
- **Phân tách không gian RF:** Các vị trí dành cho module RF (như WiFi, LoRa) được bố trí tại các góc hoặc rìa của bo mạch. Khoảng cách vật lý giữa các anten được tối đa hóa (Spatial Diversity) và đặt chéo góc nhau để giảm hiện tượng bão hòa đầu vào hoặc giao thoa sóng vô tuyến (Co-existence interference).

4. Tương thích Cơ khí và Sản xuất (DFM/DFA):

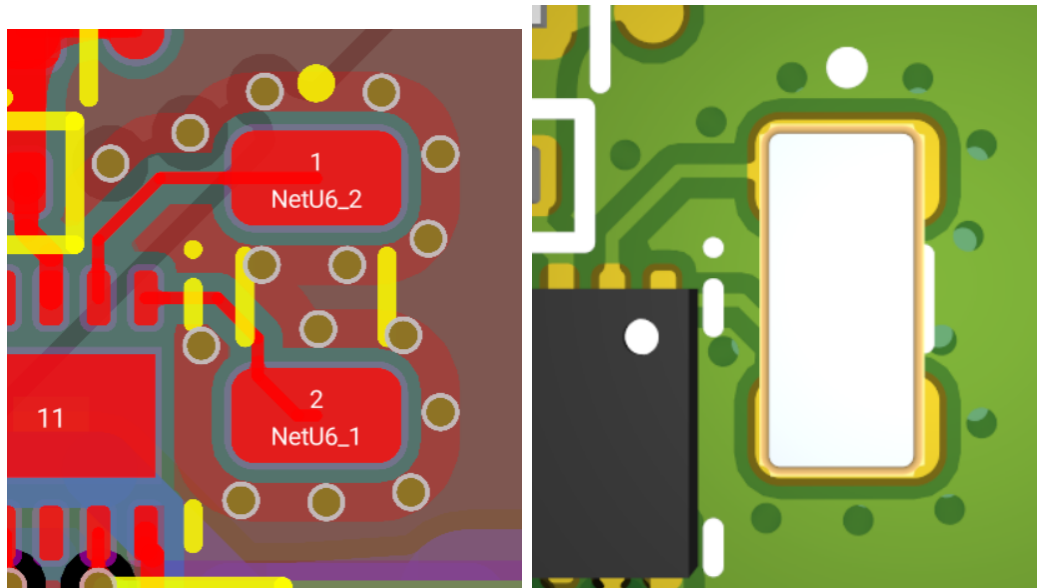
- **Giao diện kết nối:** Các thành phần tương tác với người dùng (Cổng USB, Khe thẻ nhớ, Nút nhấn, Đèn LED) được bố trí tại các mép bo mạch (Edge Placement), tuân thủ các ràng buộc về vị trí đảm bảo khả năng thao tác kết nối.

- **Vùng cấm (Keep-out Zone):** Thiết lập các vùng cấm đi dây và đặt linh kiện xung quanh các lỗ bắt vít (Mounting holes) và khu vực anten PCB để tránh chập mạch hoặc ảnh hưởng đến bức xạ sóng.

Kỹ thuật Chống nhiễu và Giảm thiểu EMI: Để đảm bảo thiết bị hoạt động ổn định trong môi trường công nghiệp và tuân thủ các tiêu chuẩn về tương thích điện từ (EMC), thiết kế PCB áp dụng các kỹ thuật layout sau:

1. Kỹ thuật Che chắn và Lồng Faraday:

- **Phủ mass toàn mạch:** Các diện tích trống trên tất cả các lớp PCB đều được phủ đồng nối đất (GND). Việc này tạo ra một lồng chắn tĩnh điện tự nhiên, giúp hấp thụ các nhiễu bức xạ từ môi trường và giảm thiểu phát xạ từ các đường mạch nội bộ.
- **Vòng bảo vệ và Via Stitching:** Đối với các khu vực nhạy cảm hoặc nguồn gây nhiễu mạnh như mạch dao động thạch anh (Crystal Oscillator) và cặp dây vi sai USB tốc độ cao, kỹ thuật "khâu via" (Via Stitching) được áp dụng. Các lỗ via nối đất được đặt bao quanh linh kiện/đường mạch với mật độ cao (khoảng cách $< \lambda/20$ tần số hoạt động). Cấu trúc này hoạt động như một bức tường chắn sóng (Shielding Wall), cô lập nhiễu điện từ lan truyền ngang qua lớp điện môi.



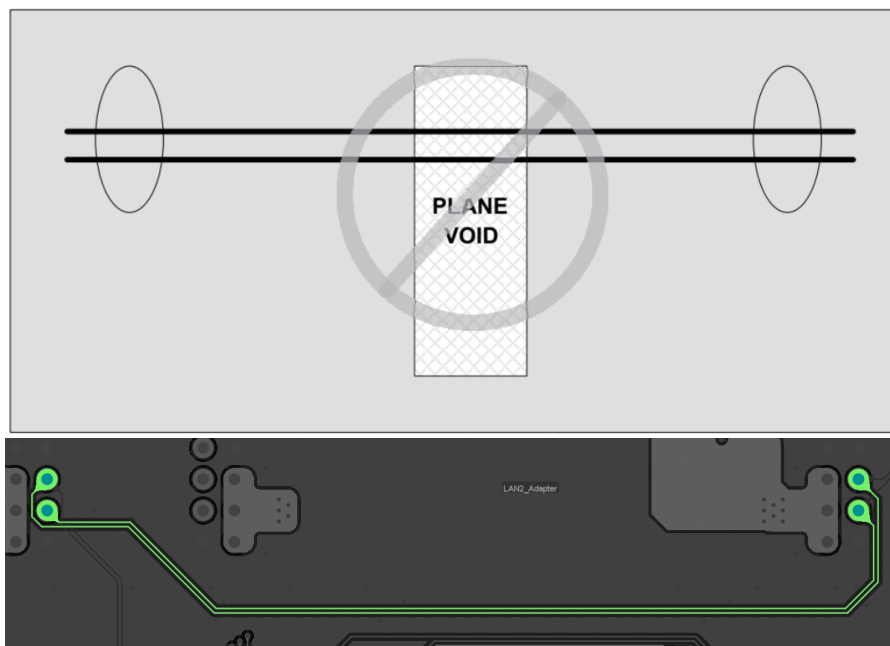
Hình 4.29: Kỹ thuật tạo vòng bảo vệ (Guard Ring) và via stitching xung quanh khối thạch anh để cô lập nhiễu.



Hình 4.30: Kỹ thuật bọc mass và stitching via dọc theo đường tín hiệu USB tốc độ cao.

2. Quản lý Đường hồi tiếp tín hiệu:

- **Nguyên lý đường dẫn cảm kháng thấp nhất:** Ở tần số cao, dòng điện hồi tiếp luôn có xu hướng chạy ngay bên dưới đường dây tín hiệu trên mặt phẳng tham chiếu (Reference Plane) để tối thiểu hóa cảm kháng vòng lặp.
- **Tránh cắt xẻ mặt phẳng đất:** Thiết kế đảm bảo các mặt phẳng đất (L2, L5) là liên tục và liền mạch. Tuyệt đối tránh đi dây tín hiệu tốc độ cao cắt ngang qua các khe hở (Splits) hoặc vùng trống (Voids) trên lớp đất. Việc tín hiệu băng qua khe hở sẽ buộc dòng hồi tiếp phải đi vòng xa hơn, tạo thành một vòng lặp diện tích lớn (Large Loop Area). Vòng lặp này hoạt động như một anten khung (Loop Antenna), vừa phát xạ nhiễu ra ngoài vừa thu nhiễu từ môi trường vào, gây suy giảm chất lượng tín hiệu.

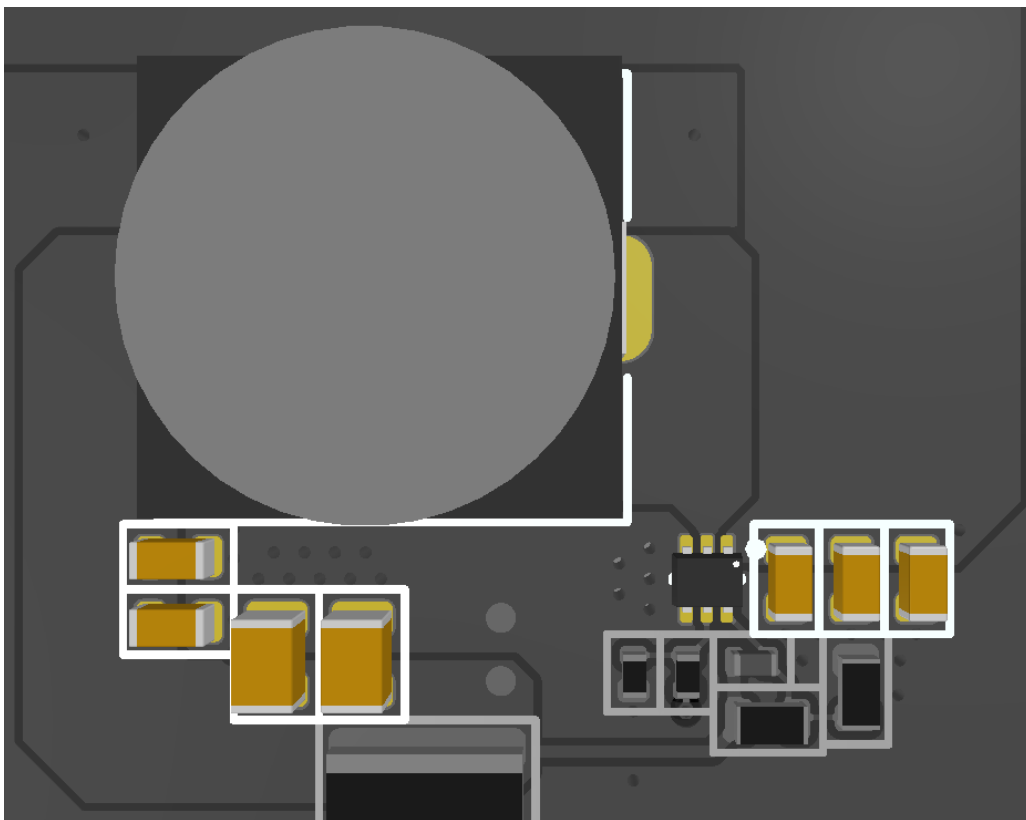


Hình 4.31: Minh họa tín hiệu Slave_I2C0 được định tuyến liên tục trên mặt phẳng GND đồng nhất, tránh các vị trí lớp phủ đã bị cắt.

Kỹ thuật Layout Hệ thống Nguồn và PDN: Hệ thống nguồn đóng vai trò then chốt trong việc đảm bảo độ ổn định cho toàn bộ thiết bị. Dựa trên các hướng dẫn thiết kế (Design Guidelines) từ nhà sản xuất IC và nguyên lý điện từ trường, quy trình layout tuân thủ các nguyên tắc nghiêm ngặt sau:

1. Tối ưu hóa Vòng lặp Dòng điện (Hot Loop):

- Các tụ điện đầu vào (C_{IN}) và đầu ra (C_{OUT}) là nguồn cung cấp năng lượng tức thời cho bộ chuyển đổi. Chúng được bố trí sát nhất có thể với các chân V_{IN}/GND và $V_{OUT}/PGND$ của IC nguồn hoặc cuộn cảm cho nguồn xung.
- Mục tiêu là giảm thiểu diện tích vật lý của "vòng lặp xung" (Hot Loop Area) - nơi có dòng điện biến thiên di/dt lớn nhất. Việc này giúp giảm cảm kháng ký sinh đường dây, hạn chế các xung gai điện áp (Voltage Spikes) và giảm thiểu bức xạ nhiễu EMI ra môi trường.



Hình 4.32: Vị trí các tụ điện trong nguồn xung +5V0_SYS.

2. Xử lý Điểm chuyển mạch (Switching Node):

- Chân SW (Switching) là nơi dao động điện áp tần số cao với biên độ lớn (dv/dt cao). Đường mạch tại đây được thiết kế dạng khối đồng (Polygon) ngắn và rộng để giảm điện trở và cảm kháng, nhưng diện tích không được quá lớn để hạn chế điện dung ký sinh gây tổn hao.
- Thiết kế đảm bảo tuyệt đối không định tuyến bất kỳ đường tín hiệu nhạy cảm nào đi ngang qua hoặc đi ở lớp ngay bên dưới (Layer 2) của cuộn cảm và điểm SW để tránh nhiễu.



Hình 4.33: Điểm chuyển mạch trong nguồn xung +5V0_SYS.

3. Kỹ thuật Hồi tiếp (Feedback) và Ground:

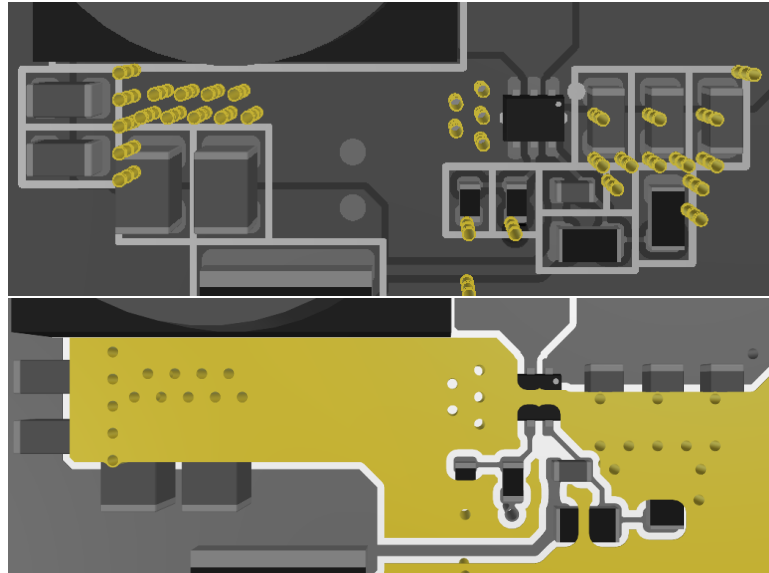
- **Kết nối Kelvin:** Đường tín hiệu hồi tiếp (Feedback) được lấy mẫu trực tiếp tại điểm sau tụ đầu ra C_{OUT} để đảm bảo điện áp đo được là chính xác nhất.
- **Cách ly nhiễu:** Đường Feedback được đi tránh xa khỏi cuộn cảm và điểm SW. Ground của đường hồi tiếp và các linh kiện tín hiệu nhỏ (như tụ Soft-start, trở định thời) được nối vào Analog Ground (AGND) hoặc chân GND tín hiệu của IC, tách biệt với Power Ground (PGND) đang chịu dòng xung lớn, nhằm tránh nhiễu mặt đất (Ground Bounce).



Hình 4.34: Định tuyến hồi tiếp trong nguồn xung +5V0_SYS.

4. Quản lý Nhiệt (Thermal):

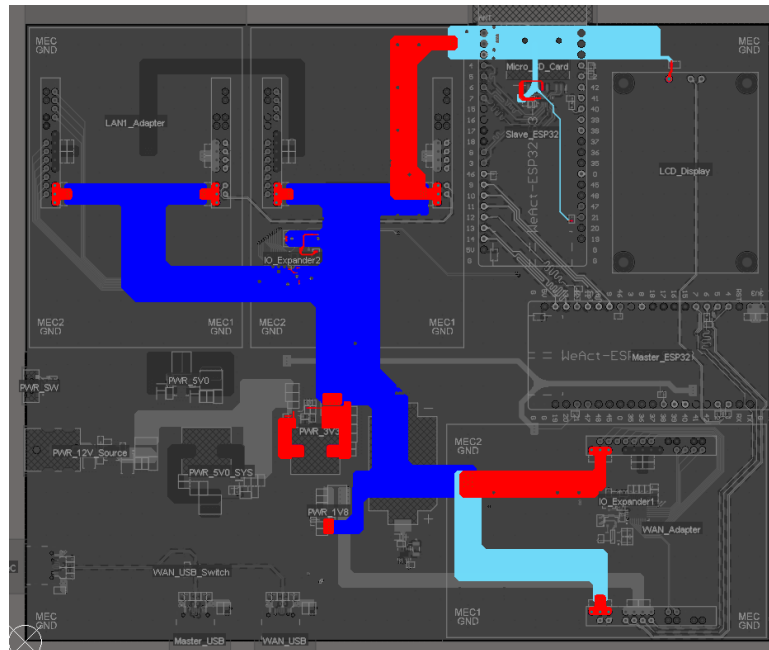
- Tận dụng Pad tản nhiệt dưới bụng IC, thiết kế sử dụng mảng via tản nhiệt dày đặc để dẫn nhiệt trực tiếp xuống các lớp đồng Ground Plane rộng lớn ở bên dưới (L2, L5). Điều này biến toàn bộ bo mạch thành một lá tản nhiệt tự nhiên, đảm bảo IC hoạt động mát mẻ dưới tải dòng lớn.



Hình 4.35: Tản nhiệt trong nguồn xung +5V0_SYS.

5. **Thiết kế Mạng phân phối nguồn (Power Distribution Network - PDN):** Mục tiêu cốt lõi là duy trì trở kháng nguồn (Z_{PDN}) thấp hơn trở kháng mục tiêu trên toàn dải tần số hoạt động. Chiến lược thực hiện bao gồm:

- **Tụ điện phẳng:** Trong cấu trúc Stack-up 6 lớp, việc bố trí Lớp 2 (Ground) và Lớp 3 (Power) liền kề nhau với lớp cách điện mỏng tạo ra một tụ điện ký sinh lớn, giúp lọc các nhiễu tần số cực cao mà tụ điện rời không đáp ứng kịp.
- **Tụ điện Decoupling đa tầng:** Sử dụng tổ hợp nhiều giá trị tụ điện song song. Các tụ giá trị nhỏ (10pF - 100nF) đặt sát chân cấp nguồn của IC để triệt tiêu nhiễu cao tần/xung nhịp MCU. Các tụ giá trị lớn (10 μ F - 100 μ F) đặt gần đó để đáp ứng nhu cầu năng lượng cho các biến động tải lớn.
- **Hình dạng đường dây:**
 - Các đường dẫn nguồn chính được thiết kế dạng mặt phẳng (Plane) hoặc đường mạch rất rộng. Theo công thức $L \propto \ln(1/W)$, đường mạch càng rộng thì cảm kháng ký sinh (L) càng nhỏ.
 - Việc giảm cảm kháng L giúp hạn chế hiện tượng sụt áp quá độ ($V_{drop} = L \cdot di/dt$) khi tải thay đổi đột ngột và giảm thiểu hiện tượng đổ chuông (Ringing) gây mất ổn định điện áp. Đồng thời, tiết diện dây lớn giúp giảm mật độ dòng điện, giảm tỏa nhiệt trên đường dây (IR Loss).

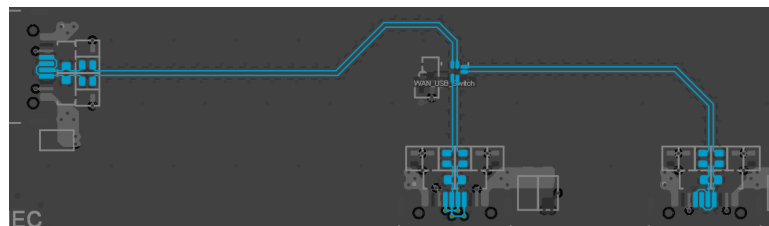


Hình 4.36: Mạng phân phối nguồn +3V3.

Kỹ thuật Định tuyến Tín hiệu Tốc độ cao (High-Speed Signal Routing) Để đảm bảo chất lượng tín hiệu (Signal Integrity) cho các giao tiếp USB, SPI và SDIO, quy trình layout tuân thủ các quy tắc định tuyến nghiêm ngặt nhằm kiểm soát trở kháng, đồng bộ pha và giảm thiểu xuyên nhiễu (Crosstalk):

1. Định tuyến Cặp Vi sai (Differential Pair Routing - USB):

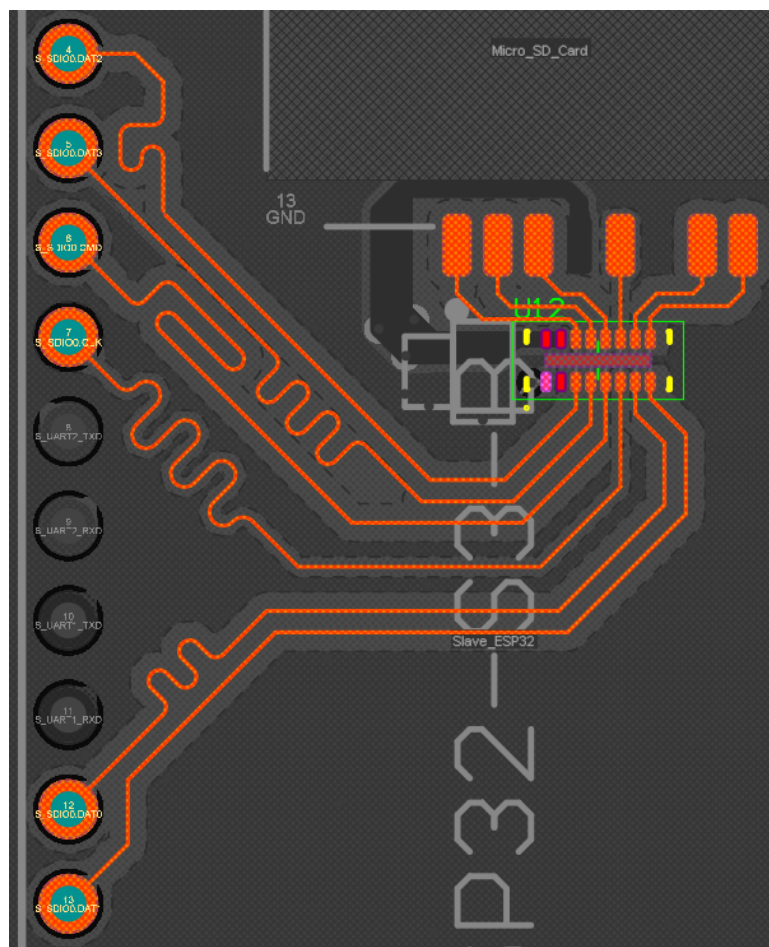
- **Kiểm soát Trở kháng:** Cặp tín hiệu vi sai USB D+/D- được thiết kế đạt trở kháng đặc tính mục tiêu $Z_{diff} = 90\Omega \pm 10\%$. Cấu trúc định tuyến sử dụng là *Differential Coplanar Waveguide* (vi sai đồng phẳng có mass bao bọc) để tăng khả năng chống nhiễu ngoại vi.
- **Đồng bộ pha:** Đảm bảo chênh lệch chiều dài giữa hai dây P và N trong cùng một cặp là nhỏ nhất có thể tùy theo tốc độ truyền dữ liệu. Kỹ thuật *Phase Matching* được thực hiện ngay tại điểm xảy ra lệch pha để đảm bảo tín hiệu đến bộ thu đồng thời, triệt tiêu nhiễu chế độ chung (Common Mode Noise).
- **Tính đối xứng:** Hai đường dây được định tuyến song song và đối xứng tuyệt đối trong suốt chiều dài đường truyền. Các góc cua sử dụng góc vát 135° hoặc cung tròn, tuyệt đối không dùng góc 90° để tránh thay đổi trở kháng đột ngột.



Hình 4.37: Định tuyến USB.

2. Định tuyến Bus Đơn cực (Single-ended Bus Routing - SPI/SDIO):

- **Trở kháng 50Ω:** Tất cả các đường tín hiệu dữ liệu và điều khiển được thiết kế với trở kháng $Z_0 = 50\Omega \pm 10\%$ theo cấu trúc *Coplanar Waveguide*.
- **Bảo vệ xung nhịp (Clock Shielding):** Tín hiệu Clock (CLK) là nguồn gây nhiễu mạnh nhất. Đường CLK được định tuyến biệt lập, bao bọc bởi hai đường mass bảo vệ và được khâu via xuống Ground Plane dọc theo đường đi để cô lập trường điện từ.
- **Cân bằng chiều dài (Trace Length Matching):** Áp dụng các kỹ thuật *Accordion* và *Trombone* để cân bằng chiều dài của các đường dữ liệu với đường Clock. Điều này đảm bảo dữ liệu được lấy mẫu chính xác tại sườn xung nhịp, tránh lỗi Setup/Hold time do độ trễ lan truyền (Propagation Delay).



Hình 4.38: Định tuyến SDIO giữa ESP32-S3 Slave với Micro SD Card.

3. Kiểm soát Nhiễu và Toàn vẹn tín hiệu (EMI & SI Control):

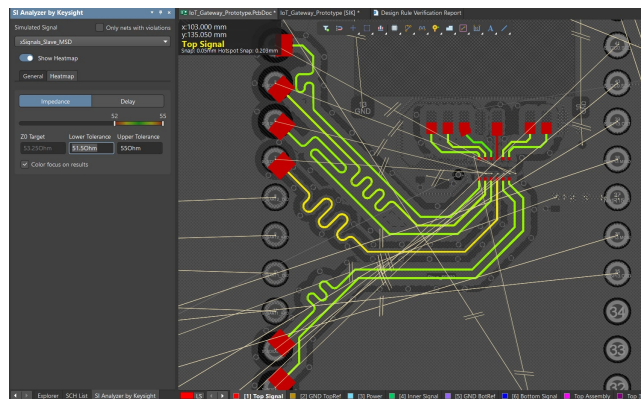
- **Giảm thiểu Via:** Hạn chế tối đa việc chuyển lớp trên đường tín hiệu tốc độ cao để tránh gây gián đoạn trở kháng. Nếu bắt buộc phải chuyển lớp, các stitching via được đặt ngay cạnh via tín hiệu để duy trì đường hồi tiếp liền mạch.
- **Định tuyến trực giao:** Để tránh hiện tượng xuyên nhiễu (Crosstalk) giữa các lớp liền kề được đi vuông góc với nhau. Tuyệt đối tránh đi dây song song chồng lên nhau.

- **Toàn vẹn mặt phẳng tham chiếu:** Tín hiệu tốc độ cao luôn được tham chiếu tới một mặt phẳng đất liền mạch (Solid Ground Plane). Tuyệt đối không định tuyến băng qua các khe hở (Splits) hoặc vùng trống (Voids) trên lớp đất để tránh làm gây khúc đường hồi tiếp dòng điện.
- **Quy tắc khoảng cách:** Áp dụng quy tắc 3W hoặc 5W (khoảng cách giữa các đường mạch lớn hơn 3-5 lần bề rộng mạch) để giảm thiểu xuyên nhiễu giữa các bus tín hiệu khác nhau. Đồng thời, duy trì khoảng cách ly đối với các linh kiện gây nhiễu như thạch anh, cuộn cảm nguồn.

4.5.2.4 Phân tích và Mô phỏng Tín hiệu (Signal Integrity Simulation)

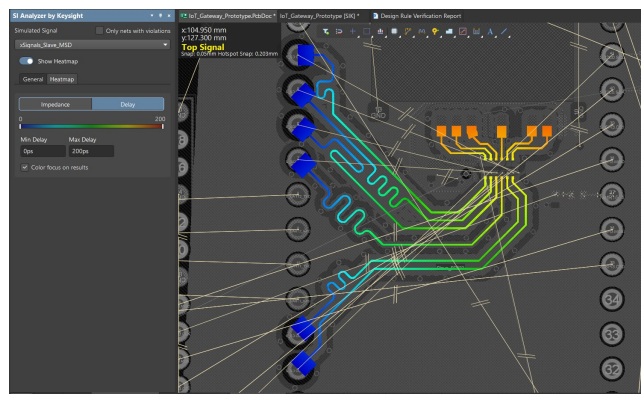
Kiểm soát Trở kháng và độ trễ lan truyền (Impedance & Delay Control) Việc mô phỏng được thực hiện trên các đường tín hiệu quan trọng để xác minh tính toán lý thuyết so với thiết kế layout thực tế.

1. Mô phỏng đường tín hiệu đơn cực (Single-ended Bus):



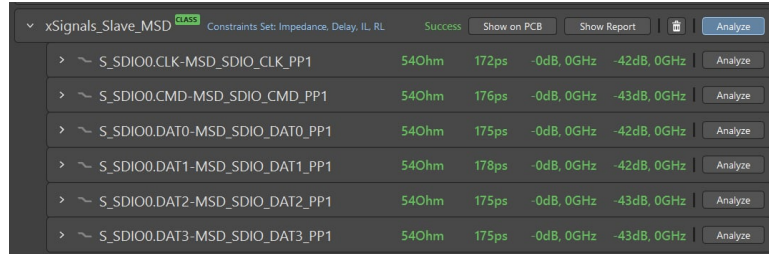
Hình 4.39: Biểu đồ phân bố trở kháng (Impedance Heatmap) đường tín hiệu đơn cực.

Kết quả mô phỏng cho thấy trở kháng đặc tính (Z_0) dao động trong khoảng từ 51.5Ω đến 55Ω . Mức biến thiên này hoàn toàn nằm trong phạm vi dung sai thiết kế cho phép ($\pm 10\%$ so với mục tiêu 50Ω), đảm bảo phối hợp trở kháng tốt với các linh kiện chuẩn. Tuy có sự mismatch tại các pinout nhưng độ dài mismatch là đủ nhỏ để không ảnh hưởng quá lớn đến trở kháng chung.



Hình 4.40: Biểu đồ phân bố độ trễ lan truyền (Propagation Delay) đường tín hiệu đơn cực.

Độ trễ lan truyền của toàn bộ các đường tín hiệu được kiểm soát dưới ngưỡng $200ps$. Sự đồng nhất của dải màu trên biểu đồ heatmap dọc theo chiều dài đường mạch cho thấy độ lệch pha giữa các tín hiệu là nhỏ, đảm bảo các bit dữ liệu đến đích đồng bộ với xung nhịp.



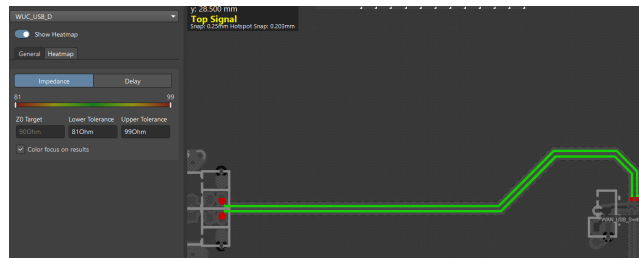
Signal	Impedance	Delay	IL	RL	Analysis
S_SDIO0.CLK-MSD_SDIO_CLK_PP1	540hm	172ps	-0dB, 0GHz	-42dB, 0GHz	Analyze
S_SDIO0.CMD-MSD_SDIO_CMD_PP1	540hm	176ps	-0dB, 0GHz	-43dB, 0GHz	Analyze
S_SDIO0.DAT0-MSD_SDIO_DAT0_PP1	540hm	175ps	-0dB, 0GHz	-42dB, 0GHz	Analyze
S_SDIO0.DAT1-MSD_SDIO_DAT1_PP1	540hm	178ps	-0dB, 0GHz	-42dB, 0GHz	Analyze
S_SDIO0.DAT2-MSD_SDIO_DAT2_PP1	540hm	175ps	-0dB, 0GHz	-43dB, 0GHz	Analyze
S_SDIO0.DAT3-MSD_SDIO_DAT3_PP1	540hm	175ps	-0dB, 0GHz	-43dB, 0GHz	Analyze

Hình 4.41: Bảng tổng hợp thông số mô phỏng đường tín hiệu đơn cực.

Số liệu chi tiết từ báo cáo xác nhận:

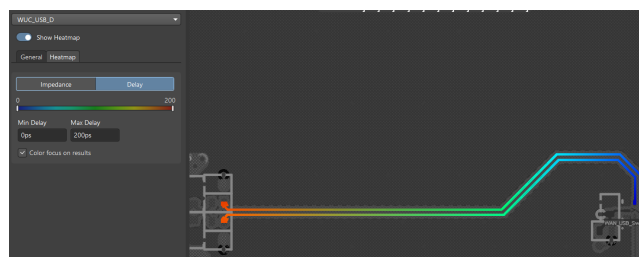
- Trở kháng trung bình đạt 54Ω .
- Độ trễ dao động trong biên độ hẹp ($175ps$ - $178ps$), cho thấy việc cân bằng chiều dài (Length matching) đã được thực hiện tốt.
- Tổn hao chèn (Insertion Loss) gần như bằng $0dB$ và Tổn hao phản xạ (Return Loss) đạt $-42dB$ (rất thấp), chứng tỏ năng lượng tín hiệu được truyền đi hiệu quả và phản xạ tại tải là không đáng kể.

2. Mô phỏng cặp tín hiệu vi sai (Differential-pair):



Hình 4.42: Biểu đồ phân bố trở kháng (Impedance Heatmap) đường vi sai.

Trở kháng vi sai (Z_{diff}) mô phỏng đạt giá trị ổn định quanh mức 90Ω . Kết quả này tuân thủ chặt chẽ tiêu chuẩn yêu cầu $90\Omega \pm 10\%$. Tuy có sự mismatch tại các pinout nhưng độ dài mismatch là đủ nhỏ để không ảnh hưởng quá lớn đến trở kháng chung.



Hình 4.43: Biểu đồ phân bố độ trễ lan truyền (Propagation Delay) đường vi sai.

Thời gian độ trễ lan truyền được duy trì dưới $200ps$. Sự tương đồng về màu sắc trên hai nhánh D+ và D- khẳng định độ lệch pha là nhỏ, giúp triệt tiêu hiệu quả nhiễu chế độ chung (Common Mode Noise).



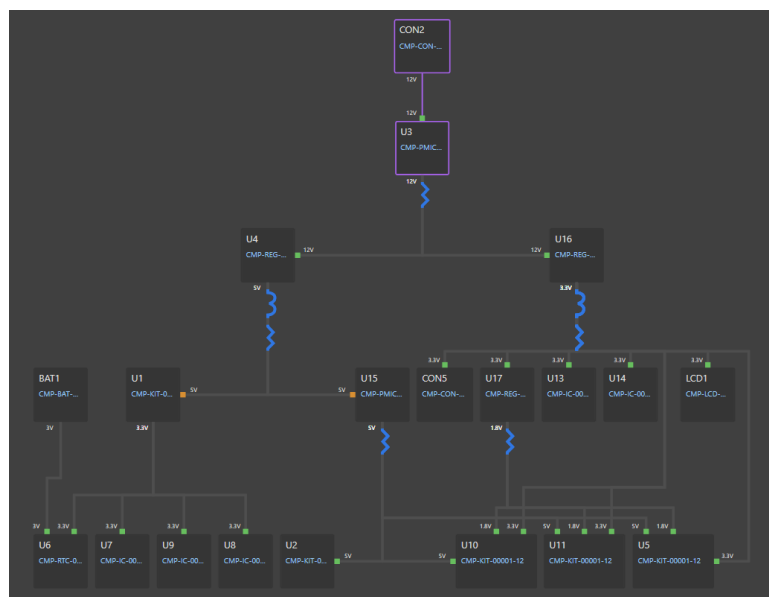
Hình 4.44: Bảng tổng hợp thông số mô phỏng đường vi sai.

Kết quả tổng hợp cho thấy:

- Trở kháng vi sai đạt chuẩn 90Ω .
- Độ trễ đường truyền là $187ps$.
- Các thông số S-parameter rất tích cực: Insertion Loss xấp xỉ $0dB$ và Return Loss đạt $-22dB$, đảm bảo tính toàn vẹn tín hiệu cho giao tiếp dữ liệu tốc độ cao.

Phân tích và Mô phỏng Mạng phân phối nguồn (Power Distribution Network - PDN)

1. **Sơ đồ Cây phân phối nguồn (Power Tree):** Hệ thống được thiết kế thành cấu trúc nguồn phân tầng đa nhánh phức tạp, được thể hiện trong Hình 4.45. Mạng lưới này không chỉ đơn thuần là cấp điện, mà là một hệ thống quản lý năng lượng phân tán với nhiều đặc điểm kỹ thuật cần phải đảm bảo trong thiết kế Mạng phân phối nguồn (Power Distribution Network - PDN).



Hình 4.45: Cấu trúc mạng phân phối nguồn.

Với cấu trúc chằng chịt và dòng tải biến động lớn, việc tính toán lý thuyết và thực hiện Mô phỏng PDN để kiểm chứng độ sụt áp (DC IR Drop) và mật độ dòng điện (Current Density), đảm bảo không xảy ra quá nhiệt cục bộ hoặc sụt áp vượt ngưỡng an toàn tại các module mở rộng.

Dựa trên thiết kế, mô phỏng tập trung vào các ray nguồn quan trọng ở trạng thái cực đoan với tải lý thuyết cao nhất, bao gồm các nguồn có tải cao:

- Nguồn +5V0_SYS (nguồn U4)
- Nguồn +5V0 (nguồn U15)
- Nguồn +3V3 (nguồn U16)

2. Phân tích và Mô phỏng Sụt áp DC (DC IR Drop Analysis):

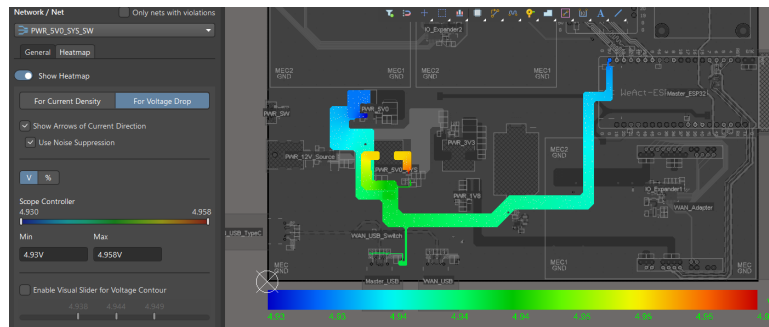
Mục tiêu của quá trình mô phỏng là xác minh tính toàn vẹn của mạng phân phối nguồn (PDN) dưới điều kiện tải nặng nhất (Worst-case scenario). Việc sụt áp xảy ra do nội trở của nguồn và điện trở ký sinh trên đường mạch.

Tiêu chí chấp nhận:

- Tổng độ sụt áp từ nguồn (Source) đến tải (Load) không vượt quá **5%** giá trị điện áp danh định.
- Điện trở đường dẫn phải được tối thiểu hóa (cỡ $m\Omega$) để giảm tổn hao nhiệt.
- Công thức tính trở kháng đường dẫn: $R_{path} = \frac{\Delta V_{drop}}{I_{load}}$.

(a) Phân tích Ray nguồn +5V0_SYS:

Mô phỏng được thực hiện với dòng tải tiêu thụ cực đại trên thiết kế là $I_{load} = 2.92A$.



Hình 4.46: Biểu đồ phân bố điện áp (Voltage Drop Heatmap) trên nguồn +5V0_SYS.

Kết quả mô phỏng và Tính toán:

- Điện áp tại điểm nguồn: $V_{source} = 4.958V$.
- Điện áp tại điểm tải xa nhất: $V_{load} = 4.930V$.
- Chênh lệch điện áp: $\Delta V = 4.958V - 4.930V = 28mV$.

Đánh giá:

- Phần trăm sụt áp so với mức danh định (5V): $\%Drop = \frac{5V - 4.93V}{5V} = 1.4\%$

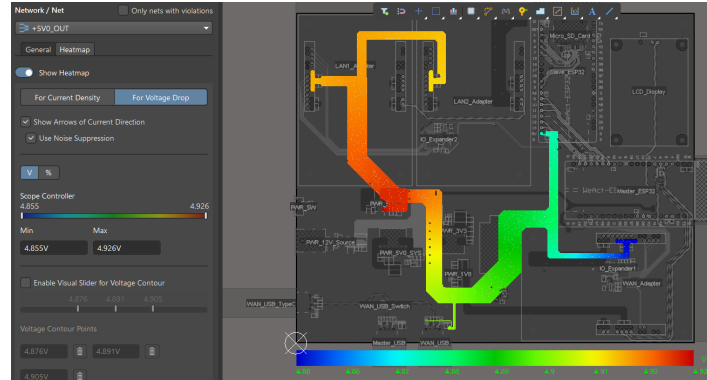
⇒ Kết quả nhỏ hơn giới hạn 5%.

- Điện trở đường dẫn: $R_{path} = \frac{V_{source} - V_{load}}{I_{load}} = \frac{28mV}{2.92A} \approx 9.59m\Omega$

⇒ Trở kháng đường dây rất thấp, cho thấy thiết kế layout nguồn tốt.

(b) Phân tích Ray nguồn +5V0:

Mô phỏng được thực hiện với dòng tải tiêu thụ cực đại trên thiết kế là $I_{load} = 2.46A$.



Hình 4.47: Biểu đồ phân bố điện áp (Voltage Drop Heatmap) trên nguồn +5V0.

Kết quả mô phỏng và Tính toán:

- Điện áp tại điểm nguồn: $V_{source} = 4.926V$.
- Điện áp tại điểm tải xa nhất: $V_{load} = 3.163V$.
- Chênh lệch điện áp: $\Delta V = 4.926V - 3.163V = 71mV$.

Đánh giá:

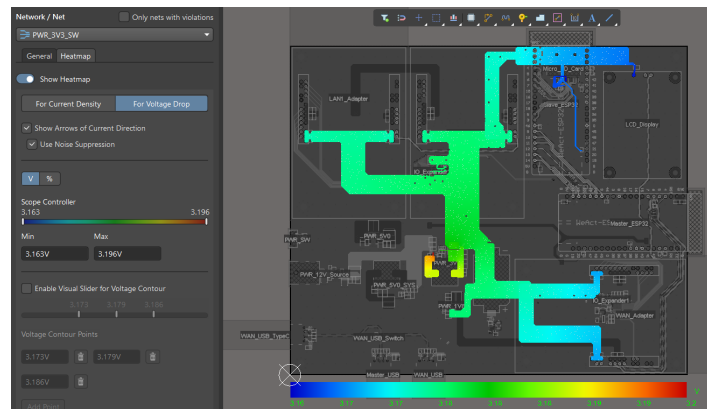
- Phần trăm sụt áp so với mức danh định (5V): $\%Drop = \frac{5V - 3.163V}{5V} = 2.9\%$
⇒ Kết quả nhỏ hơn giới hạn 5%.

- Điện trở đường dẫn: $R_{path} = \frac{V_{source} - V_{load}}{I_{load}} = \frac{71mV}{2.46A} \approx 2.89m\Omega$

⇒ Trở kháng đường dây rất thấp, cho thấy thiết kế layout nguồn tốt.

(c) Phân tích Ray nguồn +3V3:

Mô phỏng được thực hiện với dòng tải tiêu thụ cực đại trên thiết kế là $I_{load} = 3.3A$.



Hình 4.48: Biểu đồ phân bố điện áp (Voltage Drop Heatmap) trên nguồn +3V3.

Kết quả mô phỏng và Tính toán:

- Điện áp tại điểm nguồn: $V_{source} = 3.196V$.
- Điện áp tại điểm tải xa nhất: $V_{load} = 3.163V$.
- Chênh lệch điện áp: $\Delta V = 3.196V - 3.163V = 33mV$.

Đánh giá:

- Phần trăm sụt áp so với mức danh định (3.3V): $\%Drop = \frac{3.3V - 3.163V}{3.3V} = 4.15\%$
⇒ Kết quả tuy nhỏ hơn nhưng khá gần giới hạn 5%.
- Điện trở đường dẫn: $R_{path} = \frac{V_{source} - V_{load}}{I_{load}} = \frac{33mV}{3.3A} \approx 10m\Omega$
⇒ Trở kháng đường dây rất thấp, cho thấy thiết kế layout nguồn tốt. Song mức sụt áp khá gần giới hạn, cho thấy sụt áp xảy ra tại nguồn là khá cao.

3. Phân tích và Mô phỏng Mật độ dòng điện (Current Density Analysis):

Bên cạnh sụt áp, mật độ dòng điện là tham số quan trọng quyết định độ bền vật lý của bo mạch. Mật độ dòng quá cao sẽ gây ra hai vấn đề:

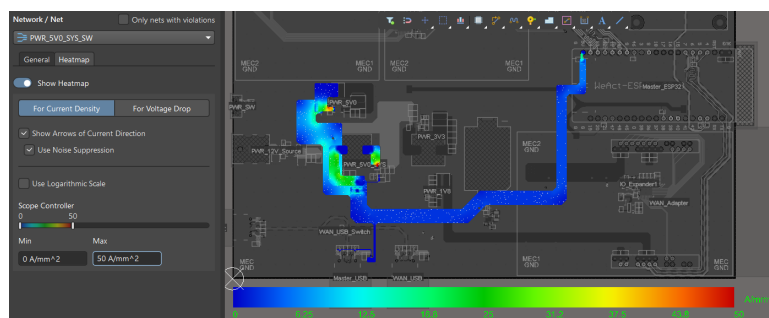
- **Quá nhiệt (Overheating):** Tổn hao nhiệt (I^2R) tập trung tại một điểm nhỏ có thể nung nóng đường mạch, làm bong tróc lớp đồng hoặc hỏng lớp cách điện PCB.
- **Giảm hiệu suất và tăng nhiễu điện từ:** Mật độ dòng cao làm tăng điện trở gây sụt áp, ngoài ra với dòng chuyển mạch nhanh, mật độ dòng quá lớn gây ra tác động EMI mạnh mẽ hơn.

Tiêu chí thiết kế:

- Tuân thủ tiêu chuẩn IPC-2221: Mức tăng nhiệt độ tối đa $\Delta T \leq 10^\circ C$.
- Ngưỡng mật độ dòng điện trung bình an toàn: $J_{avg} \leq 50A/mm^2$ (đối với lớp đồng 1oz).

(a) Phân tích Ray nguồn +5V0_SYS:

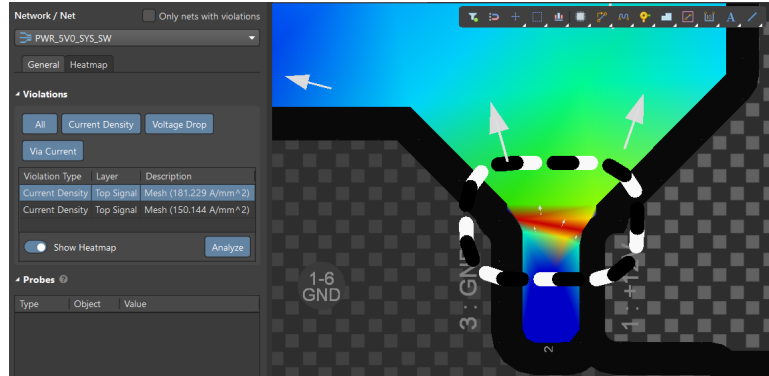
Mô phỏng được thực hiện với dòng tải tiêu thụ cực đại trên thiết kế là $I_{load} = 2.92A$.



Hình 4.49: Biểu đồ phân bố mật độ dòng điện (Current Density Heatmap) trên nguồn +5V0_SYS.

Kết quả tổng thể: Quan sát Hình 4.49, mật độ dòng điện trên phần lớn chiều dài đường mạch được duy trì ở mức thấp (màu xanh dương/xanh lá), nằm hoàn toàn trong ngưỡng an toàn ($< 30A/mm^2$). Điều này xác nhận kích thước các đường nguồn đã được thiết kế đủ rộng.

Phân tích điểm có mật độ dòng điện cao đột biến:



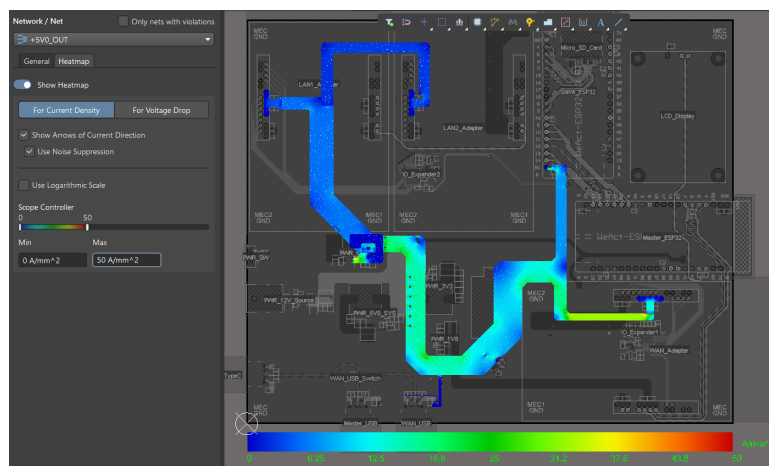
Hình 4.50: Mật độ dòng điện cao đột biến tại pinout nguồn xung +5V0_SYS.

Tại vị trí pinout của IC nguồn xung (Hình 4.50), ghi nhận mật độ dòng điện tăng vọt lên mức khá cao $J \approx 181.229A/mm^2$. Mặc dù giá trị này vượt ngưỡng an toàn, nhưng thiết kế vẫn được đảm bảo:

- Mật độ cao chỉ xuất hiện tại tiết diện rất nhỏ ngay pinout linh kiện. Đây là giới hạn vật lý của linh kiện mà không thể can thiệp bằng layout. Với tiết diện đoạn mạch chịu dòng cao là cực nhỏ ($< 0.25mm \times 0.45mm$), rủi ro về sụt áp là không đáng kể.
- Ngay sau điểm đó, đường mạch lập tức được mở rộng thành mảng đồng lớn. Nhiệt lượng sinh ra tại điểm này sẽ nhanh chóng được dẫn truyền và tản ra khu vực đồng xung quanh, ngăn chặn việc tăng nhiệt độ quá mức.

(b) Phân tích Ray nguồn +5V0:

Mô phỏng được thực hiện với dòng tải tiêu thụ cực đại trên thiết kế là $I_{load} = 2.46A$.

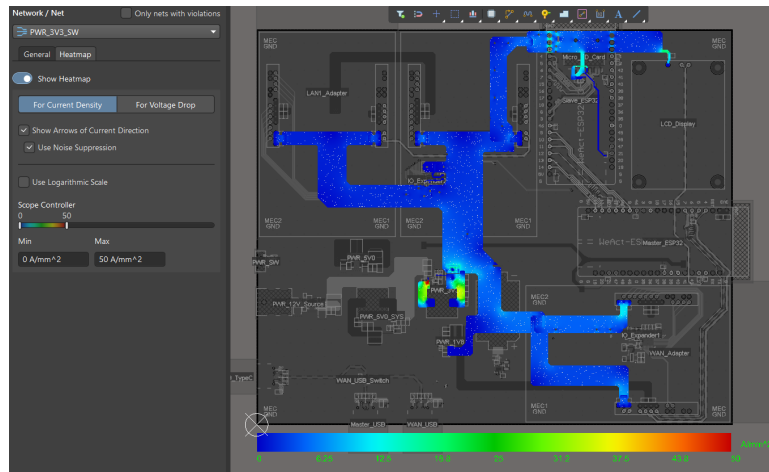


Hình 4.51: Biểu đồ phân bố mật độ dòng điện (Current Density Heatmap) trên nguồn +5V0.

Kết quả tổng thể: Quan sát Hình 4.51, mật độ dòng điện trên phần lớn chiều dài đường mạch được duy trì ở mức thấp (màu xanh dương/xanh lá), nằm hoàn toàn trong ngưỡng an toàn ($< 30A/mm^2$). Điều này xác nhận kích thước các đường nguồn đã được thiết kế đủ rộng.

(c) **Phân tích Ray nguồn +3V3:**

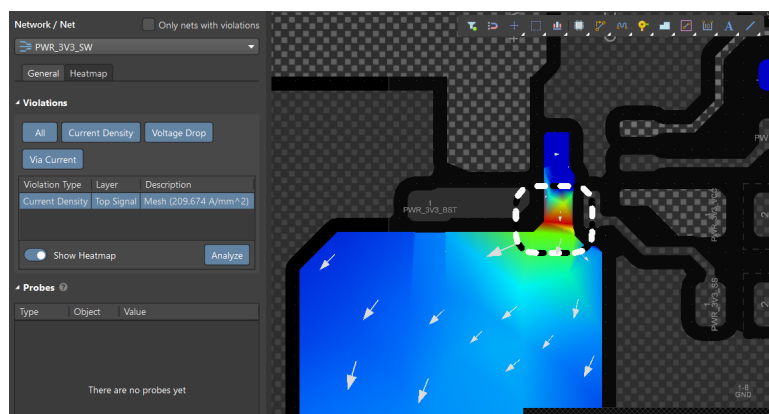
Mô phỏng được thực hiện với dòng tải tiêu thụ cực đại trên thiết kế là $I_{load} = 3.3A$.



Hình 4.52: Biểu đồ phân bố mật độ dòng điện (Current Density Heatmap) trên nguồn +3V3.

Kết quả tổng thể: Quan sát Hình 4.52, mật độ dòng điện trên phần lớn chiều dài đường mạch được duy trì ở mức thấp (màu xanh dương/xanh lá), nằm hoàn toàn trong ngưỡng an toàn ($< 30A/mm^2$). Điều này xác nhận kích thước các đường nguồn đã được thiết kế đủ rộng.

Phân tích điểm có mật độ dòng điện cao đột biến:



Hình 4.53: Mật độ dòng điện cao đột biến tại pinout nguồn xung +3V3.

Tương tự với nguồn +5V0_SYS, tại vị trí pinout của IC nguồn xung (Hình 4.53), ghi nhận mật độ dòng điện tăng vọt lên mức khá cao $J \approx 209.674A/mm^2$. Mặc dù giá trị này vượt ngưỡng an toàn, nhưng thiết kế vẫn được đảm bảo.

4.5.3 Firmware và Software

Tổng quan kiến trúc phần mềm Dual-MCU Prototype phần mềm của IoT Gateway được triển khai theo kiến trúc Dual-MCU Master-Slave và vận hành trên FreeRTOS. Hệ thống được chia thành hai miền xử lý tách bạch: LAN-MCU phụ trách thu thập và xử lý dữ liệu từ thiết bị đầu cuối/sensor; WAN-MCU phụ trách kết nối Internet/Cloud và điều phối luồng dữ liệu, cấu hình và cập nhật. Cách phân tách này giúp giảm tải cho từng MCU, tăng tính ổn định khi chạy lâu dài và tạo nền tảng mở rộng giao thức/module theo nhu cầu.

Về cấu trúc phần mềm, cả hai firmware đều tuân theo mô hình phân lớp Application / Middleware / BSP / Driver.

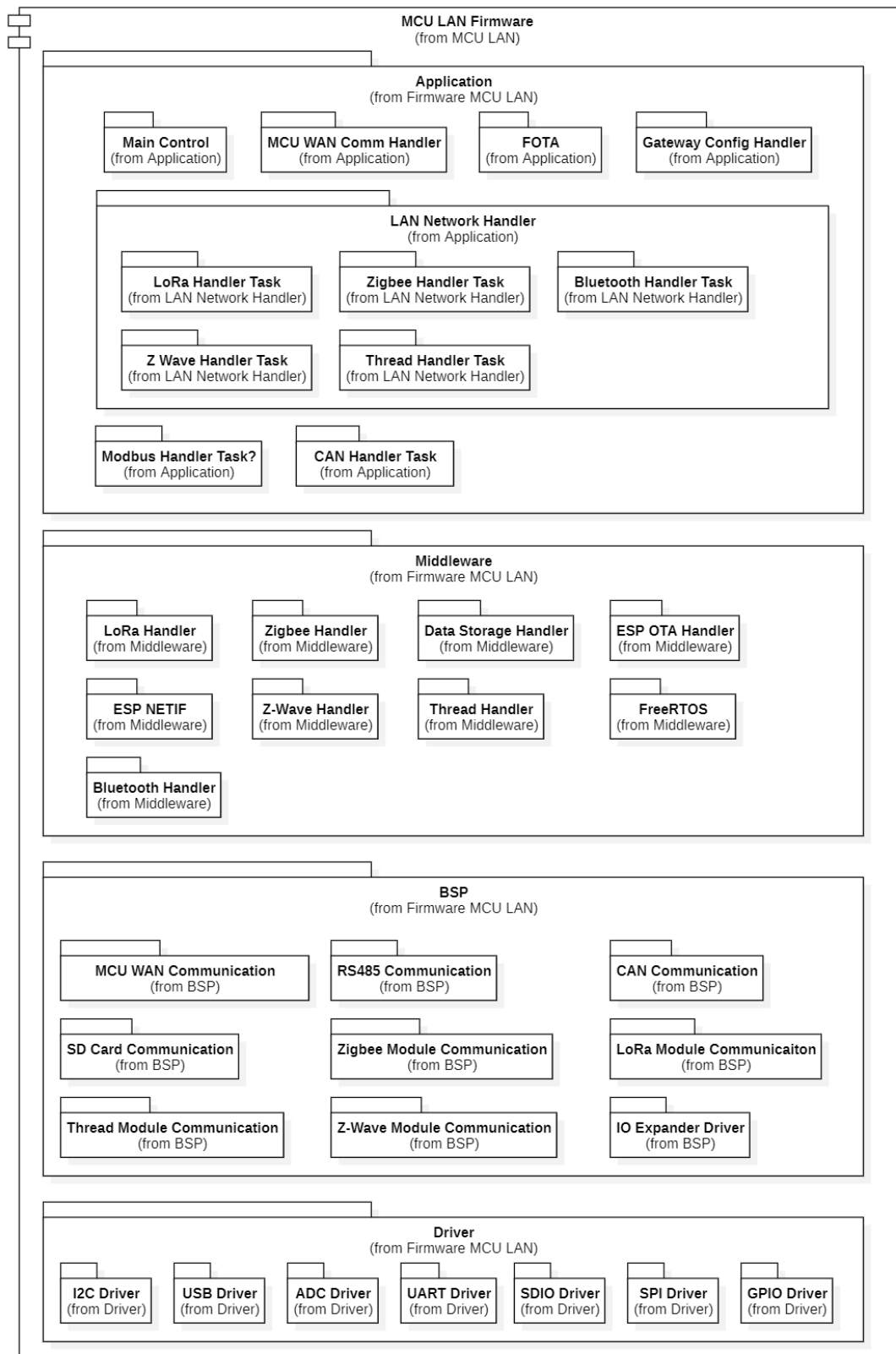
- **Application** chứa các khối điều phối cấp cao và các task xử lý chính.
- **Middleware** cung cấp lớp trừu tượng giao thức/dịch vụ nền (network stack, MQTT/OTA, storage...).
- **BSP** hiện thực giao tiếp phần cứng theo board/module (SPI/UART/SD/IO expander...).
- **Driver** là lớp thấp nhất cho ngoại vi (GPIO/I2C/SPI/UART/USB/SDIO/MAC...).

Cấu trúc này hỗ trợ tốt mục tiêu modularity (dễ thêm/bớt giao thức, module) và configurability (đổi cấu hình mà không phải sửa kiến trúc chương trình).

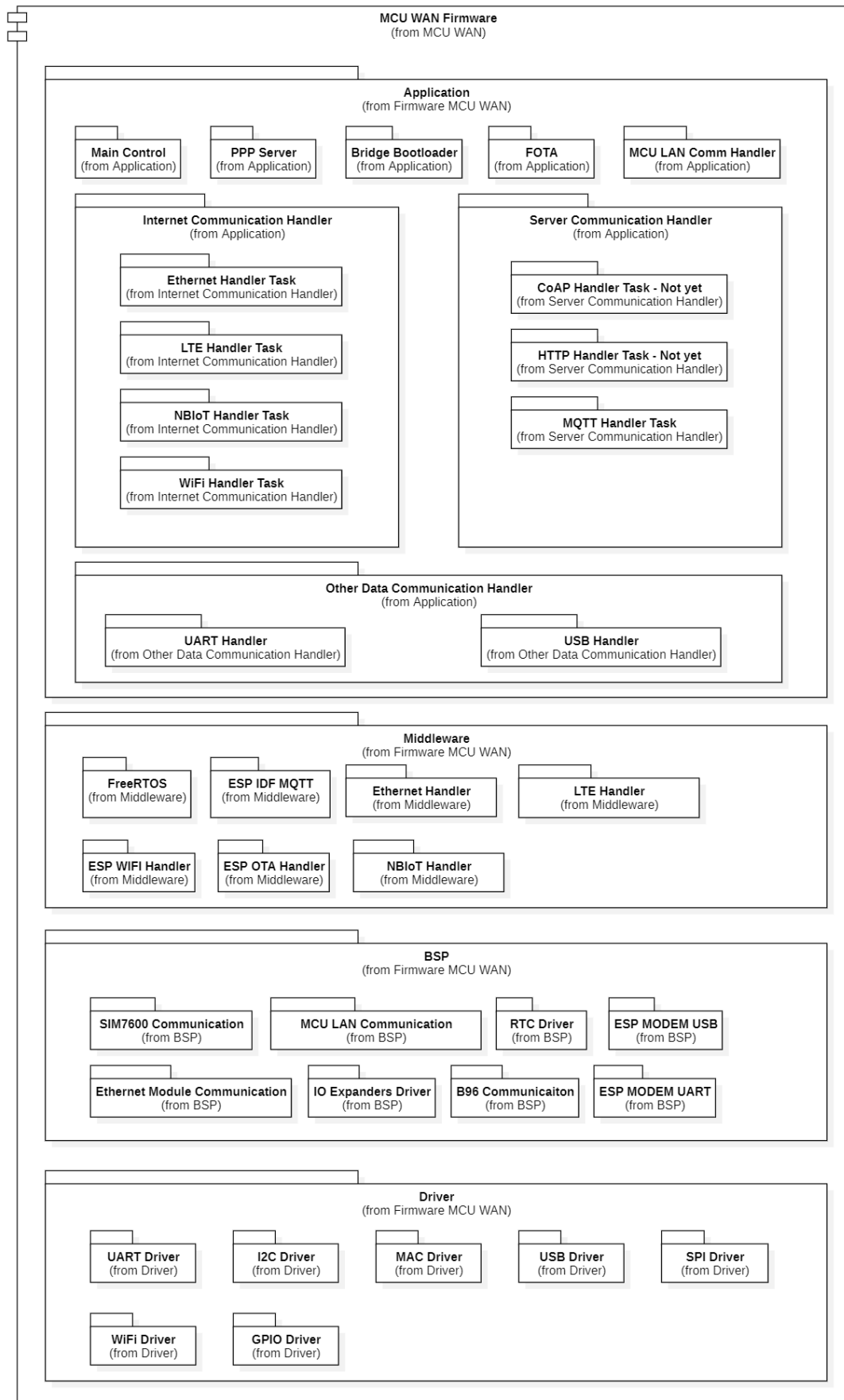
Ở phía LAN-MCU, firmware đảm nhiệm “mặt dưới” của gateway: quản lý các kết nối nội bộ và thiết bị đầu cuối. Tầng Application tổ chức theo nhiều task theo từng giao thức (LoRa, Zigbee, BLE, Thread, Z-Wave, có thể mở rộng CAN/RS-485), thực hiện nhận dữ liệu thô, kiểm tra hợp lệ và chuẩn hóa trước khi chuyển lên kênh liên MCU. Tầng Middleware cung cấp các handler giao thức, Data Storage để đệm/lưu cục bộ (SD) khi cần và OTA cho cập nhật; BSP/Driver hiện thực các giao tiếp phần cứng và ngoại vi.

Ở phía WAN-MCU, firmware đảm nhiệm “mặt trên”: kết nối Internet/Cloud và điều phối hệ thống. Tầng Application gồm Internet handler (Wi-Fi/LTE/Ethernet/PPP), server handler (ưu tiên MQTT để publish telemetry và nhận downlink), kênh cấu hình USB/UART, và khối giao tiếp với LAN-MCU để nhận uplink/chuyển downlink. Tầng Middleware sử dụng các thành phần nền (MQTT, network handlers, OTA), còn BSP/Driver quản lý modem/ethernet, RTC và các ngoại vi như SPI/UART/USB, IO expander.

Tổng thể, kiến trúc Dual-MCU tạo luồng dữ liệu rõ ràng: LAN-MCU thu thập & tiền xử lý, WAN-MCU gửi Cloud & nhận lệnh, hai MCU trao đổi qua SPI có cơ chế xác nhận để tăng độ tin cậy. Nhờ tách vai trò và phân lớp phần mềm, prototype vừa linh hoạt theo module/giao thức vừa ổn định, dễ triển khai thực tế.



Hình 4.54: MCU LAN Firmware



Hình 4.55: MCU WAN Firmware

Firmware WAN-MCU: Kết nối Internet, Cloud và điều phối hệ thống Trong prototype, WAN-MCU đóng vai trò là “đầu não giao tiếp ra Internet/Cloud” đồng thời là “router điều phối cấu hình” của toàn gateway. Các giao thức kết nối cảm biến (Zigbee, LoRa, CAN, RS-485,...) được tách khỏi WAN-MCU và triển khai tại LAN-MCU, giúp WAN-MCU giảm tải, tăng ổn định và tập trung vào các nhiệm vụ chính: thiết lập kết nối Internet, giao tiếp Cloud, truyền/nhận dữ liệu với LAN-MCU và quản trị cấu hình.

Khối Internet Communication Handler chịu trách nhiệm thiết lập và duy trì đường truyền Internet theo cấu hình người dùng. Trong prototype, các chế độ kết nối chính bao gồm:

- **Wi-Fi:** hỗ trợ hai chế độ Wi-Fi Personal và Wi-Fi Enterprise. Sau khi kết nối thành công và nhận cấu hình mạng, hệ thống thực hiện đồng bộ thời gian (SNTP) và cập nhật RTC nhằm đảm bảo timestamp thống nhất cho toàn gateway. Khối này cũng theo dõi trạng thái liên kết và tự reconnect khi mất mạng.
- **LTE:** quản lý kết nối modem LTE qua USB, bao gồm thiết lập đường truyền và giám sát chất lượng kết nối. Khi xảy ra mất kết nối, WAN-MCU thực hiện cơ chế tự khôi phục (reconnect) theo chính sách cấu hình.

PPP Server: WAN-MCU có thể bật PPP server qua UART để tạo một đường IP-link phục vụ các kịch bản triển khai đặc thù. Trong prototype, PPP server được sử dụng như một cơ chế hỗ trợ để MCU LAN thực hiện cập nhật FOTA (WAN-MCU đóng vai trò cung cấp/duy trì đường truyền).

Server Communication Handler (MQTT) – Uplink telemetry và Downlink command Trong prototype, Server Communication Handler tập trung chủ yếu vào MQTT (HTTP / CoAP được thể hiện trong sơ đồ để định hướng mở rộng nhưng chưa triển khai).

Uplink (telemetry):

- Dữ liệu uplink từ LAN-MCU sau khi chuyển lên WAN-MCU sẽ được đưa vào hàng đợi publish (publish queue).
- Một publish task riêng chịu trách nhiệm xử lý hàng đợi theo pipeline:
 1. Lấy dữ liệu từ queue,
 2. Đóng gói payload (prototype thực hiện nhị phân → hex string và bọc trong JSON theo định dạng quy ước),
 3. Publish lên broker theo topic telemetry.
- Cơ chế queue giúp tách biệt hai luồng “nhận dữ liệu từ SPI” và “gửi Cloud”, tránh việc SPI bị nghẽn khi broker chậm hoặc mạng dao động.

Downlink (RPC/command):

- WAN-MCU subscribe các topic downlink (RPC request/command).
- Khi nhận message, MQTT handler thực hiện:
 1. Trích payload (thường là JSON),
 2. Lấy trường tham số (ví dụ params, dạng chuỗi hex hoặc payload quy ước),
 3. Decode về binary và xác định đích xử lý theo loại handler (ZIG/LOR/CAN/RS4...),

4. Đưa command vào downlink queue để chuyển tiếp xuống LAN-MCU.

MCU-to-MCU Handler (WAN phía SPI Slave) – nhận uplink, trả ACK và chủ động đẩy downlink qua GPIO handshake Đây là khối quan trọng nhất của prototype vì WAN-MCU và LAN-MCU bắt buộc giao tiếp qua SPI, với mô hình LAN-MCU là SPI Master và WAN-MCU là SPI Slave. Khối MCU-to-MCU Handler đảm nhiệm hai chiều dữ liệu: uplink (LAN → WAN) và downlink (Cloud → LAN), đồng thời cung cấp cơ chế xác nhận để đảm bảo tin cậy truyền thông.

Cơ chế uplink (LAN → WAN): LAN-MCU đóng gói dữ liệu cảm biến thành gói DT và gửi qua SPI. WAN-MCU nhận gói, thực hiện kiểm tra hợp lệ tối thiểu (độ dài, CRC/sequence nếu áp dụng) và phản hồi ACK/NACK ngay trong phiên truyền để LAN-MCU quyết định gửi lại khi cần. Trong prototype, ACK được kèm “cờ Internet” nhằm phản ánh trạng thái kết nối của WAN-MCU tại thời điểm nhận dữ liệu:

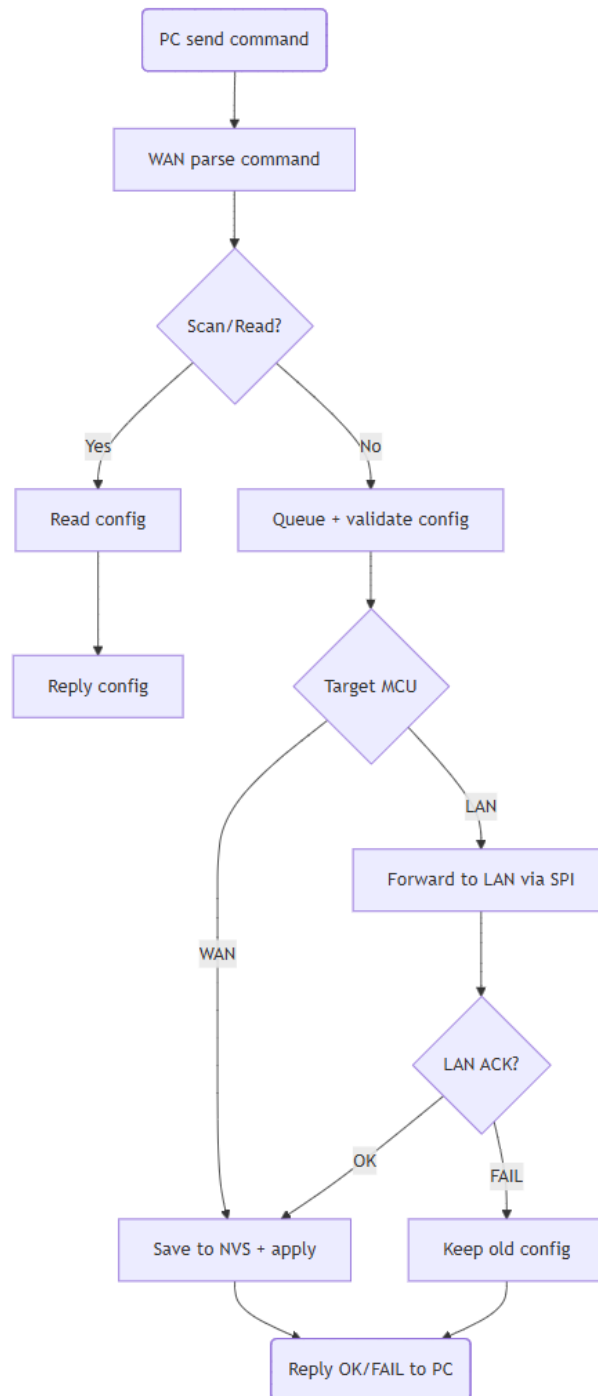
- Nếu WAN đang online (thường tương ứng trạng thái MQTT connected), ACK báo Internet OK để LAN-MCU tiếp tục gửi bình thường.
- Nếu WAN offline, ACK báo No Internet để LAN-MCU chuyển sang lưu đệm/SD (store-and-forward) thay vì tiếp tục đẩy dữ liệu lên, tránh làm đầy queue và lãng phí băng thông liên MCU.

Cơ chế downlink (Cloud → LAN): Khi nhận lệnh từ Cloud (ví dụ RPC/command) thông qua MQTT, WAN-MCU đưa lệnh vào downlink queue và thực hiện cơ chế push chủ động thay vì để LAN-MCU poll. Cụ thể, WAN-MCU sử dụng một đường GPIO toggle/handshake để thông báo cho LAN-MCU rằng đang có dữ liệu downlink hoặc cấu hình pending. Khi LAN-MCU phát hiện tín hiệu handshake, nó thực hiện phiên SPI để nhận gói lệnh từ WAN-MCU. WAN-MCU gửi gói command/DT tương ứng qua SPI; LAN-MCU kiểm tra hợp lệ và phản hồi ACK/NACK. Nếu nhận NACK hoặc timeout, WAN-MCU thực hiện retry giới hạn nhằm đảm bảo lệnh đến LAN-MCU một cách tin cậy. Sau khi LAN-MCU nhận thành công, lệnh được dispatch tới handler giao thức tương ứng để chuyển xuống sensor node.

Data Communication Handler (USB/UART) – giao tiếp với Configuration Tools trên PC WAN-MCU cung cấp kênh cấu hình tại chỗ thông qua USB/UART nhằm hỗ trợ kỹ thuật viên triển khai nhanh và thay đổi tham số vận hành mà không cần rebuild firmware.

- **Quét/đọc cấu hình hiện tại (scan config):**
 - PC gửi lệnh quét/đọc cấu hình tới gateway.
 - WAN-MCU truy xuất cấu hình đang lưu (NVS/runtime) và phản hồi dưới dạng các cặp key-value, ví dụ: loại kết nối Internet đang dùng (Wi-Fi/LTE,...), chế độ Wi-Fi (Personal/Enterprise) và tên đăng nhập, MQTT broker (host/port), topic, chu kỳ gửi dữ liệu...
 - Các trường nhạy cảm (password/secret) được ẩn hoặc rút gọn khi phản hồi để đảm bảo an toàn thông tin.
 - Có thể kèm thêm một số thông tin phục vụ kiểm tra nhanh như phiên bản firmware và trạng thái kết nối (Internet/MQTT).
- **Nhận lệnh cấu hình mới (CF...):**

- PC gửi lệnh cấu hình bắt đầu bằng prefix “CF...” kèm tham số tương ứng.
- WAN-MCU tiếp nhận, kiểm tra định dạng/độ dài cơ bản và đóng gói thành “config command” để đưa vào hàng đợi (queue) cho Config Handler xử lý, tránh xử lý nặng ngay trong luồng đọc USB/UART.
- Config Handler thực hiện validate giá trị, sau đó lưu vào NVS và áp dụng cấu hình vào các khối liên quan (Internet/MQTT...). Nếu cấu hình thuộc về LAN-MCU, WAN-MCU sẽ chuyển tiếp xuống LAN-MCU qua kênh liên MCU và nhận phản hồi OK/FAIL trước khi trả kết quả về PC.
- Sau khi hoàn tất, gateway phản hồi OK/FAIL (kèm lý do nếu lỗi) để kỹ thuật viên xác nhận cấu hình đã được ghi và áp dụng thành công.

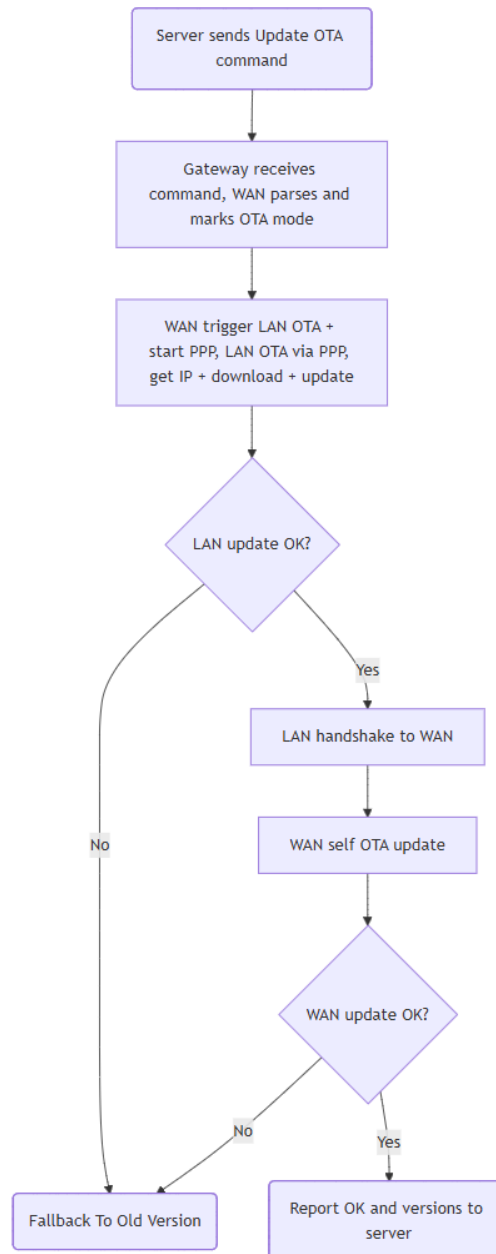


Hình 4.56: Lưu đồ giải thuật Data Communication Handler – giao tiếp cấu hình qua USB/UART

FOTA/Upgrade – cập nhật firmware cho Gateway (LAN cập nhật trước, WAN cập nhật sau) Trong prototype, cơ chế FOTA được kích hoạt từ server bằng một lệnh cấu hình đặc biệt. Khi gateway nhận được lệnh này, hệ thống chuyển sang “chế độ cập nhật” và thực hiện cập nhật theo thứ tự: LAN-MCU cập nhật trước (nhờ PPP Server do WAN-MCU cung cấp), sau đó WAN-MCU tự cập nhật firmware của chính nó. Cụ thể, khi server gửi lệnh, WAN-MCU sẽ parse lệnh và đánh dấu đây là OTA command. Thay vì cập nhật ngay, WAN-MCU forward lệnh OTA xuống LAN-MCU để LAN-MCU chuẩn bị cập nhật trước. Đồng thời, WAN-MCU khởi động PPP Server qua UART nhằm tạo một đường IP-link cho LAN-MCU truy cập mạng

và tải firmware. Nhờ đường PPP này, LAN-MCU có thể thực hiện quá trình OTA theo cơ chế HTTPS (tải image → kiểm tra → ghi firmware → reboot) một cách độc lập, trong khi WAN-MCU giữ vai trò “cấp đường truyền” và điều phối.

Sau khi LAN-MCU cập nhật xong, LAN-MCU sẽ gửi handshake thông báo cho WAN-MCU bước tiếp theo. Tại thời điểm này, WAN-MCU không phản hồi handshake theo kiểu trao đổi qua lại, mà chuyển thẳng sang bước tự cập nhật firmware của chính nó. WAN-MCU tiến hành OTA (tải → kiểm tra → ghi → reboot) và kết thúc quy trình FOTA toàn gateway. Sau cùng, gateway có thể gửi trạng thái cập nhật (OK/FAIL, version) lên server để xác nhận hoàn tất.



Hình 4.57: Lưu đồ giải thuật FOTA cho Dual-MCU Gateway

Kết nối đến các sensor node – gián tiếp qua LAN-MCU để phân tải và tăng ổn định hệ thống Trong kiến trúc Dual-MCU Master-Slave, WAN-MCU không trực tiếp chạy các

stack giao thức cảm biến (Zigbee/LoRa/CAN/RS-485...), mà tập trung vào Internet/Cloud và điều phối hệ thống. Toàn bộ phần “mặt dưới” gồm kết nối sensor node, giao tiếp đa giao thức, tiền xử lý và chuẩn hoá dữ liệu được đặt tại LAN-MCU. Thiết kế phân vai này giúp giảm tải đáng kể cho WAN-MCU (CPU/RAM/Flash), đồng thời hạn chế hiện tượng blocking giữa các tác vụ I/O nội bộ và tác vụ mạng Internet/Cloud – vốn là điểm yếu phổ biến của kiến trúc single-MCU.

Để chuẩn hoá trao đổi giữa hai MCU, dữ liệu và lệnh được đóng gói theo một định dạng nội bộ thống nhất, trong đó có trường `handler_type` nhằm định danh “nguồn/đích giao thức” (ví dụ: CAN / LOR / ZIG / RS4). Nhờ vậy, WAN-MCU có thể xử lý dữ liệu/lệnh theo hướng “định tuyến logic” mà không cần hiểu chi tiết từng stack giao thức.

Với cách tổ chức này, hệ thống vẫn “kết nối tới sensor node” đầy đủ theo nghĩa end-to-end, nhưng theo mô hình phân miền chức năng rõ ràng:

- LAN-MCU chuyên trách giao thức sensor và xử lý dữ liệu gần nguồn (near-sensor processing),
- WAN-MCU chuyên trách Internet/Cloud và điều phối (cloud gateway + control router).

Đây là hướng tiếp cận phù hợp với khảo sát kiến trúc gateway thực tế: tránh dư thừa như nền tảng Linux/SoC cho bài toán thu thập-đẩy dữ liệu, đồng thời khắc phục giới hạn tài nguyên và độ ổn định của mô hình single-MCU khi phải gánh đồng thời nhiều giao thức LAN/WAN và giao thức ứng dụng (MQTT/HTTP/CoAP).

Chương 5

Kịch bản kiểm thử - Quy trình Board Bring-up và Verification

5.1 Quy trình và Kết quả Khởi động Bo mạch

Quy trình khởi động bo mạch (Board Bring-up) được thực hiện nhằm xác minh tính toàn vẹn của phần cứng sau quá trình gia công và lắp ráp (PCBA). Mục tiêu của giai đoạn này là đảm bảo các khối nguồn hoạt động ổn định trong ngưỡng dung sai thiết kế và loại bỏ các lỗi vật lý trước khi tiến hành nạp firmware và kiểm thử chức năng. Quy trình thực nghiệm được chia thành 4 giai đoạn tuần tự.

5.1.1 Kiểm tra ngoại quan (Visual Inspection)

Trước khi cấp nguồn, công tác kiểm tra ngoại quan được thực hiện nhằm phát hiện các sai sót vật lý phát sinh trong quá trình gia công. Các hạng mục kiểm tra bao gồm:

- **Chất lượng mối hàn:** Rà soát các hiện tượng cầu thiếc (solder bridges), mối hàn lạnh (cold joints) hoặc thiếu thiếc, đặc biệt tại các khu vực linh kiện có mật độ chân cao.
- **Định hướng linh kiện:** Đối chiếu bo mạch thực tế với bản vẽ lắp ráp (Assembly Drawing) để xác nhận tính chính xác về chiều cực tính của các linh kiện phân cực (Diode, IC, Nguồn).
- **Độ sạch bề mặt:** Đảm bảo bề mặt PCB đã được vệ sinh công nghiệp, loại bỏ hoàn toàn flux thừa và các vụn kim loại có khả năng gây đoản mạch.

STT	Nhóm kiểm tra	Đối tượng kiểm tra cụ thể	Tiêu chí chấp nhận	Kết quả
A. KIỂM TRA TỔNG QUÁT				
1	Tổng quan (Physical & Solder)	Bo mạch PCB (Bề mặt, Flux, Vết xước)	Sạch sẽ, không lộ đồng, không cong vênh.	Passed
		Mối hàn tổng thể (Solder Joints)	Bóng, đều, không tổ ong, không hàn lạnh.	Passed
B. KIỂM TRA CHI TIẾT LINH KIỆN				
2	Nhóm Linh kiện (General Components)	Vi xử lý (SoC): U1, U2 (ESP32-S3-WROOM-1U)	Đúng Pinmap & Định hướng: - Chân số 1 (Pin 1) trùng khớp dấu chấm PCB. - Không bị xoay sai chiều. - Chân gài không bị dính thiếc (Short circuit).	Passed
		IO Expander: (TCA6424ARGJR - QFN32)		Passed
		USB Switch: (FSUSB42UMX - QFN10)		Passed
		RTC: (PCF85263ATL - DFN10)		Passed
		Màn hình OLED: (SH1107 Module)		Failed ¹
		MicroSD Card: (Phần chân tín hiệu SMD)		Passed
3	Nhóm Phân cực (Polarized Components)	Bán dẫn phân cực: (Diode TVS, Schottky)	Đúng Cực tính: - Cực Dương (+), Cathode (K) gia công đúng chiều.	Passed
		Nguồn: (Đế pin CR2032, DC-091)		Passed
4	Nhóm Cơ khí (Mechanical Parts)	USB Type-C Connector: (Cho U1 & U2)	Mức độ chống chịu cơ khí: - Chân vỏ (Shield) hàn chắc, chịu lực tốt.	Passed
		Expansion Headers: (WAN, LAN1, LAN2)		Passed
		DC Power Jack: (DC12V Input)		Passed
		Nút nhấn (Buttons): (Reset, Boot/User)		Passed
		Vỏ khe thẻ nhớ: (MicroSD Metal Shield)		Passed

Bảng 5.1: Kết quả kiểm tra ngoại quan.

5.1.2 Kiểm tra thông số nguội (Cold Check Analysis)

Phép đo kiểm tra nguội được thực hiện trong trạng thái bộ mạch cách ly hoàn toàn với nguồn điện. Phương pháp này sử dụng đồng hồ vạn năng để xác định trở kháng giữa các đường nguồn (Power Rails) và đất (GND). Mục đích là phát hiện sớm các lỗi ngắn mạch (Short Circuit) nghiêm trọng.

Kết quả đo đặc thực tế đã kiểm tra:

¹Không ảnh hưởng đến sản phẩm, chỉnh sửa ở giai đoạn sau

STT	Đường nguồn (Power Rail)	Tiêu chí kỹ thuật	Giá trị đo thực tế	Kết quả
1	+12V	Không ngắn mạch ($> 10\Omega$)	$1.175M\Omega$	Passed
2	+5V0_SYS	Không ngắn mạch ($> 10\Omega$)	$160.8k\Omega$	Passed
3	+5V0	Không ngắn mạch ($> 10\Omega$)	$12.0M\Omega$	Passed
4	+3V3	Không ngắn mạch ($> 10\Omega$)	$22.24k\Omega$	Passed
5	+1V8	Không ngắn mạch ($> 10\Omega$)	$29.49k\Omega$	Passed
6	+3V3_MSYS	Không ngắn mạch ($> 10\Omega$)	$30.0k\Omega$	Passed
7	+3V3_SSYS	Không ngắn mạch ($> 10\Omega$)	∞ (Open / High-Z)	Passed

Bảng 5.2: Kết quả kiểm tra trở kháng nguội các đường nguồn.

5.1.3 Kiểm tra cấp nguồn giới hạn (Smoke Test / Hot Check)

Để giảm thiểu rủi ro hư hỏng linh kiện do các lỗi tiềm ẩn không phát hiện được trong pha kiểm tra nguội, quy trình cấp nguồn lần đầu được thực hiện thông qua bộ nguồn đa năng với chế độ bảo vệ quá dòng.

Phương pháp thực hiện:

- Thiết lập điện áp đầu vào ở mức danh định ($V_{IN} = 12V$).
- Kích hoạt chế độ Giới hạn dòng điện (Current Limiting) ở ngưỡng an toàn $I_{limit} = 20mA$.
- Quan sát trạng thái chuyển đổi chế độ của bộ nguồn (CV - Constant Voltage hoặc CC - Constant Current) để đánh giá tình trạng bo mạch.

Kết quả kiểm thử: Hệ thống hoạt động ổn định ở chế độ Điện áp không đổi (CV). Dòng tiêu thụ tĩnh (Quiescent Current) đo được ở mức thấp ($\approx 20mA$), không xuất hiện hiện tượng sụt áp đầu vào hay quá nhiệt cục bộ trên các linh kiện công suất. Điều này xác nhận bo mạch không có lỗi quá dòng nghiêm trọng.

STT	Thiết lập kiểm thử	Tiêu chí chấp nhận	Kết quả đo thực tế	Kết quả
1	Bước 1: Cấp nguồn, giới hạn dòng $I_{limit} = 20mA$	Không sụt áp nguồn cấp (Giữ chế độ CV). Dòng tiêu thụ $< 20mA$.	Ổn định. $I_{load} = 4mA$.	Passed
2	Bước 2: Tăng giới hạn dòng $I_{limit} = 60mA$	Không sụt áp nguồn cấp.	Ổn định. $I_{load} = 4mA$.	Passed
3	Bước 3: Tăng giới hạn dòng $I_{limit} = 100mA$	Không sụt áp nguồn cấp.	Ổn định. $I_{load} = 4mA$.	Passed
4	Bước 4: Tăng giới hạn dòng $I_{limit} = 200mA$	Không sụt áp nguồn cấp.	Ổn định. $I_{load} = 4mA$.	Passed
5	Bước 5: Cấp nguồn bình thường	Hệ thống hoạt động ổn định, không có linh kiện phát nhiệt bất thường.	Ổn định. $I_{load} = 4mA$ (Idle).	Passed

STT	Thiết lập kiểm thử	Tiêu chí chấp nhận	Kết quả đo thực tế	Kết quả
-----	--------------------	--------------------	--------------------	---------

Bảng 5.3: Kết quả kiểm tra nóng và dòng tiêu thụ tĩnh.

5.1.4 Xác thực thông số điện áp

Sau khi xác nhận tính an toàn của hệ thống, giới hạn dòng điện được tăng đến mức vận hành tiêu chuẩn. Các mức điện áp tại các điểm kiểm tra (Test Points) được đo đạc để xác minh độ chính xác của khối nguồn Buck Converter và LDO so với tính toán thiết kế.

STT	Điểm đo	Giá trị kỳ vọng	Giá trị đo thực tế)	Kết quả
1	+12V_IN <i>Test-point TP11/13</i>	12.0V $\pm$ 10%	12.02V	Passed
2	+12V <i>Test-point TP12/14</i>	12.0V $\pm$ 10%	12.02V	Passed
3	+5V0_SYS <i>Test-point TP15/16</i>	5.0V $\pm$ 5%	5.01V	Passed
4	+5V0 <i>Test-point TP29/30</i>	5.0V $\pm$ 5%	5.01V	Passed
5	+3V3 <i>Test-point TP31/32</i>	3.3V $\pm$ 3%	3.325V	Passed
6	+1V8 <i>Test-point TP33/34</i>	1.8V $\pm$ 3%	1.796V	Passed
7	+3V3_MSYS <i>Test-point TP5/6</i>	3.3V $\pm$ 3%	3.310V	Passed
8	+3V3_SSYS <i>Test-point TP7/8</i>	3.3V $\pm$ 3%	3.295V	Passed
9	+3V0_BAT <i>Test-point TP17/18</i>	3.0V $\pm$ 3%	3.085V	Passed
10	+3V3_IOX1_VCCI <i>Test-point TP19/20</i>	3.3V $\pm$ 3%	3.28V	Passed
11	+3V3_IOX2_VCCI <i>Test-point TP25/26</i>	3.3V $\pm$ 3%	3.28V	Passed

Bảng 5.4: Kết quả đo kiểm điện áp.

5.2 Quy trình Kiểm tra sản phẩm

Sau quy trình khởi động (Bring-up) thành công, toàn bộ hệ thống được đưa vào quy trình kiểm tra chi tiết. Mục tiêu của giai đoạn này là định lượng các thông số kỹ thuật, đảm bảo các mức logic điều khiển và chất lượng nguồn điện đáp ứng yêu cầu thiết kế trước khi tích hợp firmware ứng dụng.

5.2.1 Kiểm tra trạng thái Logic và Điều khiển (Logic State Verification)

Các tín hiệu điều khiển tới hạn (Critical Control Signals) bao gồm Reset, Boot Mode và Enable được đo đặc tại thời điểm khởi động và trạng thái ổn định. Việc này nhằm xác minh cấu hình điện trở kéo (Pull-up/Pull-down) đã được thiết kế đúng, đảm bảo MCU vào đúng chế độ hoạt động (Boot mode vs Download mode) và không xảy ra hiện tượng thả nổi tín hiệu (Floating).

STT	Tín hiệu (Signal)	Điều kiện kiểm tra (Test Condition)	Mức Logic đo được (Measured Logic)	Kết quả
1	ESP_EN (Enable) (U1 & U2)	Mặc định (RC Delay khi cấp nguồn).	Logic High (3.3V). (Stable after 10ms).	Passed
2	IO0 (Boot Pin) (U1 & U2)	Trạng thái nghỉ (Idle). Không nhấn nút BOOT.	Logic High (3.3V). (Internal Pull-up OK).	Passed
3	NRST (System Reset)	Nhấn nút Reset cứng.	Logic Low (0V). (Active Low).	Passed
4	IO_INT (IO Expander Interrupt)	Mặc định (Chưa có sự kiện).	Logic High (3.3V). (External Pull-up 10kΩ).	Passed
5	USB_VBUS_DET (Phát hiện cáp USB)	Cắm cáp USB Type-C.	Logic High (5.0V → 3.3V qua phân áp).	Passed

Bảng 5.5: Kết quả kiểm tra các mức Logic điều khiển.

5.2.2 Kiểm tra Chất lượng Nguồn (Power Integrity Analysis)

Chất lượng nguồn được đánh giá thông qua đo đặc từ máy hiện sóng (Oscilloscope) để đo độ gợn (Voltage Ripple) và độ sụt áp (Voltage Drop) dưới tải. Các phép đo được thực hiện khi hệ thống đang chạy firmware kiểm thử (Test Firmware) với tải giả định để mô phỏng điều kiện hoạt động thực tế.

STT	Rail	Điểm đo tại tải	Giá trị đo thực tế			Kết quả
			Sleep Mode	Normal Mode	Full Load	
A. DÒNG ĐIỆN ĐẦU VÀO (INPUT CURRENT) - Đơn vị: mA						
*Tiêu chí: Dòng tổng < 2.35A (Công suất thiết kế)						
1	+12V	Total Input (Jack DC)	45	276	893	Checked
B. ĐỘ GỌN ĐIỆN ÁP (VOLTAGE RIPPLE) - Đơn vị: ±mV _{pp}						
*Tiêu chí: Voltage ripple < 5%						
2	+5V0_SYS	Test-point (TP15/16)	12	18	28	Passed
3	+5V0	Test-point (TP29/30)	15	19	32	Passed
4	+3V3	Test-point (TP31/32)	5	7	12	Passed
5	+1V8	Test-point (TP33/34)	4	5	7	Passed

STT	Rail	Điểm đo tại tải	Giá trị đo thực tế			Kết quả
			Sleep Mode	Normal Mode	Full Load	
6	+3V3_MSYS	Test-point (TP5/6)	5	7	8	Passed
C. ĐỘ SỤT ÁP TẠI TẢI (VOLTAGE DROP) - Đơn vị: V						
<i>*Tiêu chí: Voltage drop = voltage at source - voltage at load < 5%</i>						
7	+5V0_SYS	ESP32-S3 Master	5.01	5.01	4.96	Passed
8	+5V0	ESP32-S3 Slave	5.00	5.00	4.98	Passed
9		WAN Adapter Module	5.01	5.01	4.98	Passed
10		LAN1 Adapter Module	5.01	5.01	5.00	Passed
11		LAN2 Adapter Module	5.01	5.01	5.00	Passed
12	+3V3	Input PWR_1V8 (LDO)	3.30	3.30	3.30	Passed
13		IO Expander 2	3.30	3.30	3.29	Passed
14		WAN Adapter Module	3.30	3.30	3.28	Passed
15		LAN1 Adapter Module	3.30	3.30	3.29	Passed
16		LAN2 Adapter Module	3.30	3.30	3.29	Passed
17		Micro SD Card	3.30	3.28	3.27	Passed
18		LCD (OLED)	3.30	3.28	3.26	Passed
19	+1V8	WAN Adapter Module	1.80	1.78	1.76	Passed
20		LAN1 Adapter Module	1.80	1.80	1.79	Passed
21		LAN2 Adapter Module	1.80	1.80	1.80	Passed
22	+3V3_MSYS	RTC (PCF85263)	3.30	3.30	3.29	Passed
23		IO Expander 1	3.30	3.29	3.29	Passed
24		USB Switch (FSUSB42)	3.30	3.30	3.30	Passed
25		PWR_BTN (Pull-up)	3.30	3.30	3.30	Passed

Bảng 5.6: Kết quả kiểm tra chất lượng nguồn điện.

5.2.3 Kiểm tra Chức năng Phần cứng (Functional Verification)

Từng khối chức năng ngoại vi được kiểm tra độc lập thông qua các firmware driver cơ bản. Mục đích là xác nhận sự toàn vẹn của các đường dẫn tín hiệu và khả năng giao tiếp vật lý giữa MCU và các linh kiện ngoại vi.

STT	Khối chức năng	Phương pháp kiểm tra	Dấu hiệu nhận biết	Kết quả
A. GIAO TIẾP VI ĐIỀU KHIỂN				
1	Giao tiếp dữ liệu tốc độ cao (U1 Master ↔ U2 Slave)	Giao thức Quad SPI (SPI2): Kiểm tra đường bus 6 dây: CS, CLK, MOSI, MISO, WP, HD (IO[9:14] kết nối tới IO[8:14]). Thực hiện gửi gói dữ liệu lớn (Ping-pong buffer).	Tốc độ truyền tải đạt mức thiết kế (> 10Mbps). Dữ liệu nhận được tại U1 khớp hoàn toàn với dữ liệu gửi từ U2 (Data Integrity Check).	Passed
2	Nạp Firmware cho Slave (U1 Programming U2)	Giao thức UART + GPIO Control: U1 điều khiển chân U2_RESET và U2_BOOT để đưa U2 vào chế độ Download Mode, sau đó gửi dữ liệu qua UART0.	U1 báo cáo nạp thành công (Successfully Flashed). U2 tự động khởi động lại và chạy firmware mới.	Passed
3	Điều khiển & Đồng bộ (Handshake Signals)	GPIO Interrupt: Kiểm tra tín hiệu ngắt U2_INT từ Slave báo về Master và tín hiệu Enable từ Master xuống Slave.	Master nhận được sự kiện ngắt ngay lập tức khi Slave kích hoạt chân I046.	Passed
B. NGOẠI VI CỦA MASTER				
4	Màn hình hiển thị (OLED Module)	Giao thức I2C0 (Master): Quét địa chỉ 0x3C. Gửi lệnh hiển thị Pattern bàn cờ (Checkerboard).	Màn hình sáng đều, không có điểm chết, hiển thị đúng nội dung test.	Passed
5	Đồng hồ thời gian thực (RTC - PCF85263)	Giao thức I2C0 (Master): Quét địa chỉ 0x51. Gửi lệnh ghi thời gian hiện tại và đọc lại sau 1 phút.	Thời gian đọc về tăng đúng 60 giây so với lúc ghi.	Passed
6	Mở rộng IO (Master) (IO Expander 1 - TCA6424)	Giao thức I2C0 (Master): Quét địa chỉ 0x22. Điều khiển bật/tắt các chân Output P0/P1/P2.	Các tín hiệu điều khiển nguồn (EN_1V8C, EN_3V3C) thay đổi mức logic đúng theo lệnh.	Passed
7	USB Switch (FSUSB42UMX)	Logic Control (GPIO): Master điều khiển chân U1_USBSEL.	Khi High: USB Type-C thông với U1. Khi Low: USB Type-C chuyển hướng.	Passed

STT	Khối chức năng	Phương pháp kiểm tra	Dấu hiệu nhận biết	Kết quả
8	Cổng kết nối WAN (Cho Module LTE/Ethernet)	Multi-protocol Check: - Loopback trên UART1 (IO15/16). - Quét thiết bị trên SPI3 (IO4-7).	- UART nhận lại đúng chuỗi đã gửi. - SPI phát hiện thiết bị ngoại vi giả lập.	Passed
C. NGOẠI VI CỦA SLAVE				
9	Mở rộng IO (Slave) (IO Expander 2 - TCA6424)	Giao thức I2C0 (Slave): Quét địa chỉ 0x22 trên bus I2C của U2. Kiểm tra ngắt IOX2_INT.	U2 giao tiếp thành công, đọc/ghi được trạng thái các chân mở rộng (LAN_IO).	Passed
10	Thẻ nhớ MicroSD (Storage)	Giao thức SDIO 4-bit: U2 thực hiện Mount File System (FAT32) và ghi file log mẫu 1MB.	Tốc độ ghi đạt chuẩn Class 10. File đọc lại không bị lỗi (Check MD5).	Passed
11	Cổng mở rộng LAN1 (Cho Zig-bee/Thread)	Multi-protocol Check: - UART1 (IO17/18). - SPI3 (CS0).	Giao tiếp ổn định với Module Test LAN1 (Loopback OK).	Passed
12	Cổng mở rộng LAN2 (Cho RS485/CAN)	Multi-protocol Check: - UART2 (IO15/16). - SPI3 (CS1).	Giao tiếp ổn định với Module Test LAN2 (Loopback OK).	Passed
D. GIAO DIỆN HỆ THỐNG (SYSTEM INTERFACE)				
13	Master USB-C Port (Native USB)	USB JTAG/Serial: Cắm cáp vào PC.	PC nhận diện cổng COM (USB Serial/JTAG Controller). Nạp code trực tiếp từ PC OK.	Passed
14	Nút nhấn hệ thống (Power Button)	Physical Interaction: Nhấn nút Power Button.	ESP32-S3 Master nhận được tín hiệu từ nút nhấn và bật/tắt nguồn.	Passed

Bảng 5.7: Kết quả kiểm tra chức năng.

5.2.4 Kiểm tra Firmware và Software

Sau khi hoàn tất kiểm tra phần cứng, hệ thống được đưa vào giai đoạn Firmware & Software Verification nhằm xác nhận bo mạch có thể vận hành ổn định với firmware thực tế, đồng thời kiểm chứng các luồng chức năng cốt lõi của kiến trúc Dual MCU Master-Slave (LAN MCU Master, WAN MCU Slave trên kênh SPI).

5.2.4.1 Mục tiêu

- Xác nhận khả năng nạp và khởi chạy firmware cho cả hai MCU, đảm bảo vào đúng boot mode và ổn định dưới FreeRTOS.
- Kiểm chứng các giao tiếp bắt buộc: SPI, UART nạp firmware cho MCU còn lại, và các tín hiệu điều khiển/handshake GPIO.
- Kiểm chứng các chức năng mức hệ thống: kết nối Internet (Wi-Fi/LTE/PPP), MQTT uplink/downlink, cấu hình tại chỗ (USB/UART) và FOTA.
- Định lượng nhanh các tiêu chí “đủ dùng” cho prototype: chạy lâu không treo, truyền dữ liệu đúng/đủ, phục hồi khi mất mạng.

5.2.4.2 Quy trình nạp và khởi động firmware

1. Nạp WAN MCU qua cổng USB C (USB Serial/JTAG) bằng công cụ nạp chuẩn (ESP IDF/esptool). Sau khi nạp, kiểm tra log UART/USB và trạng thái task khởi tạo.
2. Nạp LAN MCU gián tiếp từ WAN MCU qua kênh UART + GPIO control (Reset/Boot) để đưa LAN MCU vào Download Mode và flash. LAN MCU tự reboot sau khi nạp thành công.
3. Khởi động đồng thời hệ thống và kiểm tra:
 - FreeRTOS scheduler chạy ổn định (không reset bất thường),
 - watchdog (nếu bật) không kích hoạt sai,
 - các task chính lên đủ và không bị deadlock.

5.2.4.3 Nội dung kiểm thử firmware (theo nhóm chức năng)

A. Kiểm thử nền tảng (Boot/RTOS/Log)

- Kiểm tra boot ổn định, log xuất đều, không reset ngẫu nhiên.
- Kiểm tra hoạt động song song của các task chính (LAN thu thập/chuẩn hoá; WAN Internet/MQTT; liên MCU).

B. Kiểm thử giao tiếp liên MCU (Dual MCU Integration)

- SPI data path: truyền gói dữ liệu uplink từ LAN→WAN, WAN phản hồi ACK/NACK theo kiểm tra CRC/sequence.
- GPIO handshake (downlink push): khi WAN có lệnh/command cần chuyển xuống LAN, WAN toggle GPIO handshake để LAN thực hiện phiên SPI nhận dữ liệu.
- Tiêu chí: dữ liệu không sai lệch (integrity), retry hoạt động khi lỗi CRC, không nghẽn SPI khi mạng dao động.

C. Kiểm thử Internet Communication (WAN MCU)

- **Wi-Fi:** hỗ trợ Wi-Fi Personal và Wi-Fi Enterprise; sau khi vào mạng thực hiện SNTP để đồng bộ thời gian và cập nhật RTC.
- **LTE:** kết nối modem LTE qua USB, theo dõi link và tự reconnect.

- **PPP Server:** bật PPP Server qua UART để tạo IP link hỗ trợ LAN MCU thực hiện OTA.

D. Kiểm thử Cloud/MQTT (WAN MCU)

- Duy trì kết nối MQTT; kiểm tra cơ chế tự reconnect.
- Uplink telemetry: nhận dữ liệu từ LAN (SPI) → đưa vào publish queue → đóng gói payload → publish lên broker.
- Downlink command: subscribe topic downlink → decode payload → phân loại đích (handler_type) → push xuống LAN bằng GPIO handshake + SPI.

E. Kiểm thử cấu hình tại chỗ (USB/UART)

- **Scan/Read config:** PC yêu cầu đọc cấu hình → WAN trả về key-value (mask thông tin nhạy cảm).
- **Config write:** PC gửi lệnh cấu hình (prefix CF...) → WAN enqueue → validate → lưu NVS + apply runtime; nếu cấu hình thuộc LAN thì forward xuống LAN.
- **Tiêu chí:** Cấu hình tồn tại sau reboot, và không làm gián đoạn các luồng SPI/MQTT.

F. Kiểm thử FOTA toàn gateway.

- Server gửi lệnh CFML:CFFW → WAN parse và xác định OTA mode.
- WAN trigger LAN OTA + start PPP để LAN có IP link tải firmware và cập nhật.
- Sau khi LAN cập nhật xong, LAN gửi handshake; WAN chuyển sang tự cập nhật firmware của chính nó (WAN không phản hồi handshake theo kiểu đối thoại mà bước vào OTA).
- Tiêu chí: hoàn tất cập nhật cả hai MCU và hệ thống boot lại bình thường. (Lưu ý: cơ chế rollback/fallback nếu cần được coi là hướng mở rộng, tùy cấu hình bootloader/confirm app.)

G. Kiểm tra cấu hình (Configuration Tools → Gateway) Kiểm thử cấu hình nhằm xác nhận công cụ PC có thể đọc/ghi tham số qua USB/UART, firmware validate – lưu NVS – áp dụng runtime, đồng thời nếu tham số thuộc LAN thì WAN sẽ forward xuống LAN qua SPI.

STT	Nhóm	Điều kiện	Thao tác	Tiêu chí chấp nhận	Kết quả
1	Scan device	Cắm USB/UART, gateway ở NORMAL	PC thực hiện quét thiết bị	PC nhận diện đúng cổng/thiết bị	Passed
2	Read config	Có cấu hình mặc định	PC gửi lệnh đọc cấu hình	WAN trả key-value đầy đủ (mask secret)	Passed
3	Read after write	Đã ghi cấu hình mới	PC đọc lại cấu hình	Giá trị phản ánh đúng cấu hình mới	Passed
4	Read after reboot	Reset gateway	PC đọc cấu hình	Cấu hình giữ nguyên sau reboot	Passed

Bảng 5.8: Kiểm thử Scan/Read cấu hình

STT	Nhóm tham số	Ví dụ tham số	Thao tác	Tiêu chí chấp nhận	Kết quả
1	Chọn loại Internet	Wi-Fi / LTE / PPP	Ghi internet_type	Chuyển đúng chế độ, log trạng thái đúng	Passed
2	Chu kỳ gửi	period_ms	Ghi period_ms	Áp dụng runtime, không nghẽn SPI/MQTT	Passed
3	Chính sách retry	backoff/retry	Ghi retry policy	Mất mạng → reconnect theo chính sách	Passed
4	Giá trị ngoài dải	period_ms quá lớn/0	Ghi tham số lỗi	Trả FAIL, không áp dụng	Passed

Bảng 5.9: Kiểm thử ghi cấu hình Internet (internet_type) và tham số chung

STT	Chế độ	Tham số chính	Điều kiện	Tiêu chí chấp nhận	Kết quả
1	Personal	SSID + PSK	AP hoạt động	Kết nối thành công, có IP	Passed
2	Personal	Sai mật khẩu	AP hoạt động	Không vào mạng, tự retry, không treo	Passed
3	Enterprise	SSID + user/pass (EAP)	AP Enterprise	Auth OK, có IP	Passed
4	Enterprise	Sai credential	AP Enterprise	Auth FAIL, retry có kiểm soát	Passed
5	Đồng bộ thời gian	SNTP + RTC	Internet OK	RTC được cập nhật sau khi vào mạng	Passed

Bảng 5.10: Kiểm thử Wi-Fi (Personal/Enterprise)

STT	Hạng mục	Điều kiện	Thao tác	Tiêu chí chấp nhận	Kết quả
1	LTE connect	Có modem + SIM	Chọn LTE, cấu hình APN (nếu có)	Có IP, online ổn định	Passed
2	LTE reconnect	Ngắt/khôi phục sóng	Quan sát tự reconnect	Online trở lại, không cần reboot	Passed
3	PPP Server	Có UART link LAN	Bật PPP Server	LAN nhận IP link qua PPP	Passed
4	PPP stability	Update FOTA	Truyền dữ liệu qua PPP	Link không drop bất thường	Passed

STT	Hạng mục	Điều kiện	Thao tác	Tiêu chí chấp nhận	Kết quả
-----	----------	-----------	----------	--------------------	---------

Bảng 5.11: Kiểm thử LTE và PPP

STT	Hạng mục	Thao tác	Tiêu chí chấp nhận	Kết quả
1	Broker config	Ghi host/port/user/pass	MQTT connect OK	Passed
2	Topic config	Ghi topic telemetry/downlink	Publish/subscribe đúng topic	Passed

Bảng 5.12: Kiểm thử MQTT (broker/topic/telemetry/downlink)

Kiểm tra cấu hình ở LAN:

STT	Tham số	Thao tác	Tiêu chí chấp nhận	Kết quả
1	Slot/num_slots	Thay đổi Slot/num_slots trên App	Node gửi đúng slot, không đụng nhau	Passed
2	ID gateway/node	Thay đổi ID gateway/node trên App	Lưu vào NVS của LAN MCU	Passed
3	Tham số của module LoRa	Nhập tham số trên App và ghi lên gateway	Config lưu được trong NVS và trên Module	Passed

Bảng 5.13: Kiểm thử cấu hình LoRa TDMA

STT	Tham số	Thao tác	Tiêu chí chấp nhận	Kết quả
1	Bitrate	Thay đổi bitrate trên App	Receive/Transmit frame OK	Passed
2	Filter/ID	Thay đổi filter/ID trên App	Chỉ nhận ID hợp lệ	Passed
3	Bus-off	Tạo lỗi bus-off (thao tác theo chức năng test trên App)	Tự phục hồi theo policy	Passed

Bảng 5.14: Kiểm thử cấu hình CAN

STT	Tham số	Thao tác	Tiêu chí chấp nhận	Kết quả
1	Baud/parity	Thay đổi baud/parity trên App	Loopback/thiết bị phản hồi đúng	Passed
2	Timeout	Thay đổi timeout trên App	Timeout đúng, không treo task	Passed

Bảng 5.15: Kiểm thử RS 485/Modbus (raw frame)

STT	Hạng mục	Phương pháp	Tiêu chí	Kết quả
1	Boot/FreeRTOS	Khởi động lặp, chạy lâu	Không reset/treo bất thường	Passed
2	Nạp LAN từ WAN	UART + GPIO boot/reset	Flash OK, LAN re-boot chạy image mới	Passed
3	SPI uplink	DT + CRC/seq	ACK/NACK đúng, retry hoạt động	Passed
4	GPIO handshake downlink	WAN push command	LAN nhận đúng, phản hồi OK/FAIL	Passed
5	Internet (Wi-Fi/LTE/PPP)	Connect + reconnect	Online ổn định, phục hồi sau mất mạng	Passed
6	MQTT up-link/downlink	publish/subscribe	Telemetry OK; downlink route đúng	Passed
7	USB/UART config	scan + write + reboot	Lưu NVS, áp dụng runtime	Passed
8	FOTA toàn gateway	CFML:CFFW + PPP	LAN update trước; WAN update sau; boot OK	Passed

Bảng 5.16: Kiểm thử firmware (tổng hợp)

Giai đoạn kiểm thử xác nhận firmware vận hành ổn định trên phần cứng đã bring up, đồng thời kiểm chứng các luồng quan trọng của hệ Dual MCU (SPI + handshake), Internet/Cloud (Wi Fi/LTE/MQTT), cấu hình tại chỗ và quy trình FOTA toàn gateway.

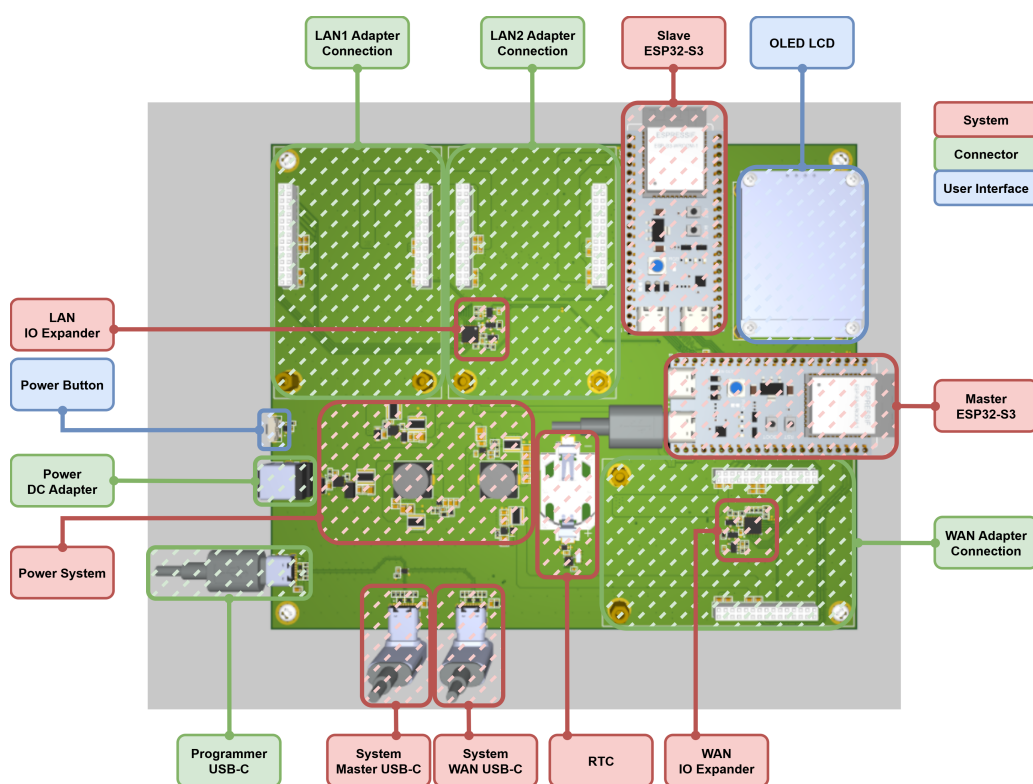
Chương 6

Triển khai và Vận hành Hệ thống

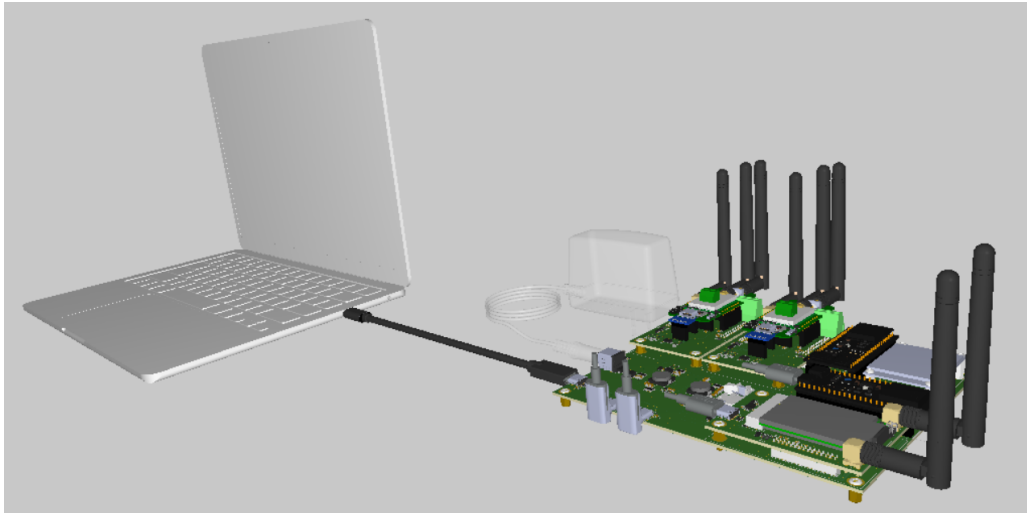
Chương này trình bày quá trình hiện thực hóa thiết kế từ các thành phần rời rạc thành một sản phẩm hoàn chỉnh, đồng thời mô tả cơ chế vận hành thông minh và quy trình tương tác của người dùng với thiết bị.

6.1 Triển khai Lắp ráp Phần cứng

Sản phẩm được xây dựng dựa trên triết lý thiết kế "Module hóa", cho phép người dùng tùy biến chức năng một cách linh hoạt thông qua việc lắp ghép các khối phần cứng.

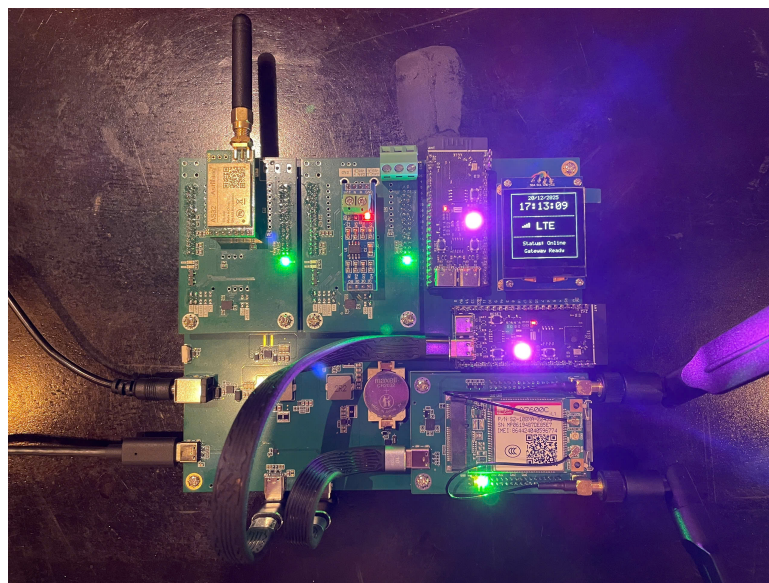


Hình 6.1: Assembly Diagram.



Hình 6.2: Mô hình 3D kết nối thiết bị.

1. **Bo mạch chủ (Baseboard):** Bo mạch chủ tích hợp hai vi điều khiển ESP32-S3 (Master và Slave) cùng hệ thống quản lý nguồn tập trung. Hệ thống nguồn cung cấp đồng thời các ray 12V, 5V, 3.3V và 1.8V để cấp nguồn cho các module.
2. **Cơ chế kết nối Module Adapter:** Việc lắp ráp các module chức năng được thông qua các chân cắm **WAN, LAN1, LAN2 Adapter Connection** đã được định nghĩa sẵn trên Baseboard:
 - **Khe cắm WAN:** Thường lắp ráp các module kết nối mạng không dây như Wi-Fi/LTE/4G hoặc Ethernet.
 - **Khe cắm LAN1/LAN2:** Dành cho các module giao tiếp công nghiệp hoặc mạng nội bộ như RS485, CAN Bus, LoRa hoặc Zigbee.
3. **Programmer USB-C:** Cổng USB-C sử dụng cho việc cấu hình IoT Gateway.



Hình 6.3: Hình ảnh sản phẩm thực tế.

6.2 Khởi động và Kích hoạt Phần mềm

Sau khi hoàn tất quá trình lắp ráp phần cứng, hệ thống gateway (với kiến trúc Dual-MCU gồm ESP32-S3 Master/WAN-MCU và Slave/LAN-MCU) được đưa vào vận hành theo chu trình: Khởi động – Nạp cấu hình – Kết nối – Vận hành dịch vụ.

6.2.1 Khởi động hệ thống (Boot & RTOS Init)

Khi được cấp nguồn, WAN-MCU đóng vai trò điều phối chính (Application Master). Vì điều khiển này thực hiện khởi tạo các thành phần phần cứng nền tảng (Drivers, BSP) và khởi chạy các tác vụ (tasks) trên hệ điều hành FreeRTOS theo mô hình handler chạy song song. Các khối chức năng chính được kích hoạt bao gồm:

- **Main Control:** Điều phối trạng thái hoạt động của toàn hệ thống.
- **Internet Communication Handler:** Quản lý các kết nối mạng (Ethernet, Wi-Fi, LTE/NB-IoT).
- **Server Communication Handler:** Quản lý giao thức tầng ứng dụng (MQTT, HTTP, CoAP).
- **MCU LAN Communication Handler:** Thiết lập kênh giao tiếp liên vi điều khiển (Inter-MCU) với LAN-MCU.
- **FOTA/Bridge Bootloader:** Sẵn sàng cho quy trình cập nhật firmware từ xa.

6.2.2 Nạp và áp dụng cấu hình vận hành (Load/Apply Configuration)

Hệ thống truy xuất toàn bộ tham số cấu hình được lưu trữ trong bộ nhớ. Gateway sẽ đọc và áp dụng các cấu hình này cho các khối Internet/Cloud và đặc biệt là tải các driver tương ứng cho các module phần cứng đang được lắp đặt. Cơ chế này đảm bảo tính **modularity** và **configurability**, cho phép hệ thống chỉ khởi chạy các tài nguyên cần thiết theo nhu cầu triển khai thực tế.

6.2.3 Thiết lập kết nối WAN và đồng bộ thời gian

Dựa trên cấu hình đã tải, gateway tiến hành thiết lập kết nối Internet ưu tiên (Wi-Fi hoặc LTE).

- **Đối với kết nối Wi-Fi,** hệ thống hỗ trợ cả hai chế độ bảo mật là Personal và Enterprise.
- Ngay sau khi kết nối mạng thành công, gateway thực hiện giao thức SNTP để đồng bộ thời gian thực và cập nhật cho RTC, đảm bảo tính chính xác của nhãn thời gian (timestamp) cho dữ liệu log và telemetry.

6.2.4 Kết nối Cloud và vòng lặp vận hành dữ liệu

Khi kết nối WAN ổn định, gateway khởi tạo kết nối đến Cloud Server (thông qua giao thức MQTT). Lúc này, hệ thống đi vào vòng lặp vận hành chính:

- **Luồng Uplink:** Dữ liệu thu thập từ LAN-MCU được truyền sang WAN-MCU, đóng gói (packetizing) và đẩy vào hàng đợi (queue) để gửi lên broker/server theo cấu hình.
- **Luồng Downlink:** Gateway duy trì kết nối để nhận lệnh điều khiển từ server và phân luồng xuống LAN-MCU hoặc xử lý tại chỗ.
- **Cơ chế bảo vệ:** Hệ thống duy trì cơ chế tự động kết nối lại (auto-reconnect) khi mất mạng để đảm bảo tính sẵn sàng cao.

6.2.5 Kích hoạt cấu hình tại chỗ (Local Configuration)

Trong trường hợp cần cấu hình thủ công bởi kỹ thuật viên, gateway hỗ trợ chế độ cấu hình qua cổng Programmer USB-C (giao tiếp USB/UART).

- **Scan/Read Config:** PC gửi yêu cầu đọc, gateway trả về các cặp key-value hiện hành (có cơ chế che thông tin nhạy cảm).
- **Write Config:** PC gửi lệnh cấu hình (prefix “CF...”), gateway đưa vào hàng đợi xử lý → kiểm tra tính hợp lệ (validate) → lưu vào NVS → áp dụng ngay (runtime apply). Nếu cấu hình thuộc về miền LAN, WAN-MCU sẽ chuyển tiếp lệnh xuống LAN-MCU để xử lý đồng bộ.

6.2.6 Cập nhật Firmware (FOTA)

Gateway hỗ trợ quy trình cập nhật firmware an toàn (Safety FOTA) nhằm giảm thiểu rủi ro hỏng thiết bị (brick). Quy trình bao gồm kiểm tra tính toàn vẹn (checksum), thực hiện cập nhật tuần tự cho từng MCU (Master và Slave) và cơ chế không ghi đè lên firmware đang chạy cho đến khi gói cập nhật được xác thực thành công.

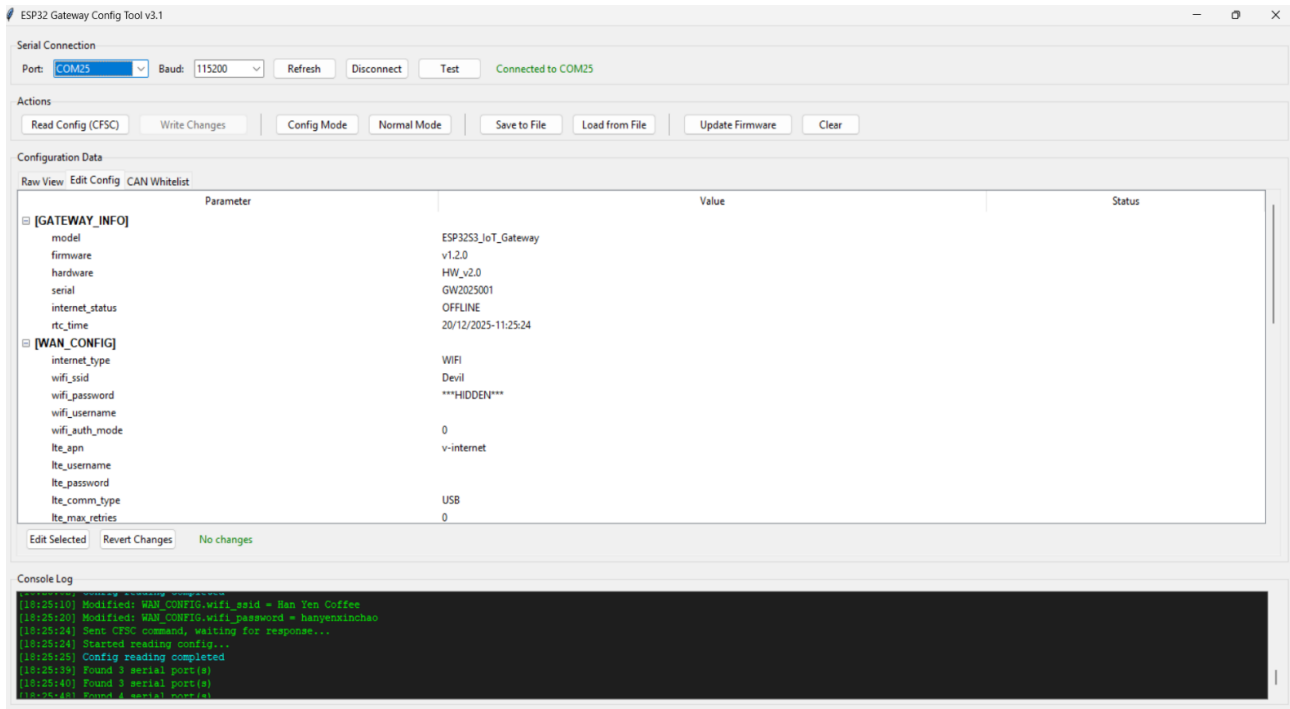
6.3 Kịch bản Tương tác Người dùng

Tính tiện dụng của sản phẩm được thể hiện qua khả năng tương tác linh hoạt với các module/giao thức và giao diện quản trị trực quan. Người dùng (kỹ thuật viên hoặc quản trị viên) tương tác với hệ thống thông qua các kịch bản chính sau:

6.3.1 Cấu hình ban đầu bằng Ứng dụng PC

Đây là kịch bản dành cho kỹ thuật viên triển khai tại hiện trường, sử dụng App cấu hình trên PC kết nối với gateway qua cổng USB-C. Quy trình thao tác tiêu chuẩn gồm:

- **Quét và nhận diện:** App tự động phát hiện gateway khi kết nối cáp và xác nhận thiết bị sẵn sàng.
- **Đọc cấu hình:** App tải toàn bộ thông số hiện tại từ gateway để hiển thị lên giao diện.
- **Chỉnh sửa tham số:** Người dùng thao tác trên giao diện dạng biểu mẫu (form) để thiết lập.
- **Ghi cấu hình:** Sau khi nhấn "Apply", App gửi lệnh xuống gateway. Firmware thực hiện validate, lưu NVS và áp dụng cấu hình mới tức thì. Nếu tham số thuộc về LAN-MCU, WAN-MCU sẽ đóng vai trò cầu nối để chuyển tiếp cấu hình.



Hình 6.4: App config trên PC

6.3.2 Giám sát vận hành qua Giao diện Web (Dashboard/Status)

Người quản trị hệ thống giám sát hoạt động của gateway từ xa thông qua Dashboard trên nền tảng Cloud/Web.

- **Điều kiện bình thường:** Dữ liệu từ các sensor node (thông qua LAN-MCU) được WAN-MCU đưa vào hàng đợi (publish queue) và gửi lên broker/server qua MQTT. Pipeline xử lý này tách biệt luồng "nhận dữ liệu" và "gửi Cloud" giúp tránh nghẽn cổ chai khi mạng chập chờn.
- **Xử lý sự cố kết nối:** Khi mất kết nối WAN hoặc không thể tới Server, hệ thống tự động kích hoạt cơ chế đệm (queue) và thử gửi lại (retry). Nếu mất kết nối kéo dài, khối Data Storage Handler sẽ chuyển sang lưu trữ dữ liệu tạm thời vào thẻ nhớ microSD, đảm bảo không mất mát dữ liệu (zero data loss) và tự động đồng bộ lại khi kết nối được khôi phục.

6.3.3 Bảo trì và Cập nhật từ xa (OTA)

Trong giai đoạn vận hành và bảo trì, người dùng có thể nâng cấp tính năng hoặc vá lỗi cho cả hai vi điều khiển (Master và Slave) thông qua tính năng OTA trực tiếp từ giao diện Web quản trị.

- Quy trình này giúp tiết kiệm đáng kể thời gian và chi phí đi lại so với việc nạp code thủ công tại hiện trường.
- Cơ chế cập nhật được thiết kế ưu tiên tính an toàn: kiểm tra toàn vẹn dữ liệu (checksum/signature), cập nhật tuần tự từng MCU và có cơ chế khôi phục (rollback) nếu quá trình cập nhật gặp lỗi, đảm bảo hệ thống luôn trong trạng thái có thể khởi động được.

6.4 Kết quả Triển khai Sản phẩm

Chương 7

Kết luận

7.1 Tổng kết Giai đoạn 1

7.2 Thách thức

1. **Làm chủ kiến trúc hệ thống phức tạp:** Việc thiết kế hệ thống phức tạp đòi hỏi sự nhất quán tuyệt đối giữa kiến trúc phần cứng và logic điều khiển firmware. Quy trình phát triển phải đảm bảo tính đồng bộ từ thiết kế hệ thống đến thiết kế phần cứng và firmware. Một sai sót nhỏ hệ thống cũng có thể dẫn đến việc phải tái thiết kế phần cứng, gây lãng phí thời gian và chi phí.
2. **Yêu cầu khắt khe về kỹ thuật thiết kế PCB nâng cao:** Việc triển khai các giao tiếp tốc độ cao như USB 2.0, SDIO và Quad SPI trên cấu trúc PCB 6 lớp đòi hỏi kỹ năng kiểm soát trở kháng đặc tính và đồng bộ trễ truyền dẫn cực kỳ chính xác. Bên cạnh đó, việc quản lý mạng phân phối nguồn (PDN) để đảm bảo sụt áp DC không vượt quá 5% và giải quyết các vấn đề về mật độ dòng điện là một thử thách lớn về cả tư duy layout lẫn kỹ năng mô phỏng kiểm chứng.
3. **Hạn chế về nguồn lực và hạ tầng sản xuất:** Về nhân lực, độ phức tạp của một IoT gateway đòi hỏi khối lượng công việc rất lớn cho cả phần cứng và firmware, tạo áp lực lớn khi triển khai với đội ngũ hạn chế. Về chi phí, mặc dù mục tiêu hướng tới giá thành sản phẩm thương mại thấp, nhưng chi phí thực tế cho việc nghiên cứu và phát triển vẫn khá cao, bao gồm chi phí linh kiện, gia công mạch và các thiết bị đo đạc kiểm chứng.

7.3 Hướng phát triển trong giai đoạn tiếp theo

Mặc dù sản phẩm trong giai đoạn 1 đã đạt được những kết quả khả quan và đáp ứng đầy đủ các mục tiêu thiết kế cốt lõi của một IoT Gateway dạng mô-đun, việc đưa thiết bị vào vận hành thực tế trong môi trường công nghiệp đòi hỏi những bước nâng cấp và tối ưu hóa sâu rộng hơn. Trong giai đoạn tiếp theo, sản phẩm cần được phân tích ưu nhược điểm của phiên bản hiện tại, hiệu chỉnh lại đặc tả chi tiết cho sản phẩm, và lên kế hoạch chi tiết cho quá trình hoàn thiện sản phẩm.

7.3.1 Hoàn thiện và khắc phục các hạn chế thiết kế

Dù hệ thống đã vận hành ổn định, quá trình thử nghiệm thực tế cho thấy vẫn tồn tại một số lỗi nhỏ về mặt logic phần cứng và bố trí linh kiện.

- **Tối ưu hóa thiết kế:** Sản phẩm hiện tại có thiết kế Module Adapter khá lớn, còn sử dụng các module MCU cắm rời. Trong giai đoạn tiếp theo sẽ tích hợp hoàn toàn lên Baseboard và tối ưu hóa kích thước PCB.
- **Cải thiện Layout PCB:** Tinh chỉnh lại các đường mạch tín hiệu nhạy cảm để giảm thiểu hiện tượng nhiễu xuyên âm (Crosstalk) và cải thiện tính toàn vẹn tín hiệu (Signal Integrity) khi hệ thống hoạt động ở tần số cao.

7.3.2 Nâng cấp và Mở rộng năng lực Phần cứng

Nhằm đưa sản phẩm từ một bản thử nghiệm thành một thiết bị ở quy mô công nghiệp. Sản phẩm prototype hiện tại chỉ đáp ứng được các chức năng cốt lõi của một gateway, song để sản phẩm thực sự hoàn thiện và có thể sử dụng ngoài thực tế thì vẫn còn nhiều chức năng cần phải được đánh giá và nâng cấp.

7.3.3 Phát triển Firmware chuyên sâu và tối ưu hóa hiệu suất

Hiện tại, phần mềm hệ thống trên cả hai vi điều khiển (Master và Slave) mới chỉ dừng lại ở mức độ nguyên mẫu (Proof of Concept) để chứng minh luồng dữ liệu cơ bản. Để sản phẩm đạt độ hoàn thiện cao và sẵn sàng cho triển khai thực tế, các nội dung sau cần được ưu tiên phát triển:

- **Hoàn thiện Driver và Stack giao thức cho mạng cảm biến (LAN Domain):**
 - Nâng cấp các giao thức hiện hữu (LoRa, Zigbee, RS485): Hiện tại, các module này mới chỉ được phát triển ở mức độ thử nghiệm (demo) gửi/nhận gói tin đơn giản (raw data). Cần triển khai đầy đủ các tầng của ngăn xếp giao thức (Protocol Stack) như cơ chế địa chỉ hóa (Addressing), quản lý mạng (Network Management), và kiểm tra lỗi gói tin (Packet Error Checking) để đảm bảo độ tin cậy.
 - Phát triển mới các giao thức chưa được hỗ trợ: Tiến hành lập trình driver và tích hợp các giao thức đã có trong thiết kế nhưng chưa được hiện thực hóa, bao gồm Z-Wave, Thread và Bluetooth Low Energy (BLE). Việc này đòi hỏi tích hợp các thư viện stack chuẩn và cấu hình tài nguyên phần cứng (Timer, Interrupt) phù hợp trên LAN-MCU.
- **Mở rộng khả năng kết nối mạng và giao thức ứng dụng (WAN Domain):**
 - Đa dạng hóa kênh kết nối Internet: Hiện tại hệ thống chưa hỗ trợ Ethernet và NB-IoT. Cần phát triển driver cho module Ethernet (như W5500 hoặc EMAC tích hợp) và module NB-IoT để tăng tính dự phòng đường truyền khi Wi-Fi hoặc LTE gặp sự cố.
 - Bổ sung giao thức tầng ứng dụng: Ngoài MQTT hiện đang hoạt động, cần phát triển thêm các client cho HTTP/HTTPS (phục vụ gọi REST API) và CoAP (Constrained Application Protocol) để tương thích với các nền tảng IoT khác nhau và tối ưu hóa băng thông cho các mạng di động.

- **Cải tiến kiến trúc luồng dữ liệu và độ tin cậy hệ thống:**

- Tối ưu hóa luồng Gửi/Nhận (TX/RX Pipeline): Thiết kế lại cơ chế đệm (Buffering) và quản lý hàng đợi (Queue management) giữa các tác vụ (Tasks) trong FreeRTOS. Chuyển đổi từ cơ chế xử lý tuần tự hoặc blocking sang cơ chế hướng sự kiện (Event-driven) hoặc sử dụng DMA (Direct Memory Access) để tăng thông lượng và giảm tải cho CPU.
- Xây dựng cơ chế xử lý lỗi (Error Handling) toàn diện: Hiện tại cơ chế xử lý lỗi còn sơ sài. Cần bổ sung các trình phục vụ ngắt và giám sát trạng thái (State Machine) để tự động phát hiện và khôi phục (Self-healing) khi xảy ra các lỗi như: mất kết nối module, treo bus SPI/I2C, hoặc tràn bộ nhớ đệm.

7.3.4 Tối ưu hóa chi phí và Hướng tới sản xuất

Tối ưu hóa chi phí thiết kế phần cứng bằng cách phân tích và lựa chọn các linh kiện tương đương có giá thành tốt hơn nhưng vẫn đảm bảo kỹ thuật, kích thước PCB thu nhỏ lại nhằm giảm chi phí gia công.

Chương 8

Phụ lục

8.1 Tài liệu tham khảo

Tài liệu tham khảo

- [1] Altium. Pcb manufacturing and impedance control: How to specify your requirements, 2024. Accessed: 2025-11-20.
- [2] Altium. What is high-speed design?, 2024. Accessed: 2025-11-20.
- [3] Gunjan Beniwal and Anita Singhrova. “a systematic literature review on iot gateways”. *Journal of King Saud University - Computer and Information Sciences*, 34(10, Part B):9541–9563, 2022.
- [4] Eric Bogatin. *Signal and Power Integrity - Simplified*. Prentice Hall PTR, Upper Saddle River, NJ, 2nd edition, 2010. Accessed: 2025-11-20.
- [5] Wilder Castellanos, Jose Macias, Harold Pinilla, and Jose David Alvarado. Internet of things: a multiprotocol gateway as solution of the interoperability problem. *arXiv preprint arXiv:2108.00098v1*, 2021.
- [6] Renesas Electronics. *DA14531 Ultra-Low Power BLE SoC*. Renesas Electronics Corporation, 2024.
- [7] Renesas Electronics. *RF Design Guidelines*. Renesas Electronics Corporation, 2024.
- [8] Espressif Systems. *ESP32-S3 Hardware Design Guidelines*. Espressif Systems, release master edition, December 2025.
- [9] André Glória, Francisco Cercas, and Nuno Souto. Design and implementation of an iot gateway to create smart environments. *Procedia Computer Science*, 109:568–575, 2017. 8th International Conference on Ambient Systems, Networks and Technologies, ANT-2017 and the 7th International Conference on Sustainable Energy Information Technology, SEIT 2017, 16-19 May 2017, Madeira, Portugal.
- [10] Pradyumna Gokhale, Omkar Bhat, and Sagar Bhat. Introduction to iot. *International Advanced Research Journal in Science, Engineering and Technology*, 7, 2024.
- [11] IEEE ComSoc Tech Blog. Lpwan standards and technologies. <https://techblog.comsoc.org/tag/lpwan-standard/>, 2024. Accessed: December 19, 2025.
- [12] Texas Instruments. *High-Speed Layout Guidelines*. Texas Instruments, Dallas, Texas, 11 2006. Accessed: 2025-11-20.
- [13] International Telecommunication Union. Multiple radio access technologies. Technical paper, ITU-T, June 2012. ITU-T SG13 Technical paper.

- [14] International Telecommunication Union. Recommendation itu-t y.4101/y.2067: Common requirements and capabilities of a gateway for internet of things applications. Technical report, ITU-T, October 2017.
- [15] JLCPCB. The importance of impedance control in pcb design, 2023. Accessed: 2025-11-20.
- [16] Byungseok Kang and Hyunseung Choo. An experimental study of a reliable iot gateway. *ICT Express*, 4(3):130–133, 2018.
- [17] Rabah Kenaza, Ameer Khemane, Hakim Bendjenna, Abdallah Meraoumia, and Lakhdar Laimeche. Internet of things (iot): Architecture, applications, and security challenges. In *2022 4th International Conference on Pattern Analysis and Intelligent Systems (PAIS)*. IEEE, 2022.
- [18] Radouan Ait Mouha. Internet of things (iot). *Journal of Digital and Image Processing*, 3(2), 2021.
- [19] Dialog Semiconductor. Da14530/531 hardware guidelines. Application Note B-075, Renesas Electronics Corporation, December 2022.
- [20] Dialog Semiconductor. Designing printed antennas for bluetooth low energy. Application Note B-027, Renesas Electronics Corporation, June 2024.
- [21] SIMCom Wireless Solutions. SIM7600 Series_PCIE_Hardware Design_V1.01. Technical Manual V1.01, SIMCom Wireless Solutions, 2020. Accessed: 2025-12-21.
- [22] Cadence Design Systems. High-speed pcb design guidelines, 2024. Accessed: 2025-11-20.
- [23] Brian C. Wadell. *Transmission Line Design Handbook*. Artech House, Boston, 1991.

8.2 Mã nguồn/Tài liệu thiết kế

- Tài liệu thiết kế Phần cứng:
- Mã nguồn Firmware: